

# MBRL Lecture 1. Learning Dynamic Systems

Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

Planning

Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data

Data-Driven  
Dynamic  
Model

Planning

Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data

Inductive  
Biases on  
Dynamics

Planning

Policy

Model-Free RL

Data

Dynamic  
Model

Policy

Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model



Planning



Policy

Linear Dynamic Systems and Control

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data



Data-Driven  
Dynamic  
Model



Planning



Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data



Inductive  
Biases on  
Dynamics



Planning



Policy

Model-Free RL

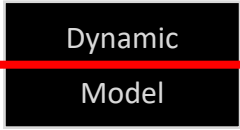
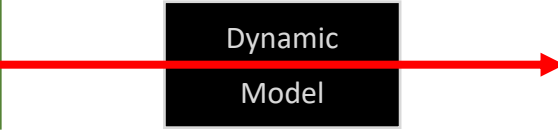
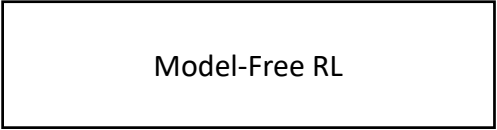
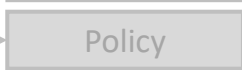
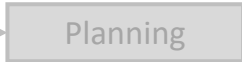
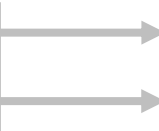
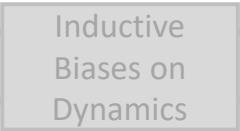
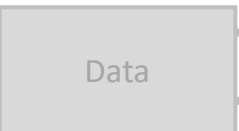
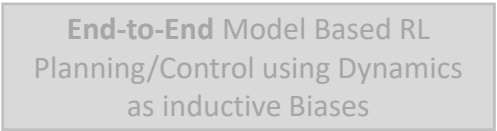
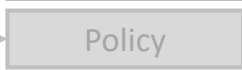
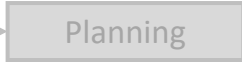
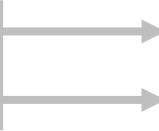
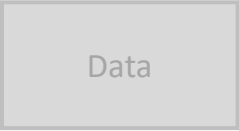
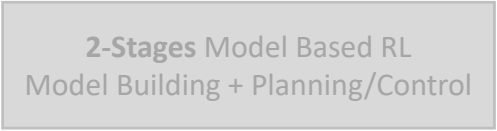
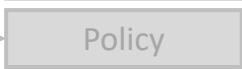
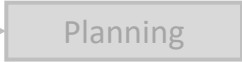
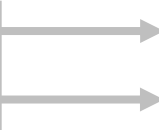
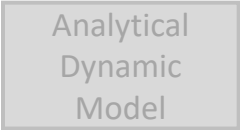
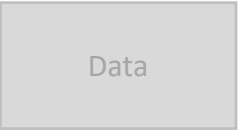
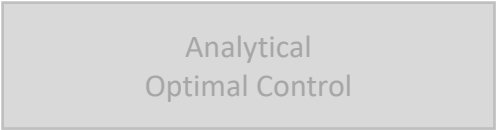
Data



Dynamic  
Model



Policy



Model Free Reinforcement Learning



Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

Planning  
Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data

Data-Driven  
Dynamic  
Model

Planning  
Policy

**MBRL Lecture 1**

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data

Inductive  
Biases on  
Dynamics

Planning  
Policy

Model-Free RL

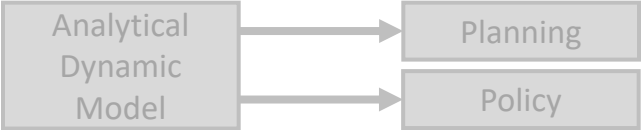
Data

Dynamic  
Model

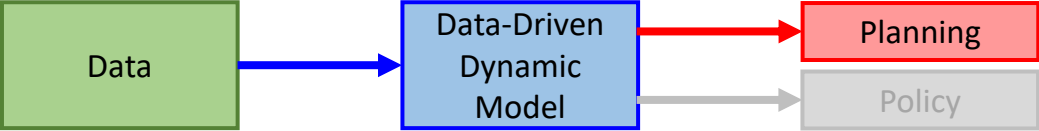
Policy

Analytical  
Optimal Control

Data

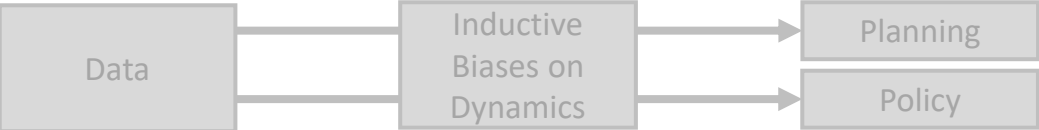


**2-Stages** Model Based RL  
Model Building + Planning/Control

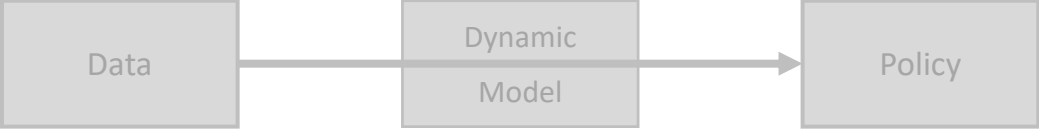


**MBRL Lecture 2**

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases



Model-Free RL



Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

Planning  
Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data

Data-Driven  
Dynamic  
Model

Planning  
Policy

**MBRL Lecture 3**

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data

Inductive  
Biases on  
Dynamics

Planning  
Policy

Model-Free RL

Data

Dynamic  
Model

Policy

Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

Planning

Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data

Data-Driven  
Dynamic  
Model

Planning

Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data

Inductive  
Biases on  
Dynamics

Planning

Policy

Model-Free RL

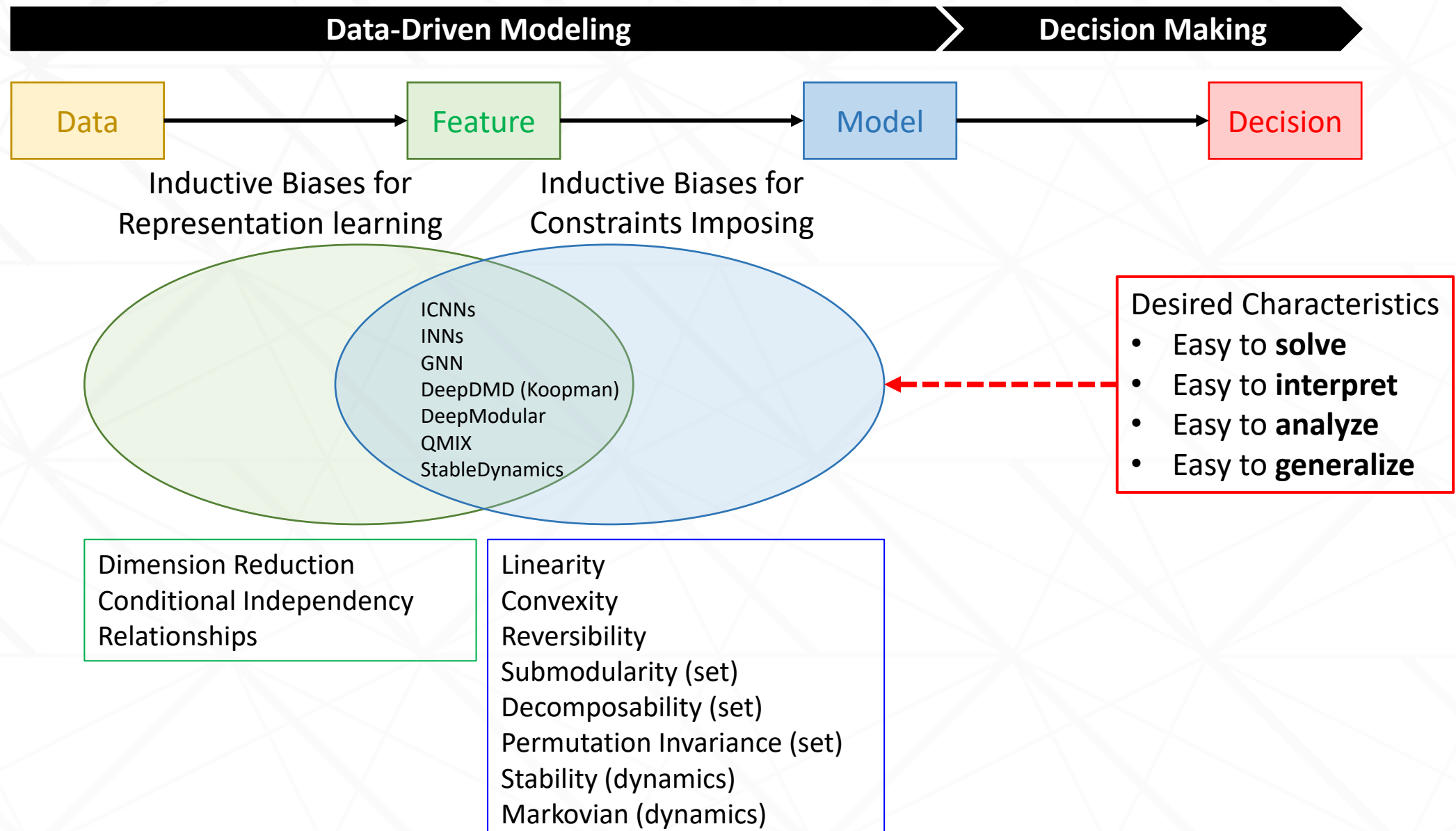
Data

Dynamic  
Model

Policy

**MBRL Lecture 4**

# Inductive Biases for Mathematical Properties



## MBRL Lecture 2. Dynamic Model Fitting and Planning

# Motivation: What if we knew the transition dynamics?

- Recap: the reinforcement learning objective

$$\theta^* = \operatorname{argmax}_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[ \sum_t r(s_t, a_t) \right]$$

$$\frac{p_{\theta}(s_1, a_1, \dots, s_T, a_T)}{\pi_{\theta}(\tau)} = p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

# Motivation: What if we knew the transition dynamics?

- Recap: the reinforcement learning objective

$$\theta^* = \operatorname{argmax}_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[ \sum_t r(s_t, a_t) \right]$$

$$\frac{p_{\theta}(s_1, a_1, \dots, s_T, a_T)}{\pi_{\theta}(\tau)} = p(s_1) \prod_{t=1}^T \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

- What if we know the dynamics of the environment?
  - Can we utilize this knowledge to optimize the optimum control policy?



# Motivation: What if we knew the transition dynamics?

- Often we do know the dynamics
    - Games (e.g., Atari games, chess, Go)
    - Easily modeled systems (e.g., navigating a car)
    - Simulated Environments (e.g., simulated robots, video games)
  - Often we can learn the dynamics
    - System identification – fit unknown parameters of a known model
    - Learning – fit a general purpose model to observed transition data
- Knowing the dynamics helpful for (1) **planning a sequence of actions** and (2) **deriving a policy**

Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

Planning

Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data

Data-Driven  
Dynamic  
Model

Planning

Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data

Inductive  
Biases on  
Dynamics

Planning

Policy

Model-Free RL

Data

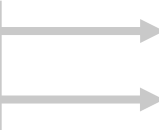
Dynamic  
Model

Policy

Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

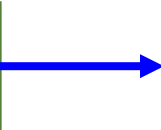


Planning

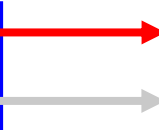
Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data



Data-Driven  
Dynamic  
Model



Planning

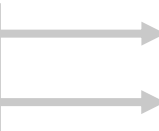
Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data



Inductive  
Biases on  
Dynamics



Planning

Policy

Model-Free RL

Data



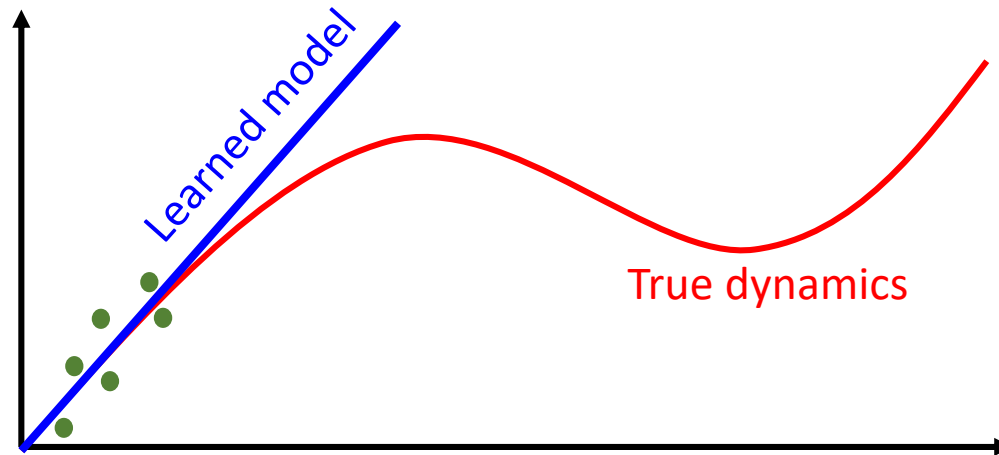
Dynamic  
Model



Policy

# Motivation

- Naïve approach:
  - Learn a model describing the dynamic of a target system
  - Use the model for planning or control
- Naïve approach works for only exceptionally simple cases
  - It works only when the constructed model generalized well over the entire state space



- The **effect of distributional shift** makes the Naive approach unsuccessful in many cases.

# Overview

Model Based RL (2 steps approach for planning)

- Version 1: Model Building + Open Loop Planning
- Version 2 : Iterative Model Building + Open Loop Planning
- Version 3 : Iterative Model Building + MPC (+ Model Uncertainty)
- Version 4 : Iterative Model Building + MPC + Latent Space Model

## Paper to Read:

- ✓ Paper: “PILCO: A Model-Based and Data-Efficient Approach to Policy Search”
- ✓ Paper: “Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning”
- ✓ Paper: “Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models”
- ✓ Paper: “Optimal Control Via Neural Networks: A Convex Approach”
- ✓ Paper: “Learning Compositional Koopman Operators for Model-Based Control”

# Version 1: Model Building + Open Loop Planning

## Version 1: Model Building + Open Loop Planning

1. Run base policy  $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$  (e.g., random policy) to collect  $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$  (Experiencing)
2. Learn dynamics model  $f(\mathbf{s}, \mathbf{a})$  to minimize  $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$  (Model Fitting)
3. Plan through  $f(\mathbf{s}, \mathbf{a})$  to choose actions (Model Based Planning)
4. Execute those actions

Does it work?

# Version 1: Model Building + Open Loop Planning

## Version 1: Model Building + Open Loop Planning

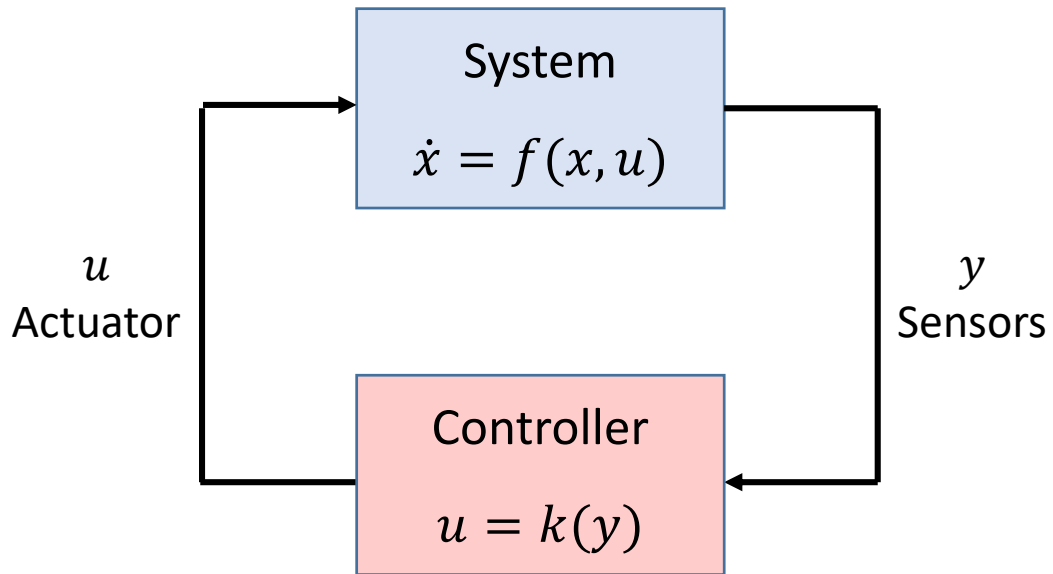
1. Run base policy  $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$  (e.g., random policy) to collect  $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$  (Experiencing)
2. Learn dynamics model  $f(\mathbf{s}, \mathbf{a})$  to minimize  $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$  (Model Fitting)
3. Plan through  $f(\mathbf{s}, \mathbf{a})$  to choose actions (Model Based Planning)
4. Execute those actions

Does it work?

Yes:

- Classical system identification method use this approach
  - System identification works as (1) excite a system and measure the response of the system and (2) fit a **predefined model** to the measured input and output data
- Particularly effective if we can hand-engineer a dynamics representation using our **knowledge of physics**, and fit just a few parameters
  - Generalizable knowledge is required.
  - If we fit the Newton's law, it can be applied to describe any motion of a rigid body.

# Version 1: Model Building + Open Loop Planning



## Data-Driven System Identification

### Challenges:

- Nonlinear
- Unknown Dynamics
- High Dimensional
  - ✓ Neuroscience
  - ✓ Climate
  - ✓ Epidemics
- Limited Measurements

➤ Machine learning can resolve some of these issues



# Version 1: Model Building + Open Loop Planning

## (Classical System Identification)

- Learn a model  $f(x, u)$  that generalize the system's behavior over the new state and action
- Linear system identification seeks to estimate  $A$  and  $B$

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$$

- Once the linear system is identified, control theory can be used to control the system
- Model can be reduced to capture the most dominant important behavior (model reduction)

$$\dot{\mathbf{a}} = \tilde{A}\mathbf{a} + \tilde{B}\mathbf{u}$$

- ✓ Reduced model with lower dimension state allows fast prediction and fast control
- ✓ **Dynamic mode decomposition (DMD)**
- Nonlinear dynamic system identification
$$\dot{\mathbf{x}} = f(x, u)$$
  - Piecewise linear model
  - **Koopman operator theory:** Nonlinear system can be expressed as a linear system with the infinite dim. Measurements
- Modern system identification based on machine learning
  - Deep learning (MLP, RNN, LSTM)
  - Gaussian Process

# Version 1: Model Building + Open Loop Planning

(Classical System Identification)



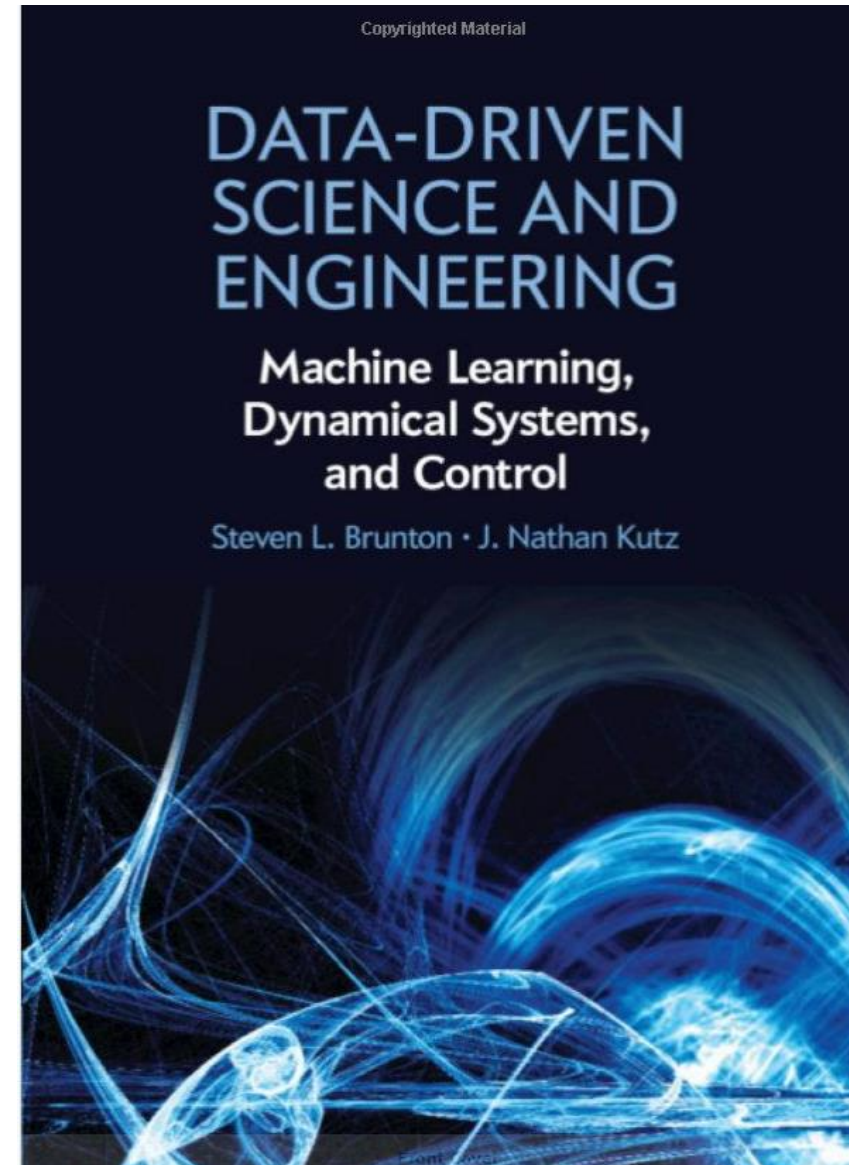
## Data-Driven Control with Machine Learning

40 videos • 106,394 views • Last updated on May 13, 2020



Many tasks in control theory are hard, non-convex optimization problems. Enter machine learning, a growing set of powerful optimization techniques based on a wealth of data.

This lecture series will go into depth on data-driven control, following Chapters 9 & 10 from: "Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control" by Brunton and Kutz



# Version 1: Model Building + Open Loop Planning

## Version 1: Model Building + Open Loop Planning

1. Run base policy  $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$  (e.g., random policy) to collect  $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$  (Experiencing)
2. Learn dynamics model  $f(\mathbf{s}, \mathbf{a})$  to minimize  $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$  (Model Fitting)
3. Plan through  $f(\mathbf{s}, \mathbf{a})$  to choose actions (Model Based Planning)
4. Execute those actions

Does it work?

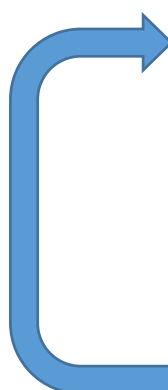
No:

$$p_{\pi_f}(\mathbf{s}_t) \neq p_{\pi_0}(\mathbf{s}_t)$$

- Distribution mismatch problem prevent us from determining the optimum action.
  - We can determine the optimum action using the learned model, but the determined optimum action is only optimum in the trained model perspective.
- Distribution mismatch problem becomes exacerbated as we use more expressive model classes

# Version 2: Iterative Model Building + Open Loop Planning

## Version 2: Iterative Model Building + Open Loop Planning

1. Run base policy  $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$  (e.g., random policy) to collect  $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$  (Experiencing)
  2. Learn dynamics model  $f(\mathbf{s}, \mathbf{a})$  to minimize  $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$  (Model Fitting)
  3. Plan through  $f(\mathbf{s}, \mathbf{a})$  to choose actions (Model Based Planning)
  4. Execute those actions
  5. Append  $\{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_j\}$  to dataset  $\mathcal{D}$  (Collecting Data)
- 

Can we make  $p_{\pi_f}(\mathbf{s}_t) = p_{\pi_0}(\mathbf{s}_t)$

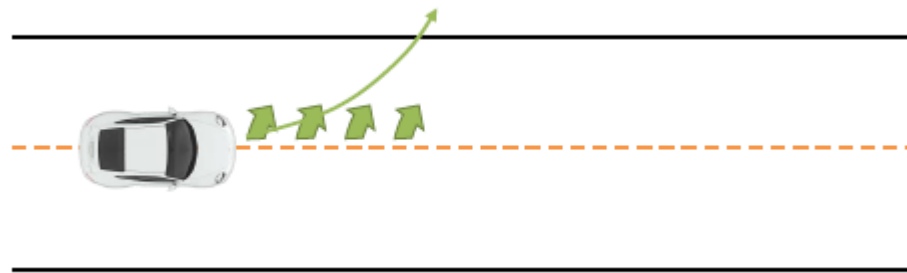
We need to collect data from  $p_{\pi_f}(\mathbf{s}_t)$  that describing how the system actually moving

# Version 2: Iterative Model Building + Open Loop Planning

## Version 2: Iterative Model Building + Open Loop Planning

1. Run base policy  $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$  (e.g., random policy) to collect  $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$  (Experiencing)
2. Learn dynamics model  $f(\mathbf{s}, \mathbf{a})$  to minimize  $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$  (Model Fitting)
3. Plan through  $f(\mathbf{s}, \mathbf{a})$  to choose actions (Model Based Planning)
4. Execute those actions
5. Append  $\{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_j\}$  to dataset  $\mathcal{D}$  (Collecting Data)

What if we make a mistake?



# Version 3: Iterative Model Building + MPC

## Version 3: Iterative Model Building + MPC

1. Run base policy  $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$  (e.g., random policy) to collect  $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$  (Experiencing)
2. Learn dynamics model  $f(\mathbf{s}, \mathbf{a})$  to minimize  $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$  (Model Fitting)
3. Plan through  $f(\mathbf{s}, \mathbf{a})$  to choose actions (Model Based Planning)
4. Execute the first planned action, observe resulting  $\mathbf{s}'$  (MPC)
5. Append  $\{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_j\}$  to dataset  $\mathcal{D}$  (Collecting Data)

Every N steps



- The more you replan, the less perfect each individual plan needs to be
- Can use shorter horizons
- Even random sampling can often work well here!



## Neural Network Dynamics for Model-Based Deep Reinforcement Learning with Model-Free Fine-Tuning

Anusha Nagabandi, Gregory Kahn, Ronald S. Fearing, Sergey Levine  
University of California, Berkeley

*Abstract*—Model-free deep reinforcement learning algorithms have been shown to be capable of learning a wide range of robotic skills, but typically require a very large number of samples to achieve good performance. Model-based algorithms, in principle, can provide for much more efficient learning, but have proven difficult to extend to expressive, high-capacity models such as deep neural networks. In this work, we demonstrate that medium-sized neural network models can in fact be combined with model predictive control (MPC) to achieve excellent sample complexity in a model-based reinforcement learning algorithm, producing stable and plausible gaits to accomplish various complex locomotion tasks. We also propose using deep neural network dynamics models to initialize a model-free learner, in order to combine the sample efficiency of model-based approaches with the high task-specific performance of model-free methods. We empirically demonstrate on MuJoCo locomotion tasks that our pure model-based approach trained on just random action data can follow arbitrary trajectories with excellent sample efficiency, and that our hybrid algorithm can accelerate model-free learning on high-speed benchmark tasks, achieving sample efficiency gains of  $3-5\times$  on swimmer, cheetah, hopper, and ant agents. Videos can be found at <https://sites.google.com/view/mbmf>

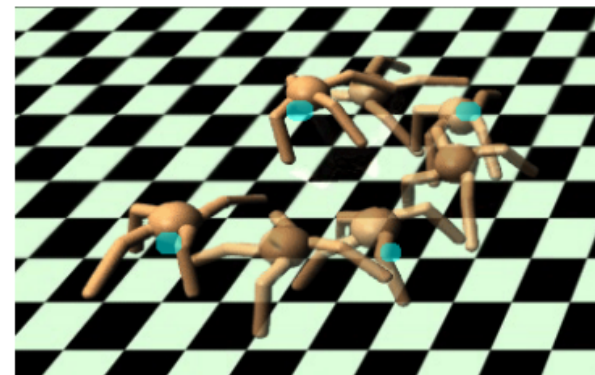


Fig. 1: Our method can learn a model that enables a simulated quadrupedal robot to autonomously discover a walking gait that follows user-defined waypoints at test time. Training for this task used  $7e5$  time steps, collected without any knowledge of the test-time navigation task.

combined with a few important design decisions such as data aggregation. The resulting models can then be used for model-based control, which we perform using model predictive control (MPC) with a simple random-sampling shooting method [11]. We demonstrate that this method can acquire effective locomotion gaits for a variety of MuJoCo

# Version 3: Iterative Model Building + MPC

## Algorithm 1 Model-based Reinforcement Learning

- 1: gather dataset  $\mathcal{D}_{\text{RAND}}$  of random trajectories
- 2: initialize empty dataset  $\mathcal{D}_{\text{RL}}$ , and randomly initialize  $\hat{f}_\theta$
- 3: **for** iter=1 **to** max\_iter **do**
- 4:   train  $\hat{f}_\theta(\mathbf{s}, \mathbf{a})$  by performing gradient descent on Eqn. 2, using  $\mathcal{D}_{\text{RAND}}$  and  $\mathcal{D}_{\text{RL}}$
- 5:   **for**  $t = 1$  **to**  $T$  **do**
- 6:     get agent's current state  $\mathbf{s}_t$
- 7:     use  $\hat{f}_\theta$  to estimate optimal action sequence  $\mathbf{A}_t^{(H)}$  (Eqn. 4)
- 8:     execute first action  $\mathbf{a}_t$  from selected action sequence  $\mathbf{A}_t^{(H)}$
- 9:     add  $(\mathbf{s}_t, \mathbf{a}_t)$  to  $\mathcal{D}_{\text{RL}}$
- 10:   **end for**
- 11: **end for**

**Training the model:** We train the dynamics model  $\hat{f}_\theta(\mathbf{s}_t, \mathbf{a}_t)$  by minimizing the error

$$\mathcal{E}(\theta) = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1}) \in \mathcal{D}} \frac{1}{2} \|\mathbf{s}_{t+1} - \mathbf{s}_t - \hat{f}_\theta(\mathbf{s}_t, \mathbf{a}_t)\|^2 \quad (2)$$

$$\mathbf{A}_t^{(H)} = \arg \max_{\mathbf{A}_t^{(H)}} \sum_{t'=t}^{t+H-1} r(\hat{\mathbf{s}}_{t'}, \mathbf{a}_{t'}) \quad :$$
$$\hat{\mathbf{s}}_t = \mathbf{s}_t, \hat{\mathbf{s}}_{t'+1} = \hat{\mathbf{s}}_{t'} + \hat{f}_\theta(\hat{\mathbf{s}}_{t'}, \mathbf{a}_{t'}). \quad (4)$$

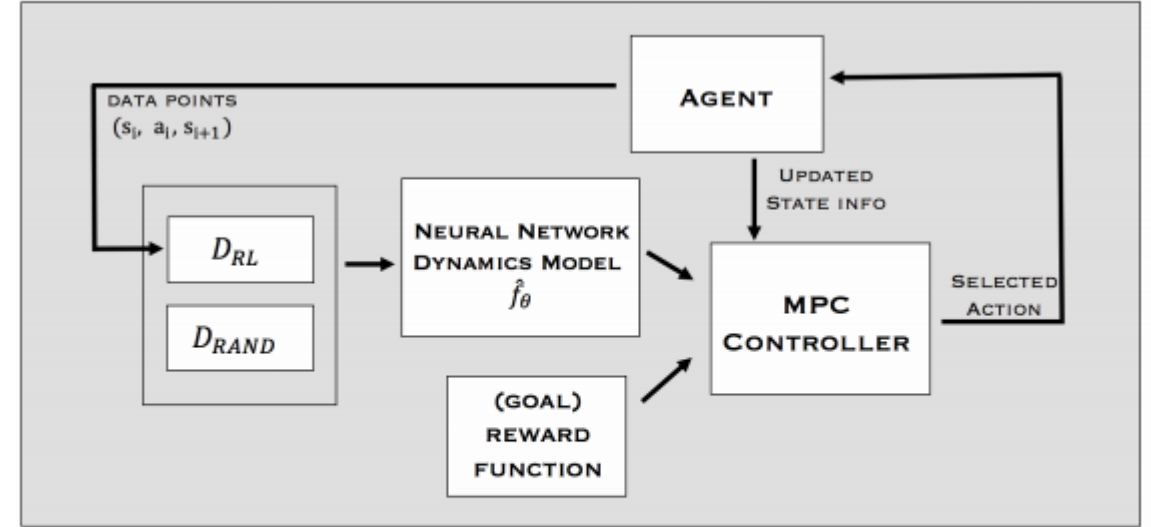
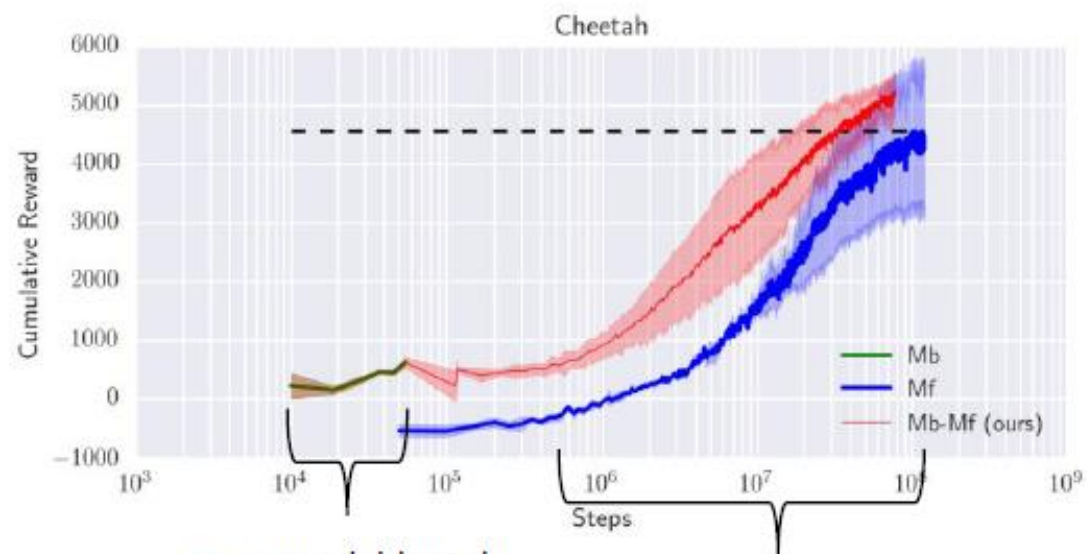


Fig. 2: Illustration of Algorithm 1. On the first iteration, random actions are performed and used to initialize  $\mathcal{D}_{\text{RAND}}$ . On all following iterations, this iterative procedure is used to train the dynamics model, run the MPC controller for action selection, aggregate data, and retrain the model.

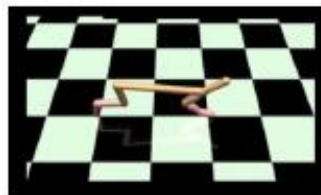


# Version 3: Iterative Model Building + MPC



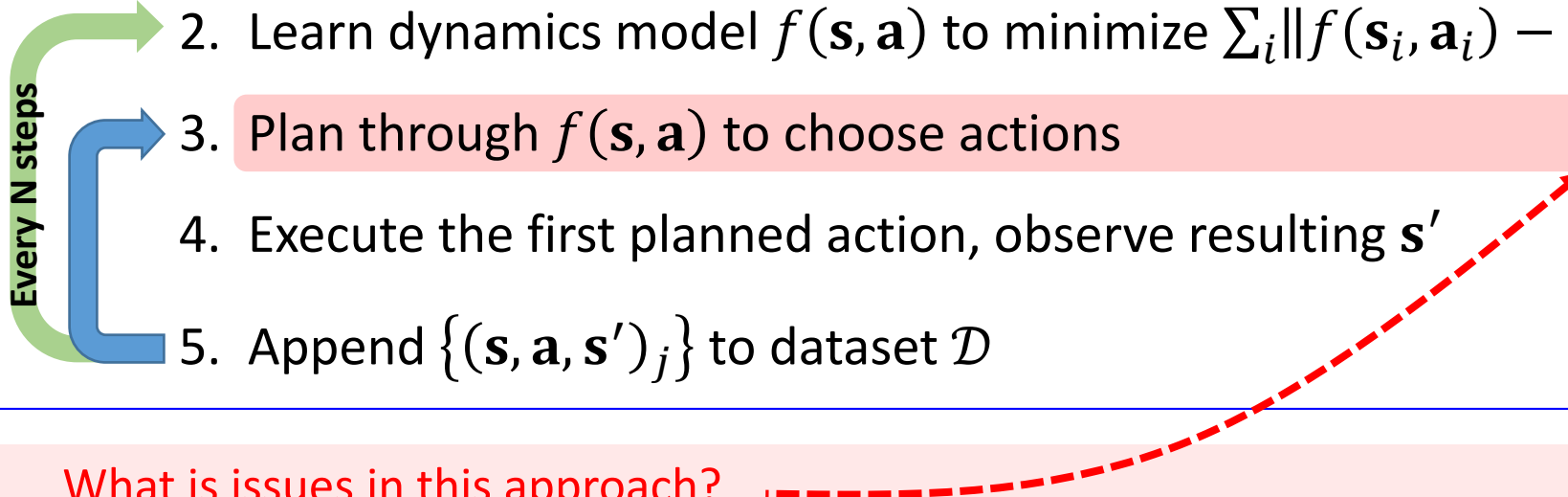
pure model-based  
(about 10 minutes real time)

model-free training  
(about 10 days...)



# Version 3: Iterative Model Building + MPC

## Version 3: Iterative Model Building + MPC

1. Run base policy  $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$  (e.g., random policy) to collect  $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$  (Experiencing)
  2. Learn dynamics model  $f(\mathbf{s}, \mathbf{a})$  to minimize  $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$  (Model Fitting)
  3. Plan through  $f(\mathbf{s}, \mathbf{a})$  to choose actions (Model Based Planning)
  4. Execute the first planned action, observe resulting  $\mathbf{s}'$  (MPC)
  5. Append  $\{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_j\}$  to dataset  $\mathcal{D}$  (Collecting Data)
- 

What is issues in this approach?

- Only take actions that are believed to give high reward in expectation (w.r.t. uncertain dynamics)
- This avoids “exploring” the world, depriving the possibility to choose a better action for higher reward

We need to explore the model

- To this end, we need to take into account the **model uncertainty**

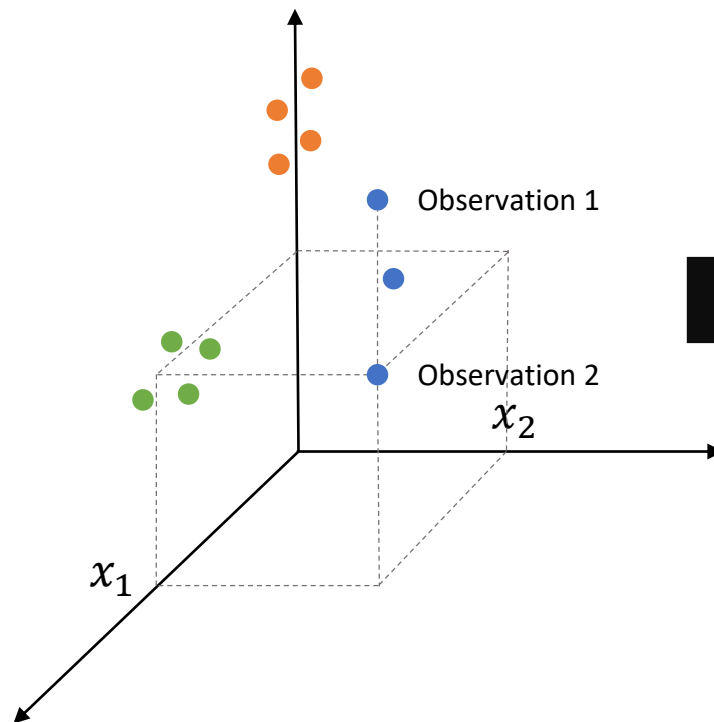
# Version 3: Iterative Model Building + MPC + Model Uncertainty

How can we estimate the uncertainty in the dynamics and utilize them to make an action?

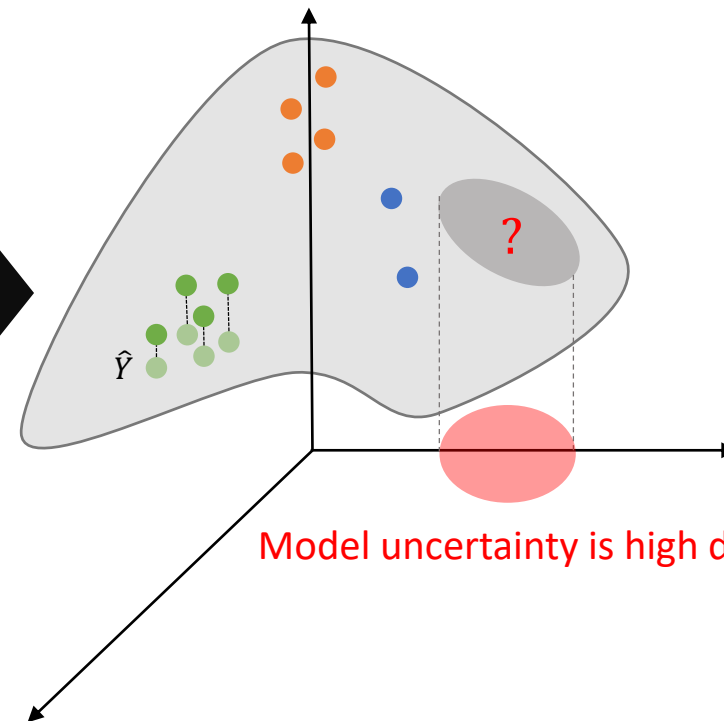
Nature :  $Y = f(X) + \varepsilon, \varepsilon \sim N(0, \sigma_s^2)$

Model :  $\hat{Y} = \hat{f}(X)$  (True 함수  $f(X)$  의 근사함수)

$$\begin{aligned} E\|Y - \hat{f}(X)\|_2^2 &= E\|Y - f(X) + f(X) - \hat{f}(X)\|_2^2 = E\|Y - f(X)\|_2^2 + E\|f(X) - \hat{f}(X)\|_2^2 \\ &= \sigma_s^2 + \sigma_m^2 \\ &= \text{systemic noise} + \text{model uncertainty} \end{aligned}$$



Training



Model uncertainty is high due to the lack of training data

# Version 3: Iterative Model Building + MPC + Model Uncertainty

How can we estimate the uncertainty in the dynamics and utilize them to make an action?

---

## Weight Uncertainty in Neural Networks

---

Charles Blundell  
Julien Cornebise  
Koray Kavukcuoglu  
Daan Wierstra  
Google DeepMind

CBLUNDELL@GOOGLE.COM  
JUCOR@GOOGLE.COM  
KORAYK@GOOGLE.COM  
WIERSTRA@GOOGLE.COM

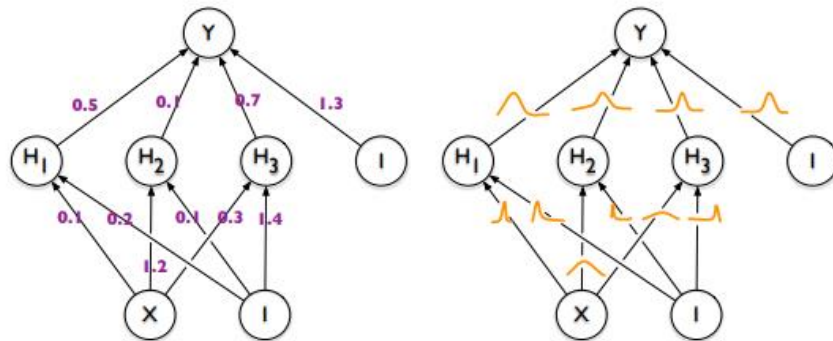


Figure 1. Left: each weight has a fixed value, as provided by classical backpropagation. Right: each weight is assigned a distribution, as provided by Bayes by Backprop.

### Point estimation

$$\begin{aligned}\mathbf{w}^{\text{MLE}} &= \arg \max_{\mathbf{w}} \log P(\mathcal{D}|\mathbf{w}) \\ &= \arg \max_{\mathbf{w}} \sum_i \log P(\mathbf{y}_i|\mathbf{x}_i, \mathbf{w}).\end{aligned}$$
$$\begin{aligned}\mathbf{w}^{\text{MAP}} &= \arg \max_{\mathbf{w}} \log P(\mathbf{w}|\mathcal{D}) \\ &= \arg \max_{\mathbf{w}} \log P(\mathcal{D}|\mathbf{w}) + \log P(\mathbf{w}).\end{aligned}$$

### Full Bayesian prediction

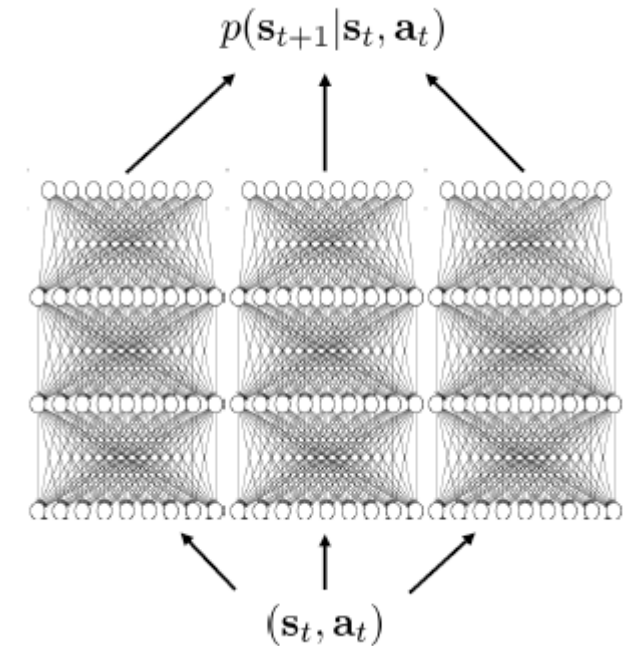
$$P(\hat{\mathbf{y}}|\hat{\mathbf{x}}) = \mathbb{E}_{P(\mathbf{w}|\mathcal{D})}[\check{P}(\hat{\mathbf{y}}|\hat{\mathbf{x}}, \mathbf{w})].$$

# Version 3: Iterative Model Building + MPC + Model Uncertainty

How can we estimate the uncertainty in the dynamics and utilize them to make an action?

- Train multiple models and use them to estimate the uncertainty in the model

$$p(\theta|\mathcal{D}) \approx \frac{1}{N} \sum_i \delta(\theta_i)$$
$$\int p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta) p(\theta|\mathcal{D}) d\theta \approx \frac{1}{N} \sum_i p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta_i)$$



- How to train?
  - need to generate “independent” datasets to get “independent” models
  - $\theta_i$  is trained on  $D_i$ , sampled with replacement from  $D$

# Version 3: Iterative Model Building + MPC + Model Uncertainty

How can we estimate the uncertainty in the dynamics and utilize them to make an action?

## Gaussian Process Non-linear Auto Regressive Model

- The effectiveness of MPC crucially dependent on state dynamic model (or state trajectory prediction model)

$$x_{t+1} = f_t(x_{0:t}, u_{0:t}, \hat{w}_{0:t})$$

- Most engineering approaches assume a transition model are

- ✓ Markovian
- ✓ Stationary
- ✓ Linear
- ✓ Additive Noise
- ✓ Noise is Gaussian

$$x_{t+1} = Ax_t + Bu_t + w_t \text{ with } w_t \sim N(0, \sigma_w^2) \text{ or } x_{t+1} \sim N(Ax_t + Bu_t + C, \sigma_w^2)$$

- Gaussian Process Non-linear Auto Regressive (GPNAR) Model is used to model flexible transition model which can be

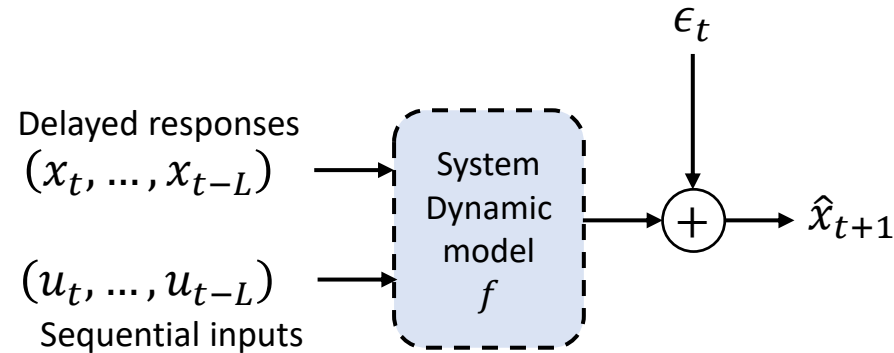
- ✓ Non-Markovian
- ✓ Non-Stationary
- ✓ Non-linear
- ✓ Additive Noise
- ✓ Noise is Gaussian

$$x_{t+1} = f_t(x_{0:t}, u_{0:t}, \hat{w}_{0:t})$$

# Version 3: Iterative Model Building + MPC + Model Uncertainty

How can we estimate the uncertainty in the dynamics and utilize them to make an action?

## Gaussian Process Non-linear Auto Regressive Model



- The next state  $\hat{y}_t$  at time step  $k$  depends on the delayed states  $(x_t, \dots, x_{t-L})$  and the exogenous control inputs  $(u_t, \dots, u_{t-L})$  as:

$$\hat{x}_{t+1} = f(x_{t-L:t}, u_{t-L:t}) + \epsilon_t \text{ with } \epsilon_t \sim N(0, \sigma_\epsilon^2)$$

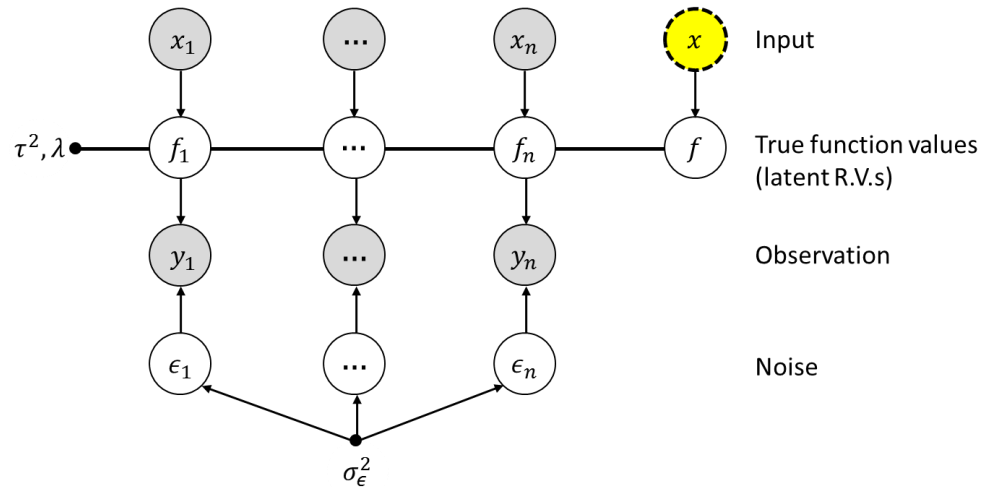
- We use Gaussian Process regression to model the transition model  $f(x_t)$ 
  - ✓ **Non-parametric model**: it can model complex input and output relationship
    - Used for non-linear transition model
  - ✓ **Bayesian approach**: It can update the model with most recent data
    - Used for non-stationary transition model
  - ✓ **Probabilistic approach**: it can predict the distribution of the target output
    - Used for robust decision making

# Version 3: Iterative Model Building + MPC + Model Uncertainty

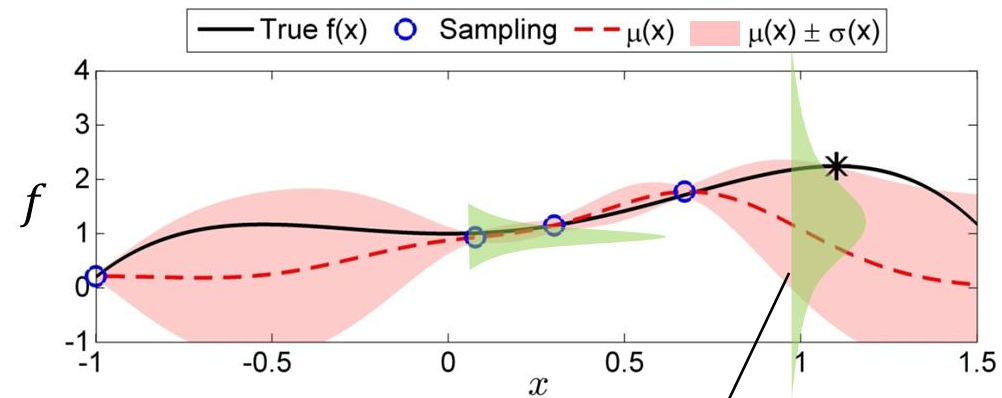
How can we estimate the uncertainty in the dynamics and utilize them to make an action?

## Gaussian Process Non-linear Auto Regressive Model

- It is empirical hierarchical Bayesian Model



- Inputs  $x_{1:n} = \{x_1, \dots, x_n\}$
- True function values  $f_{1:n} = \{f_1, \dots, f_n\}$
- Noisy observations  $y_{1:n} = \{y_1, \dots, y_n\}$
- $\theta = (\tau^2, \lambda, \sigma_\epsilon^2)$  is the hyper parameters



$$p(f|D_n, \theta, x)$$

Prior on true function values: 
$$p(f^{1:n}, f|z^{1:n}, z, \tau, \lambda) = N\left(\begin{bmatrix} f^{1:n} \\ f \end{bmatrix} \middle| 0, \begin{bmatrix} K & k \\ k^T & k(z, z) \end{bmatrix}\right)$$

Likelihood: 
$$p(y^{1:n}|f^{1:n}, \sigma_\epsilon) = N(y^{1:n}|f^{1:n}, \sigma_\epsilon^2 I)$$

Posterior: 
$$p(f|D_n, \theta, x)$$

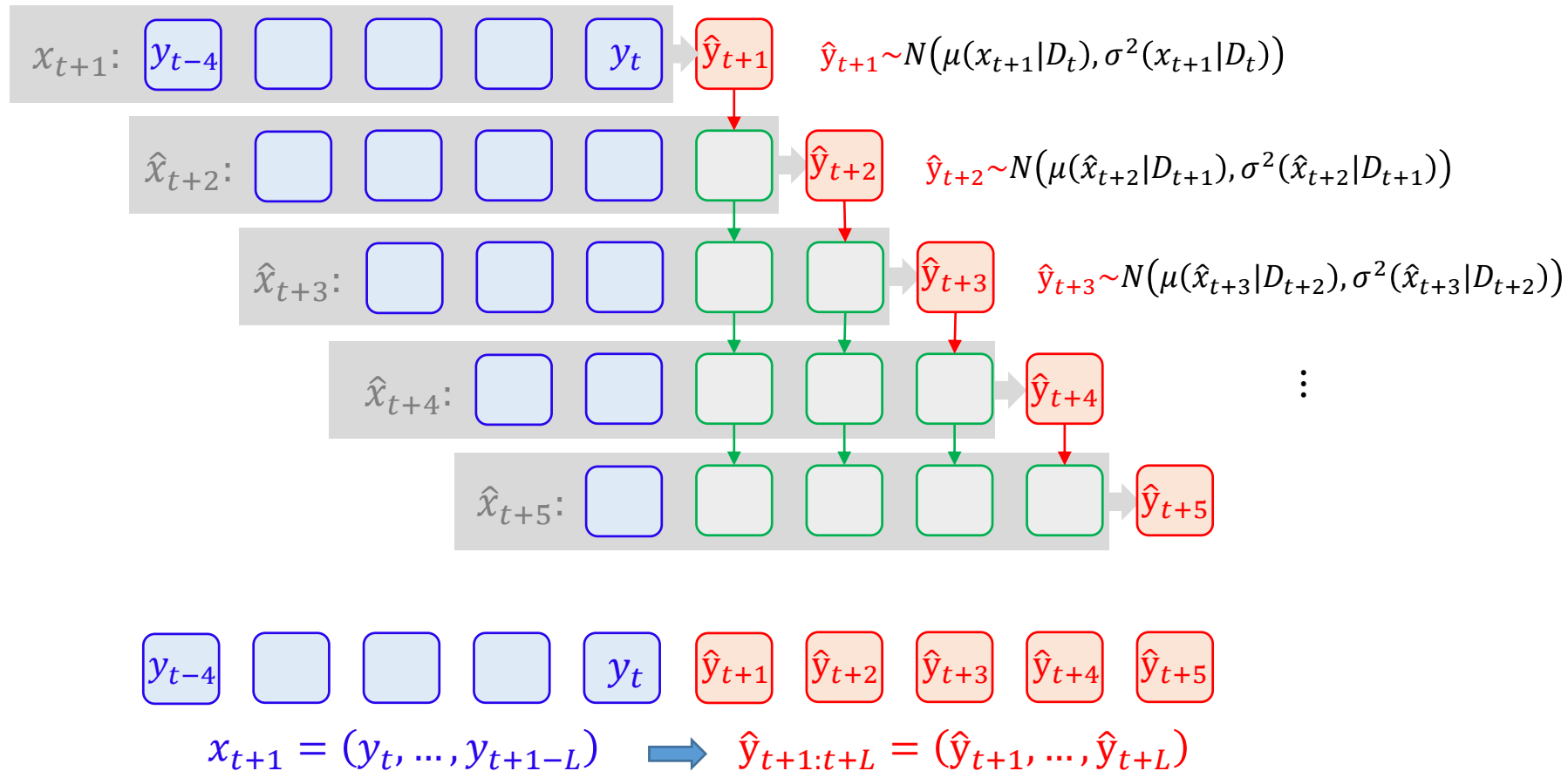


# Version 3: Iterative Model Building + MPC + Model Uncertainty

How can we estimate the uncertainty in the dynamics and utilize them to make an action?

## Gaussian Process Non-linear Auto Regressive Model

- Given the historical response data  $(y_t, \dots, y_{t+1-L})$  one can predict the trajectory of the output  $(\hat{y}_{t+1}, \dots, \hat{y}_{t+k})$  by iteratively applying an one-step ahead prediction model  $\hat{y}_{t+1} = f(y_t, \dots, y_{t+1-L}) + \epsilon_{t+1}$  as:



# Version 3: Iterative Model Building + MPC + Model Uncertainty

How can we estimate the uncertainty in the dynamics and utilize them to make an action?

## Gaussian Process Non-linear Auto Regressive Model

- Given the historical response data  $(y_t, \dots, y_{t+1-L})$  one can predict the trajectory of the output  $(\hat{y}_{t+1}, \dots, \hat{y}_{t+k})$  by iteratively applying an one-step ahead prediction model  $\hat{y}_{t+1} = f(y_t, \dots, y_{t+1-L}) + \epsilon_{t+1}$  as:

$$\hat{y}_{t+1} = f(y_t, \dots, y_{t+1-L}) + e_{t+1}$$

$$\hat{y}_{t+2} = f(\hat{y}_{t+1}, y_t, \dots, y_{t+2-L}) + e_{t+2}$$

$$\hat{y}_{t+3} = f(\hat{y}_{t+2}, \hat{y}_{t+1}, y_t, \dots, y_{t+3-L}) + e_{t+3}$$

$\vdots$

$$\hat{y}_{t+k} = f(\hat{y}_{t+k-1}, \dots, \hat{y}_{t+1}, y_t, \dots, y_{t+k-L}) + e_{t+k}$$

$$\hat{y}_{t+k} = f(\hat{x}_{t+k}) + e_{t+k} \quad \text{with} \quad \hat{x}_{t+k} = (\underbrace{\hat{y}_{t+k-1}, \dots, \hat{y}_{t+1}}_{\text{R.V.s}}, \underbrace{y_t, \dots, y_{t+k-L}}_{\text{Data}})$$

R.V.s

Data

# Version 3: Iterative Model Building + MPC + Model Uncertainty

How can we estimate the uncertainty in the dynamics and utilize them to make an action?

## Gaussian Process Non-linear Auto Regressive Model

- If we assume that the state input  $x$  follow Gaussian distribution,  $x \sim p(x) = N(\mu_x, \Sigma_x)$ , the predictive distribution on  $f(x)$  can be expressed as

$$p(f(x)|\mu_x, \Sigma_x) = \int p(f(x)|x, D) p(x) dx$$

- ✓ where  $p(f(x)|x, D) = N(\mu(x|D), \sigma^2(x|D))$

Given  $x$ ,  $p(f(x)|x, D)$  is GPR

$$\text{With } \begin{cases} \mu(x|D_n) = k^T (K + \sigma_\epsilon^2 I)^{-1} y^{1:n} \\ \sigma^2(x|D_n) = k(x, x) - k^T (K + \sigma_\epsilon^2 I)^{-1} k \end{cases}$$

- Because the integral is intractable, we perform numerical approximation for  $p(f(x)|\mu_x, \Sigma_x)$  by using simple Monte-Carlo approach:

$$p(f(x)|\mu_x, \Sigma_x) = \int p(f(x)|x, D) p(x) dx \approx \frac{1}{T} \sum_{t=1}^T p(f(x^t)|x^t, D)$$

- ✓ with  $x^t$  for  $t = 1, \dots, T$  are independently sampled from  $p(x)$

# Version 3: Iterative Model Building + MPC + Model Uncertainty

How can we estimate the uncertainty in the dynamics and utilize them to make an action?

## Gaussian Process Non-linear Auto Regressive Model

- We generate a large number of trajectories and Compute the mean trajectory and its the standard deviation

For  $i=1:\text{NumSim}$

$$\begin{aligned}\hat{y}_{t+1}^{(i)} &= f(y_t, \dots, y_{t+1-L}) + e_{t+1} \\ \hat{y}_{t+2}^{(i)} &= f(\hat{y}_{t+1}^{(i)}, y_t, \dots, y_{t+2-L}) + e_{t+2} \\ \hat{y}_{t+3}^{(i)} &= f(\hat{y}_{t+2}^{(i)}, \hat{y}_{t+1}^{(i)}, y_t, \dots, y_{t+3-L}) + e_{t+3} \\ &\vdots \\ \hat{y}_{t+k}^{(i)} &= f(\hat{y}_{t+k-1}^{(i)}, \dots, \hat{y}_{t+1}^{(i)}, y_t, \dots, y_{t+k-L}) + e_{t+k} \\ &\vdots \\ \hat{y}_{t+L}^{(i)} &= f(\hat{y}_{t+L-1}^{(i)}, \dots, \hat{y}_{t+1}^{(i)}, y_t) + e_{t+L}\end{aligned}$$

Compute the mean trajectory and the standard deviation

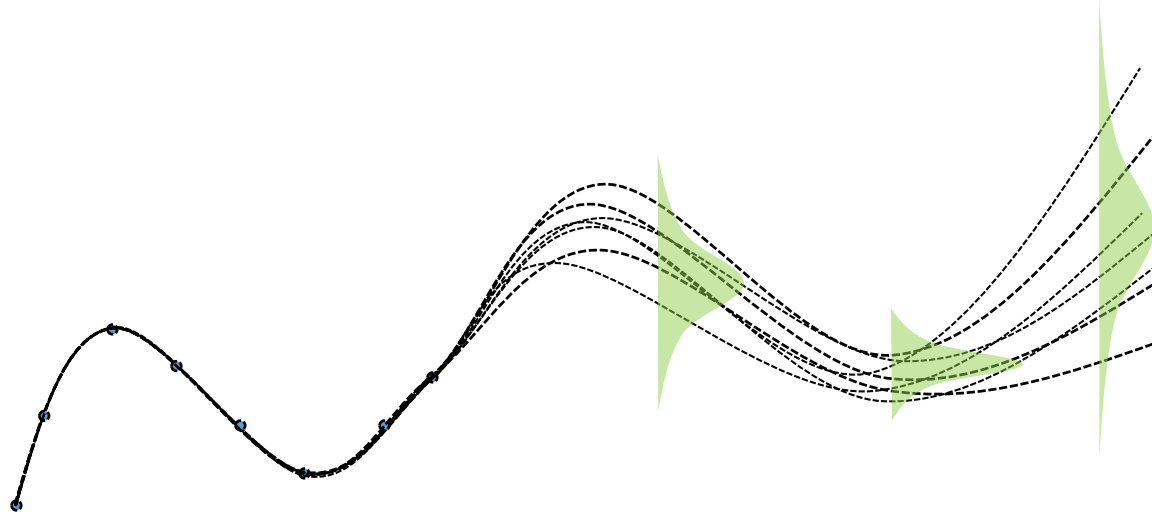
$$(y_t, \dots, y_{t+1-L}) \quad \left\{ \left( \hat{y}_{t+1}^{(i)}, \dots, \hat{y}_{t+L}^{(i)} \right) \right\}_{i=1}^{\text{NumSim}} \quad \begin{aligned} &(\bar{y}_{t+1}, \dots, \bar{y}_{t+L}) \\ &(\sigma_{t+1}, \dots, \sigma_{t+L}) \end{aligned}$$

# Version 3: Iterative Model Building + MPC + Model Uncertainty

How can we estimate the uncertainty in the dynamics and utilize them to make an action?

## Gaussian Process Non-linear Auto Regressive Model

- We generate a large number of trajectories and Compute the mean trajectory and the standard deviation



# Version 3: Iterative Model Building + MPC + Model Uncertainty

## Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models

Kurtland Chua

Roberto Calandra

Rowan McAllister

Sergey Levine

Berkeley Artificial Intelligence Research  
University of California, Berkeley

{kchua, roberto.calandra, rmcallister, svlevine}@berkeley.edu

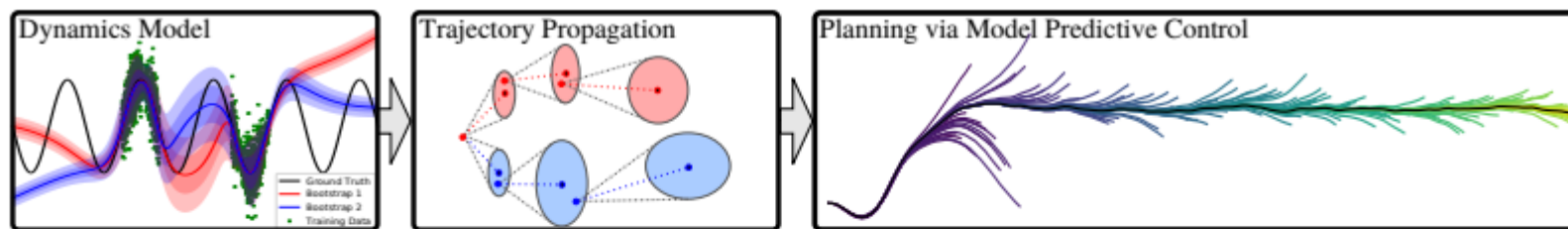


Figure 1: Our method (PE-TS): **Model**: Our probabilistic ensemble (PE) dynamics model is shown as an ensemble of two bootstraps (bootstrap disagreement far from data captures epistemic uncertainty: our subjective uncertainty due to a lack of data), each a probabilistic neural network that captures aleatoric uncertainty (inherent variance of the observed data). **Propagation**: Our trajectory sampling (TS) propagation technique uses our dynamics model to re-sample each particle (with associated bootstrap) according to its probabilistic prediction at each point in time, up until horizon  $T$ . **Planning**: At each time step, our MPC algorithm computes an optimal action sequence, applies the first action in the sequence, and repeats until the task-horizon.

# Version 3: Iterative Model Building + MPC + Model Uncertainty

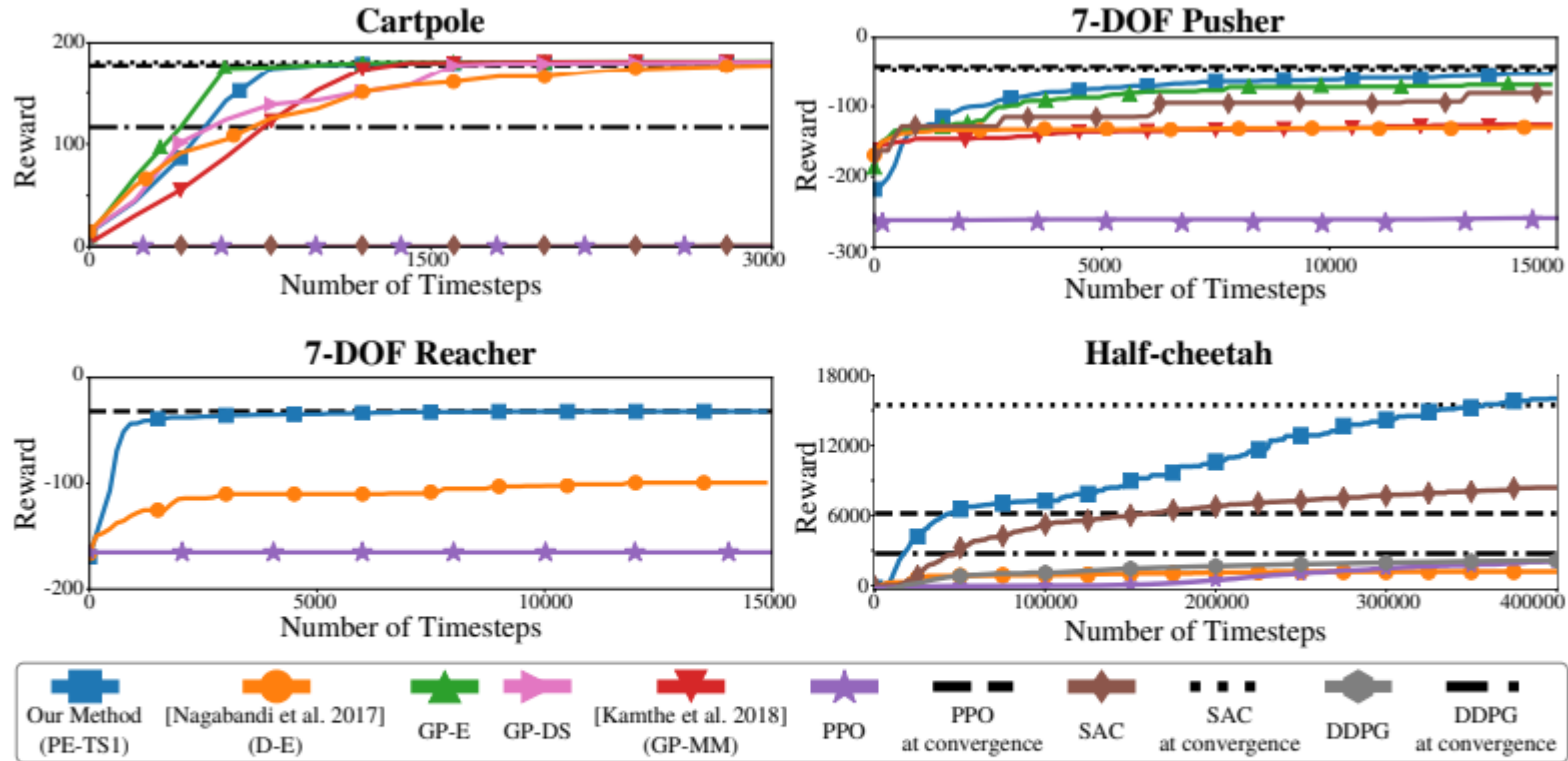
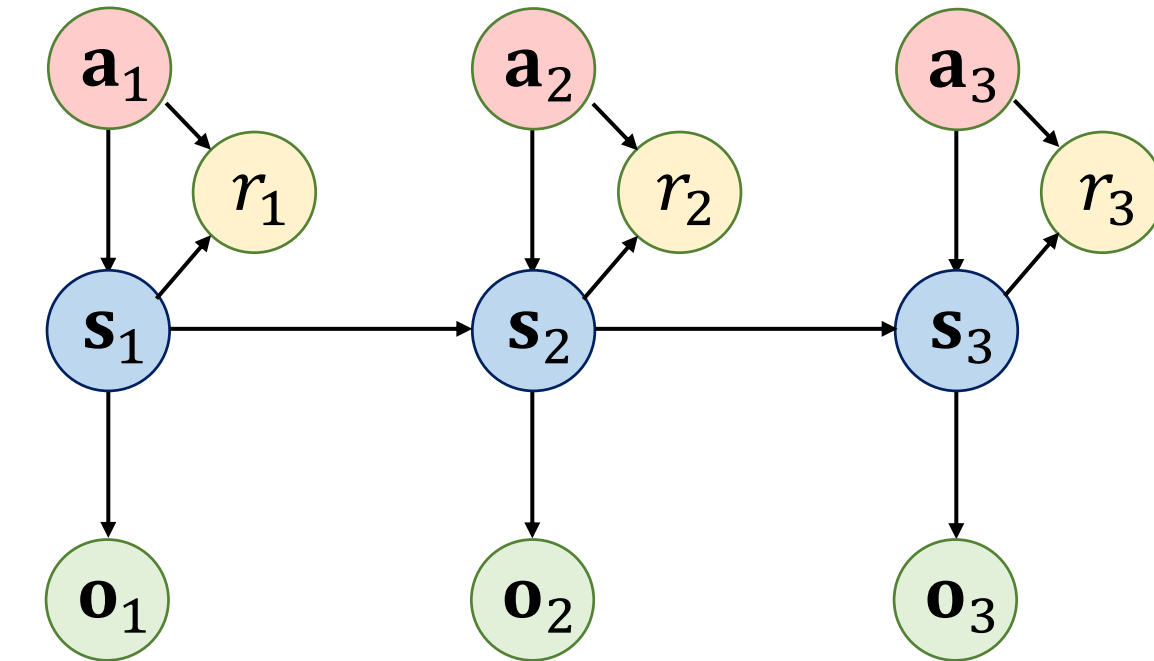


Figure 3: Learning curves for different tasks and algorithm. For all tasks, our algorithm learns in under 100K time steps or 100 trials. With the exception of Cartpole, which is sufficiently low-dimensional to efficiently learn a GP model, our proposed algorithm significantly outperform all other baselines. For each experiment, one time step equals 0.01 seconds, except Cartpole with 0.02 seconds. For visual clarity, we plot the average over 10 experiments of the maximum rewards seen so far.

# Version 4: Iterative Model Building + MPC + Latent Space Model

What about complex observations?



$$f(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{s}_{t+1}$$

What is hard about this?

- ✓ High dimensionality
- ✓ Redundancy
- ✓ Partial observability

$$p(\mathbf{o}_t | \mathbf{s}_t)$$

- Observation model
- High-dimensional but not dynamic

$$p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

- Transition (dynamic model)
- Low dimension but dynamic

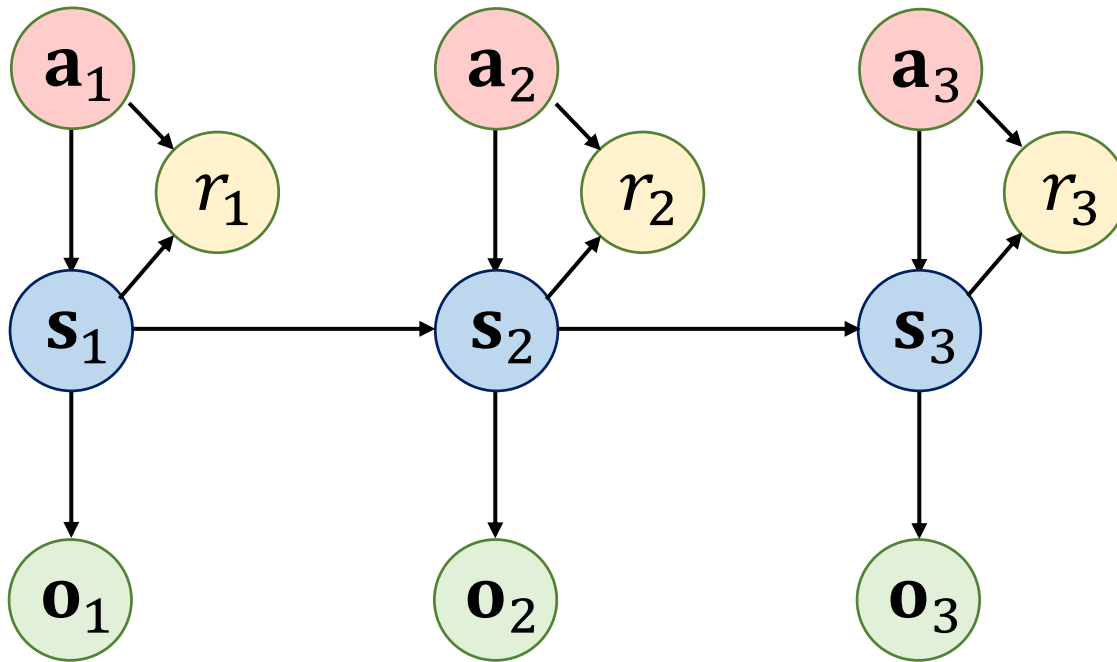
$$p(r_t | \mathbf{s}_t, \mathbf{a}_t)$$

- Reward model



# Version 4: Iterative Model Building + MPC + Latent Space Model

How to train?



$$p(\mathbf{o}_t | \mathbf{s}_t)$$

- Observation model
- High-dimensional but not dynamic

$$p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

- Transition (dynamic model)
- Low dimension but dynamic

$$p(r_t | \mathbf{s}_t, \mathbf{a}_t)$$

- Reward model

Standard (fully observed) model:

$$\max_{\phi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \log p_{\phi}(\mathbf{s}_{t+1,i} | \mathbf{s}_{t,i}, \mathbf{a}_{t,i})$$

Latent space model

$$\max_{\phi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T E[\log p_{\phi}(\mathbf{s}_{t+1,i} | \mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \log p_{\phi}(\mathbf{o}_{t,i} | \mathbf{s}_{t,i})]$$

# Version 4: Iterative Model Building + MPC + Latent Space Model

## How to train?

Latent space model

$$\max_{\phi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T E[\log p_{\phi}(\mathbf{s}_{t+1,i} | \mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \log p_{\phi}(\mathbf{o}_{t,i} | \mathbf{s}_{t,i})]$$

Learn approximate posterior  $q_{\psi}(\mathbf{s}_t | \mathbf{o}_{1:t}, \mathbf{a}_{1:t})$  : Encoder

A single step stochastic encoder is  $q_{\psi}(\mathbf{s}_t | \mathbf{o}_t)$  (variational inference is required to learn the stochastic encoder)

A single step deterministic encoder

$$q_{\psi}(\mathbf{s}_t | \mathbf{o}_t) = \delta(\mathbf{s}_t = g_{\psi}(\mathbf{o}_t)) \Rightarrow \mathbf{s}_t = g_{\psi}(\mathbf{o}_t)$$

Learn both encoder, transition model, image reconstruction, and reward model

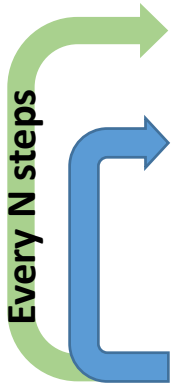
$$\max_{\phi, \psi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^T \underbrace{\log p_{\phi}(g_{\psi}(\mathbf{o}_{t+1,i}) | g_{\psi}(\mathbf{o}_{t,i}), \mathbf{a}_{t,i})}_{\text{Latent space model}} + \underbrace{\log p_{\phi}(\mathbf{o}_{t,i} | g_{\psi}(\mathbf{o}_{t,i}))}_{\text{Image reconstruction}} + \underbrace{\log p_{\phi}(r_{t,i} | g_{\psi}(\mathbf{o}_{t,i}))}_{\text{Reward model}}$$

Many practical methods use a stochastic encoder to model uncertainty

# Version 4: Iterative Model Building + MPC + Latent Space Model

## Version 4: Iterative Model Building + MPC+ Model Uncertainty

1. Run base policy  $\pi_0(\mathbf{a}_t|\mathbf{o}_t)$  (e.g., random policy) to collect  $\mathcal{D} = \{(\mathbf{o}, \mathbf{a}, \mathbf{o}')_i\}$  (Experiencing)
2. Learn  $p_\phi(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ ,  $p_\phi(r_t|\mathbf{s}_t, \mathbf{a}_t)$ ,  $p_\phi(\mathbf{o}_t|\mathbf{s}_t)$ , and  $g_\psi(\mathbf{o}_t)$  (Model Fitting)
3. Plan through the model to choose actions (Model Based Planning)
4. Execute the first planned action, observe resulting  $\mathbf{o}'$  (MPC)
5. Append  $\{(\mathbf{o}, \mathbf{a}, \mathbf{o}')_j\}$  to dataset  $\mathcal{D}$  (Collecting Data)



# Version 4: Iterative Model Building + MPC + Latent Space Model

## Embed to Control: A Locally Linear Latent Dynamics Model for Control from Raw Images

Manuel Watter\* Jost Tobias Springenberg\*

Joschka Boedecker

University of Freiburg, Germany

{watterm, springj, jboeck}@cs.uni-freiburg.de

Martin Riedmiller

Google DeepMind

London, UK

riedmiller@google.com

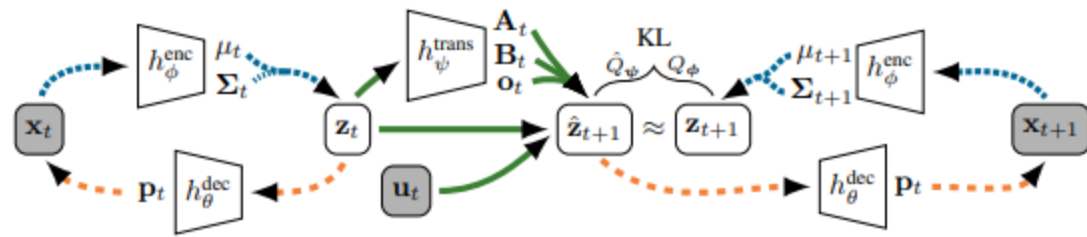


Figure 1: The information flow in the E2C model. From left to right, we encode and decode an image  $x_t$  with the networks  $h_\phi^{\text{enc}}$  and  $h_\theta^{\text{dec}}$ , where we use the latent code  $z_t$  for the transition step. The  $h_\psi^{\text{trans}}$  network computes the local matrices  $A_t, B_t, o_t$  with which we can predict  $\hat{z}_{t+1}$  from  $z_t$  and  $u_t$ . Similarity to the encoding  $z_{t+1}$  is enforced by a KL divergence on their distributions and reconstruction is again performed by  $h_\theta^{\text{dec}}$ .

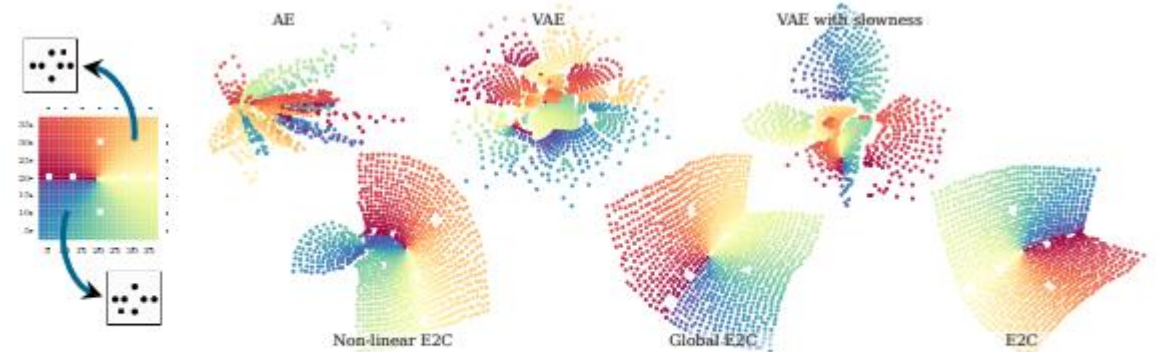


Figure 2: The true state space of the planar system (left) with examples (obstacles encoded as circles) and the inferred spaces (right) of different models. The spaces are spanned by generating images for every valid position of the agent and embedding them with the respective encoders.

# Version 4: Iterative Model Building + MPC + Latent Space Model

## SOLAR: Deep Structured Representations for Model-Based Reinforcement Learning

Marvin Zhang<sup>\*1</sup> Sharad Vikram<sup>\*2</sup> Laura Smith<sup>1</sup> Pieter Abbeel<sup>1</sup> Matthew J. Johnson<sup>3</sup> Sergey Levine<sup>1</sup>

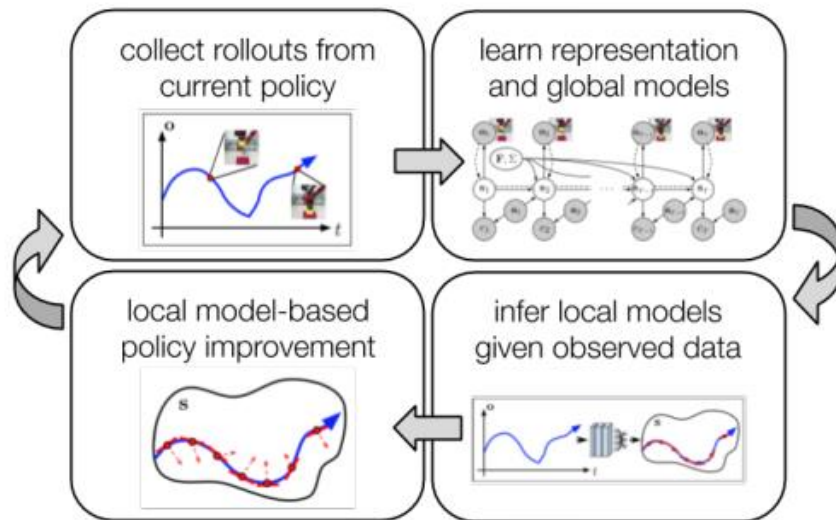


Figure 2. A high-level schematic of our method. We discuss the details of the model and inference procedure in [Section 3](#) and [Section 4](#). We then explain our algorithm in [Section 5](#).

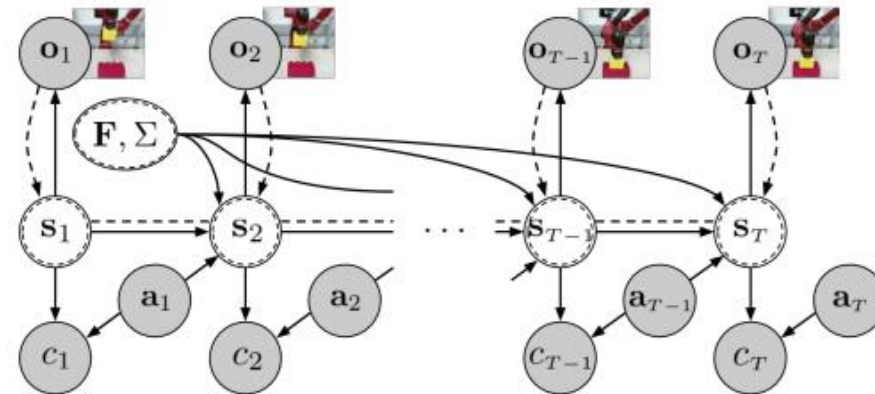
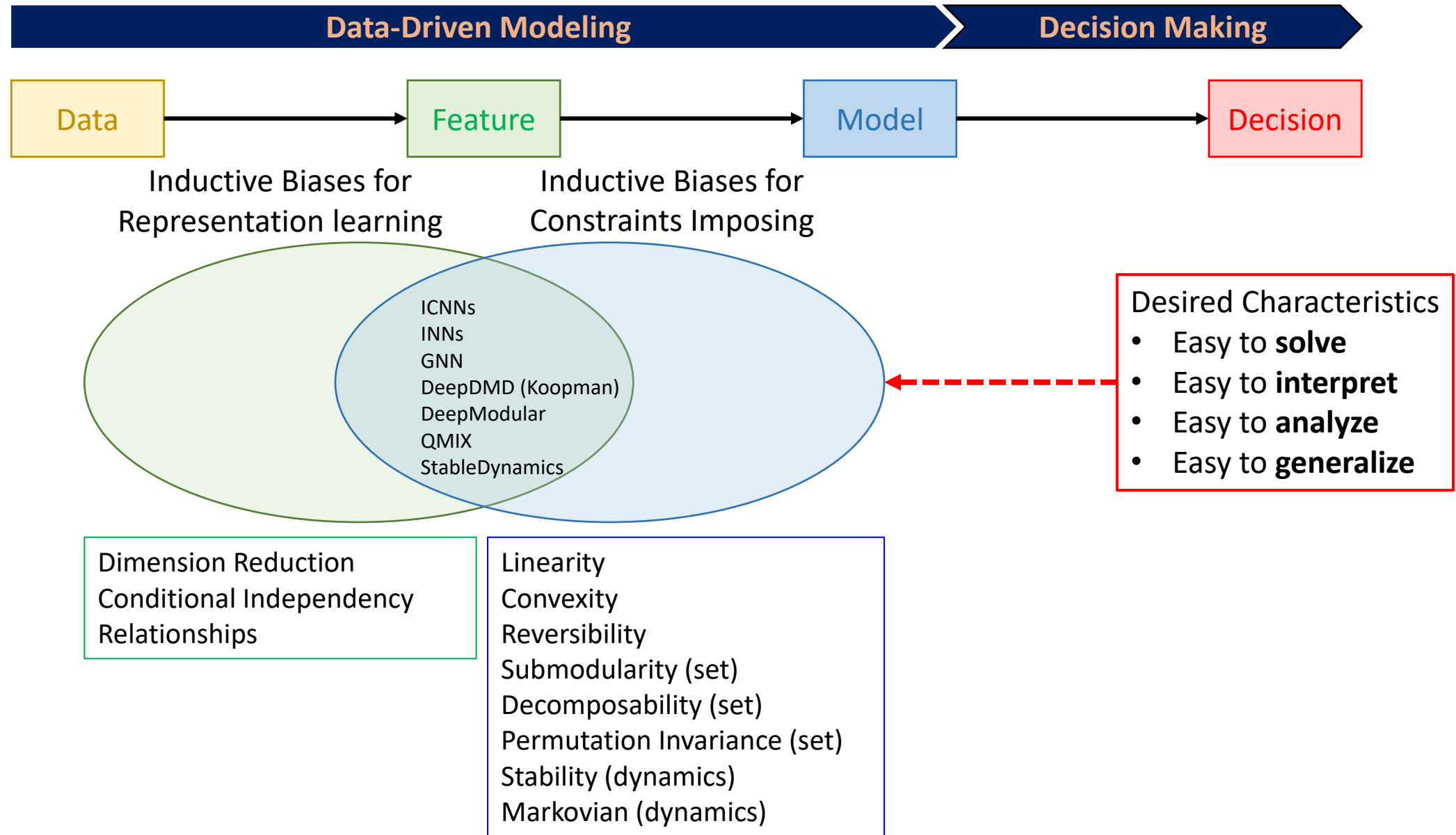


Figure 3. Our generative model, in solid lines, and variational family and recognition network, in dashed lines. In practice, the observations we work with are RGB images, and we use convolutional neural networks for both the recognition and observation models. The distributions for each node are specified in [Section 3](#).

# Input Convex Neural Network for Planning



# Input Convex Neural Network for Planning

## Input Convex Neural Networks

Brandon Amos<sup>1</sup> Lei Xu<sup>2\*</sup> J. Zico Kolter<sup>1</sup>

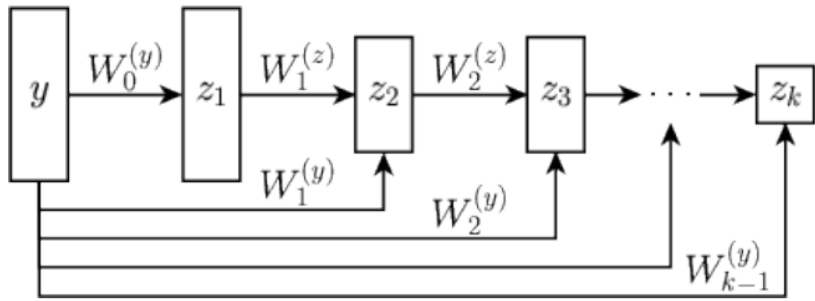


Figure 1. A fully input convex neural network (FICNN).

$$z_{i+1} = g_i \left( W_i^{(z)} z_i + W_i^{(y)} y + b_i \right), \quad f(y; \theta) = z_k$$

**Proposition 1.** The function  $f$  is convex in  $y$  provided that all  $W_{1:k-1}^{(z)}$  are non-negative, and all functions  $g_i$  are convex and non-decreasing.

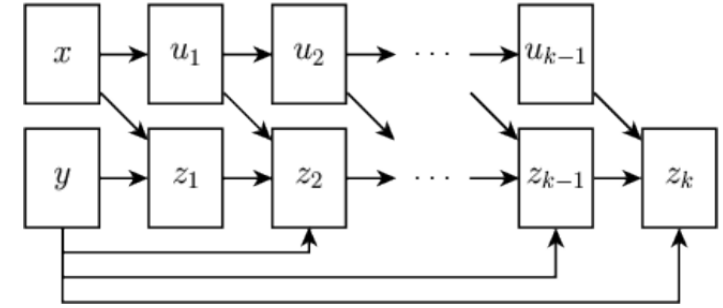
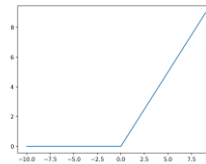


Figure 2. A partially input convex neural network (PICNN).

$$u_{i+1} = \tilde{g}_i(\tilde{W}_i u_i + \tilde{b}_i)$$

$$z_{i+1} = g_i \left( W_i^{(z)} \left( z_i \circ [W_i^{(zu)} u_i + b_i^{(z)}]_+ \right) + \right.$$

$$\left. W_i^{(y)} \left( y \circ (W_i^{(yu)} u_i + b_i^{(y)}) \right) + W_i^{(u)} u_i + b_i \right)$$

$$f(x, y; \theta) = z_k, \quad u_0 = x$$

**Proposition 2.** A PICNN network with  $k$  layers can represent any FICNN with  $k$  layers and any purely feedforward network with  $k$  layers.



# Input Convex Neural Network for Planning

Published as a conference paper at ICLR 2019

## OPTIMAL CONTROL VIA NEURAL NETWORKS: A CONVEX APPROACH

Yize Chen\*, Yuanyuan Shi\*, Baosen Zhang

Department of Electrical and Computer Engineering,  
University of Washington,  
Seattle, WA 98195, USA

{yizechen, yyshi, zhangbao}@uw.edu

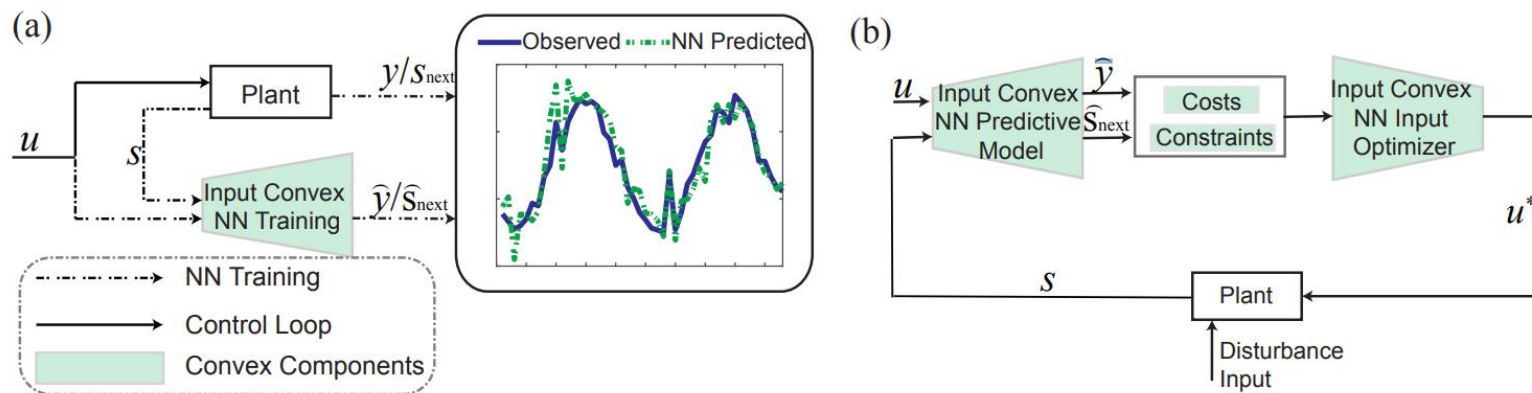


Figure 1: Our proposed model-based method, (a) an input convex neural network is first trained to learn the system dynamics, then (b) we solve a convex predictive control problem to find the optimal actions which are input convex neural networks' inputs. The optimization steps are also based on objectives and dynamics constraints represented by the trained networks.



# Input Convex Neural Network for Planning

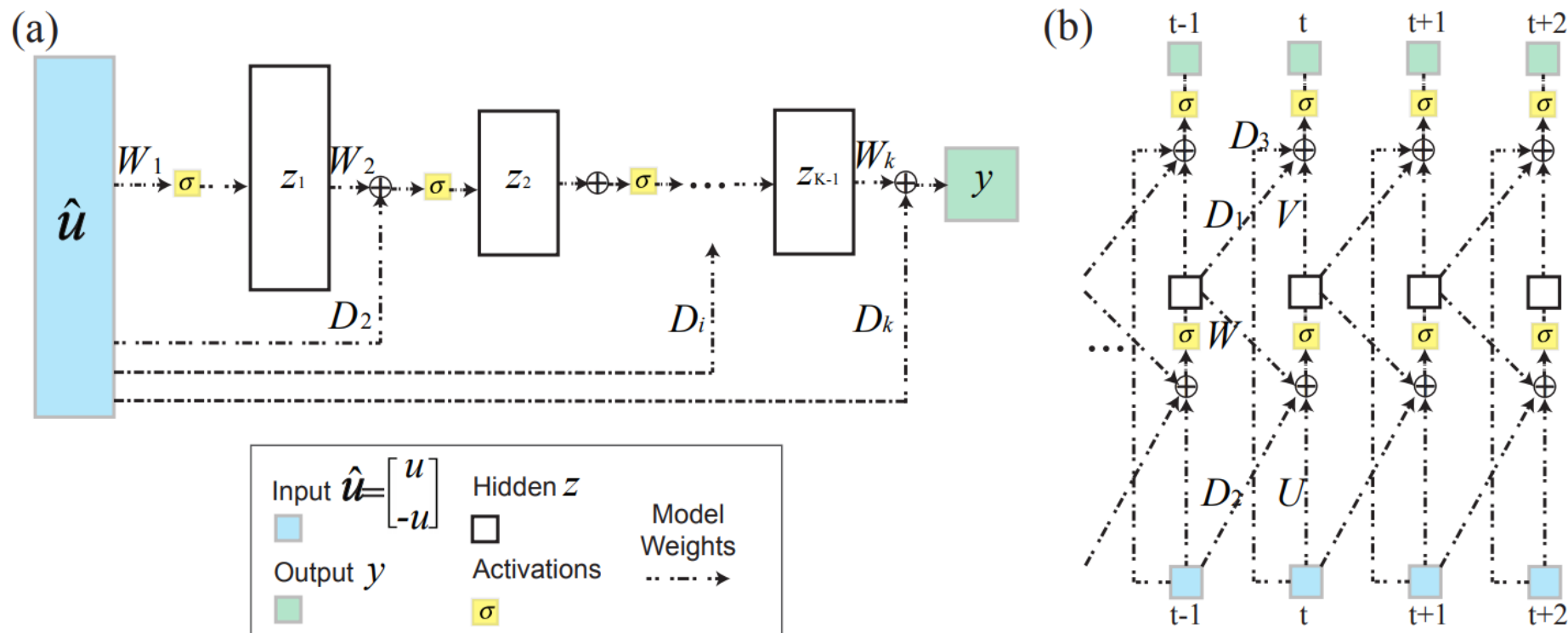


Figure 2: Input convex neural network. (a) Input convex feed-forward neural networks (ICNN). One notable addition is the direct “passthrough” layers  $\mathbf{D}_{2:k}$  that connect the inputs to hidden units for better model representation ability. (b) The proposed input convex recurrent neural networks (ICRNN) architectures. In our control settings, we keep all weights in both networks nonnegative, while expanding the inputs with  $-\mathbf{u}$ .

# Input Convex Neural Network for Planning

$$\begin{aligned} \min_{\mathbf{u}} f(\mathbf{u}; \mathbf{W}) \\ \text{s.t. } \mathbf{u} \in \mathcal{U}, \end{aligned}$$

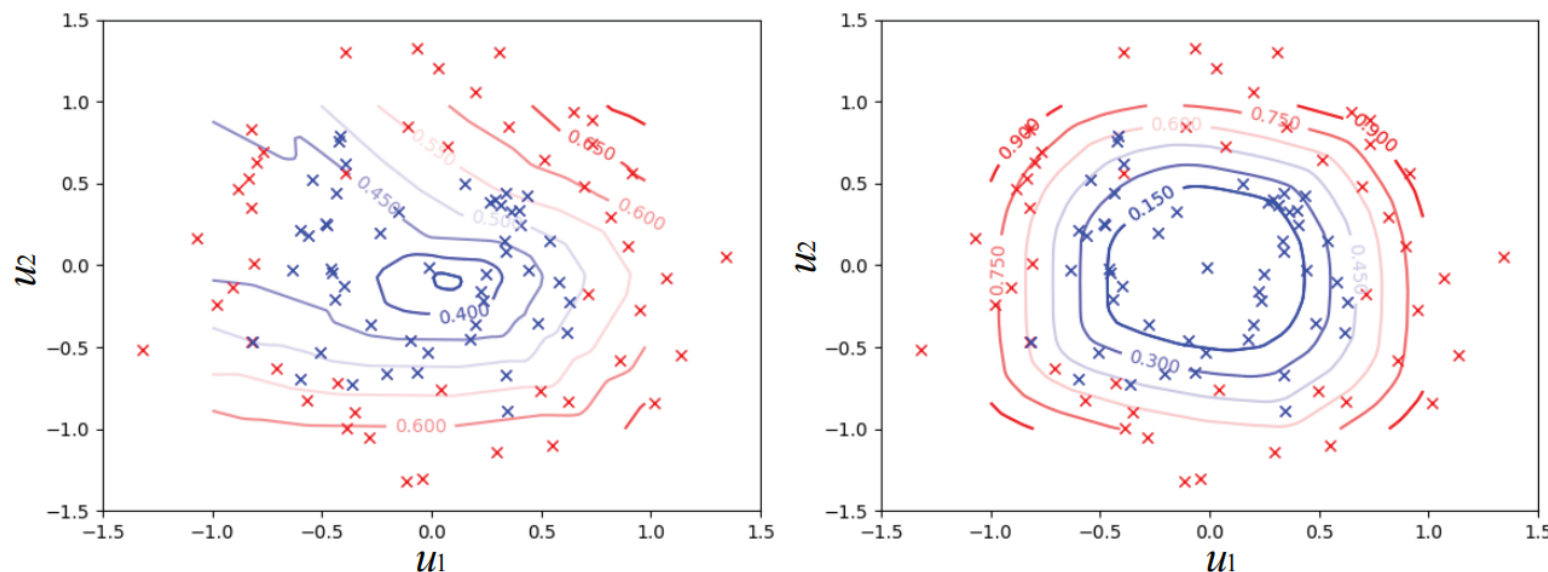


Figure 5: Toy example on classifying circle data with label 0 (blue cross) and label 1 (red cross) along with conventional neural networks (left) and ICNN (right) decision contour lines. A decision maker is interested in finding a  $\mathbf{u}$  that has the highest probability of being labeled 0.

# Input Convex Neural Network for Planning

$$\begin{aligned} & \underset{\mathbf{u}_t, \dots, \mathbf{u}_{t+T}}{\text{minimize}} && \sum_{\tau=t}^{t+T} f(\mathbf{x}_{\tau-n_w}, \dots, \mathbf{x}_{\tau}) \\ & \text{subject to} && \mathbf{s}_{\tau} = g(\mathbf{x}_{\tau-n_w}, \dots, \mathbf{x}_{\tau-1}, \mathbf{u}_{\tau}), \forall \tau \in [t, t+T] \\ & && \underline{\mathbf{u}}_{\tau} \leq \mathbf{u}_{\tau} \leq \bar{\mathbf{u}}_{\tau}, \forall \tau \in [t, t+T] \\ & && \underline{\mathbf{s}}_{\tau} \leq \mathbf{s}_{\tau} \leq \bar{\mathbf{s}}_{\tau}, \forall \tau \in [t, t+T] \end{aligned}$$

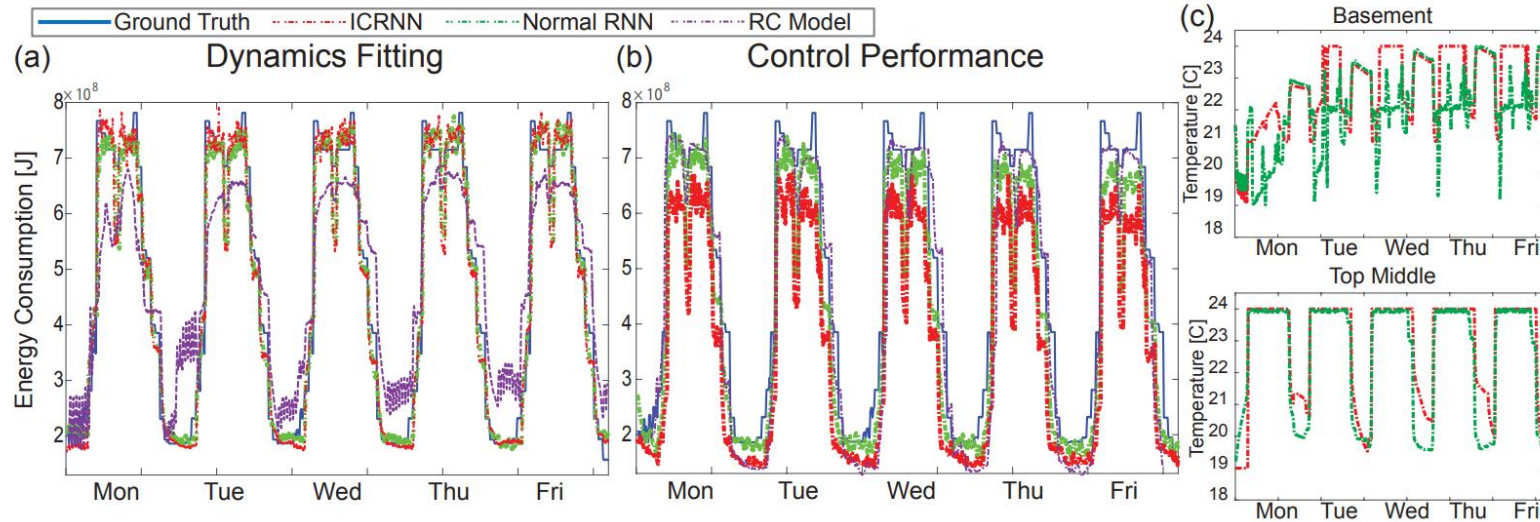


Figure 4: Results for constrained optimization of building energy management. (a) ICRNN is able to model the building dynamics as accurately as conventional RNN; (b) Compared to conventional RNN model, ICRNN finds control actions which lead to 11.52% more of energy savings, and (c) ICRNN provides stable control actions while decisions generated by conventional RNN vary dramatically.

# Input Convex Neural Network for Planning

Proceedings of Machine Learning Research vol xxx:1–11, 2021

## Input Convex Neural Networks for Building MPC

**Felix Büning**

**Adrian Schalbetter**

*Automatic Control Laboratory, ETH Zürich, Switzerland*

*Urban Energy Systems Laboratory, Empa Dübendorf, Switzerland*

FELIX.BUENNING@EMPA.CH

ADRIASCH@STUDENT.ETHZ.CH

**Ahmed Aboudonia**

**Mathias Hudoba de Badyn**

*Automatic Control Laboratory, ETH Zürich, Switzerland*

AHMEDAB@CONTROL.EE.ETHZ.CH

MBADYN@ETHZ.CH

**Philipp Heer**

*Urban Energy Systems Laboratory, Empa Dübendorf, Switzerland*

PHILIPP.HEER@EMPA.CH

**John Lygeros**

*Automatic Control Laboratory, ETH Zürich, Switzerland*

JLYGEROS@ETHZ.CH

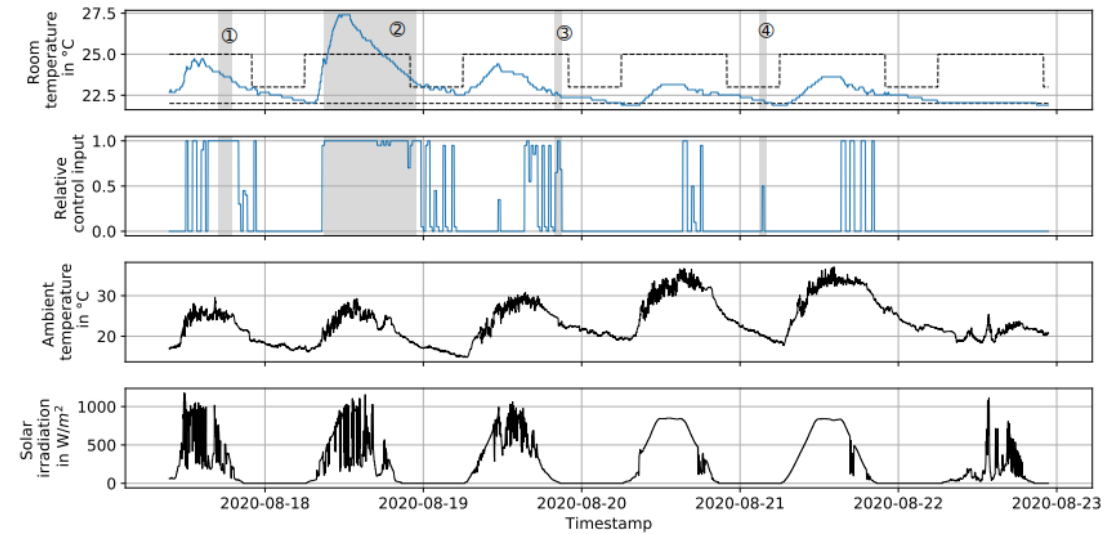


Figure 6: Experimental results of room temperature control by MPC with a FICNN

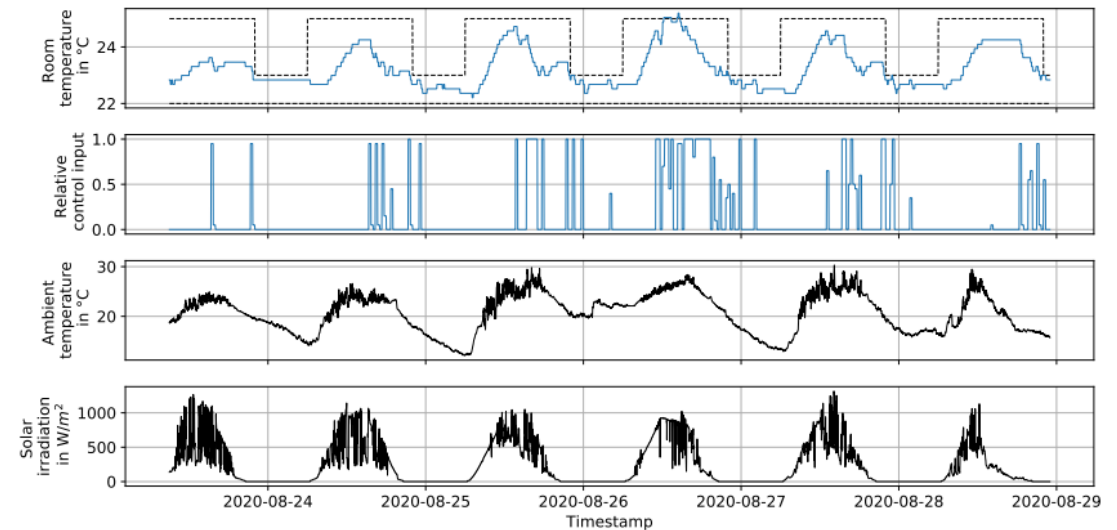


Figure 7: Experimental results of room temperature control by MPC with a PICNN

## MBRL Lecture 3. Dynamic Model Fitting and Policy Learning

Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

Planning

Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data

Data-Driven  
Dynamic  
Model

Planning

Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data

Inductive  
Biases on  
Dynamics

Planning

Policy

Model-Free RL

Data

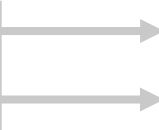
Dynamic  
Model

Policy

Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

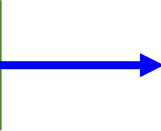


Planning

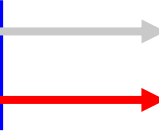
Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data



Data-Driven  
Dynamic  
Model



Planning

Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data



Inductive  
Biases on  
Dynamics



Planning

Policy

Model-Free RL

Data



Dynamic  
Model



Policy

# Overview

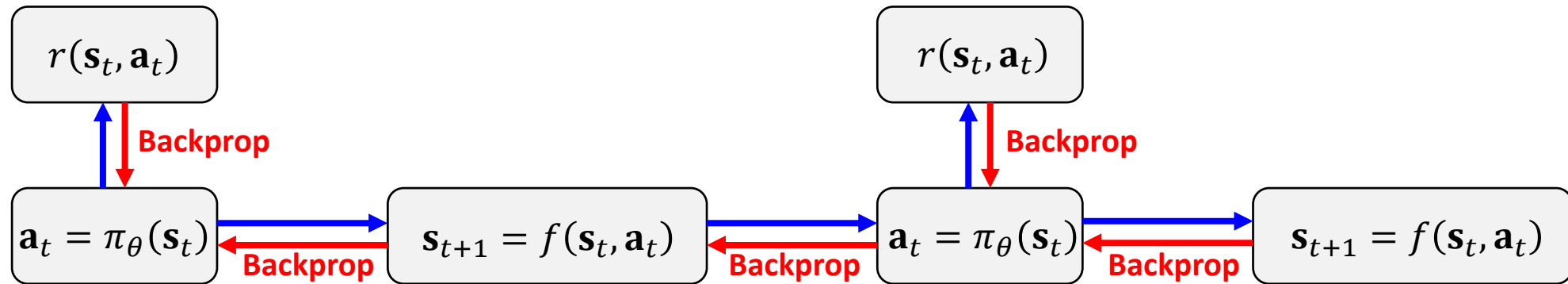
Ways to learn a policy using the learned dynamic model:

- Learn a policy by **Backpropgating dynamic model into policy**  
“PILCO: A Model-Based and Data-Efficient Approach to Policy Search”
- Learn a policy by **Imitating optimal control** in a constrained optimization framework  
“Learning Neural Network Policies with Guided Policy Search under Unknown Dynamics”  
“End-to-End Training of Deep Visuomotor Policies”  
“PLATO: Policy Learning using Adaptive Trajectory Optimization”
- Learn a policy by **using data simulated** by the learned model (e.g., Dyna)  
“Continuous deep Q-learning with model-based acceleration”



# Learn a policy by Backpropagating dynamic model into policy

- Backpropagate directly into the policy?



Steps to learn a policy by backpropagating into the policy

1. Run base policy  $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$  (e.g., random policy) to collect  $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. Learn dynamics model  $f(\mathbf{s}, \mathbf{a})$  to minimize  $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. Backpropagate through  $f(\mathbf{s}, \mathbf{a})$  into the policy to optimize  $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$
4. Run base policy  $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$ , appending the visited tuples  $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$  to  $\mathcal{D}$

# Learn a policy by Backpropgating dynamic model into policy

“PILCO: A Model-Based and Data-Efficient Approach to Policy Search”

- The paper introduce PILCO, a practical data-efficient model-based policy search method
- PILCO reduces model bias, one of the key problems of model-based reinforcement learning
  - Learn a probabilistic dynamics model and explicitly incorporating model uncertainty into long-term planning
  - Can cope with very little data and facilitates learning from scratch in only a few trials
- Policy evaluation is performed in closed form using state-of-the-art approximate inference
- Policy improvement is conducted using policy gradient computed analytically.

# Learn a policy by Backpropagating dynamic model into policy

## “PILCO: A Model-Based and Data-Efficient Approach to Policy Search”

How to overcome Model bias?

- Model-based method is promising to efficiently extract valuable information from available data than model-free method only when biases used for modeling are correct.
- One reason why model-based methods are not widely used in learning from scratch is that they suffer from model bias, i.e., they inherently assume that the learned dynamics model sufficiently accurately resembles the real environment
- Model bias is especially an issue when only a few samples and no informative prior knowledge about the task to be learned are available.

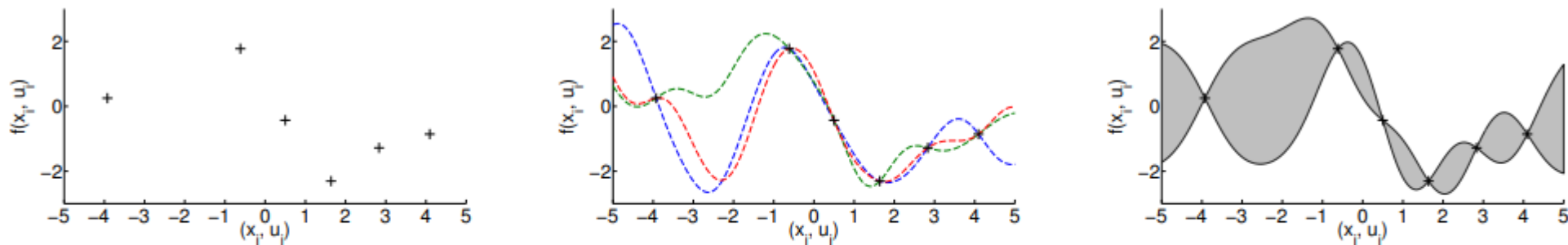


Figure 1. Small data set of observed transitions (left), multiple plausible deterministic function approximators (center), probabilistic function approximator (right). The probabilistic approximator models uncertainty about the latent function.

# Learn a policy by Backpropgating dynamic model into policy

“PILCO: A Model-Based and Data-Efficient Approach to Policy Search”

Consider dynamic system:

$$\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_{t-1})$$

The objective is to find a deterministic policy/controller  $\pi: \mathbf{x} \mapsto \pi(\mathbf{x}) = \mathbf{u}$  that minimizes the expected return:

$$J^\pi(\theta) = \sum_{t=0}^T E_{\mathbf{x}_t} [c(\mathbf{x}_t)], \quad \mathbf{x}_0 \sim N(\mu_0, \Sigma_0)$$

- ✓  $\pi$  is parameterized by  $\theta$
- ✓ The cost function  $c(\mathbf{x}_t)$  is the cost of being in state  $\mathbf{x}$  at time  $t$  (encoding some information about a target state  $\mathbf{x}_{target}$ )

# Learn a policy by Backpropagating dynamic model into policy

“PILCO: A Model-Based and Data-Efficient Approach to Policy Search”

## Dynamic model learning

- PILCO’s probabilistic dynamics model is implemented as a GP
  - ✓ Input:  $(\mathbf{x}_{t-1}, \mathbf{u}_{t-1}) \in \mathbb{R}^{D+F}$
  - ✓ Target: differences:  $\Delta_t = \mathbf{x}_t - \mathbf{x}_{t-1} + \epsilon \in \mathbb{R}^D, \epsilon \sim N(0, \Sigma_\epsilon)$   
 $\mathbf{x}_t = f(\mathbf{x}_{t-1}, \mathbf{u}_{t-1})$

- The GP yields one-step predictions

$$\begin{aligned} p(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{u}_{t-1}) &= N(\mathbf{x}_t | \mu_t, \Sigma_t) \\ \mu_t &= \mathbf{x}_{t-1} + \mathbb{E}_f[\Delta_t] \\ \Sigma_t &= \text{var}_f[\Delta_t] \end{aligned}$$

- The posterior predictive distribution  $p(\Delta_* | \tilde{\mathbf{x}}_*)$  for arbitrary, but known, test input  $\tilde{\mathbf{x}}_*$  is Gaussian with mean and variance:

$$\begin{aligned} m_f(\tilde{\mathbf{x}}_*) &= \mathbb{E}_f[\Delta_*] = \mathbf{k}_*^\top (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y} = \mathbf{k}_*^\top \boldsymbol{\beta}, \\ \sigma_f^2(\Delta_*) &= \text{var}_f[\Delta_*] = k_{**} - \mathbf{k}_*^\top (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}_*, \end{aligned}$$

# Learn a policy by Backpropagating dynamic model into policy

“PILCO: A Model-Based and Data-Efficient Approach to Policy Search”

## Policy Evaluation

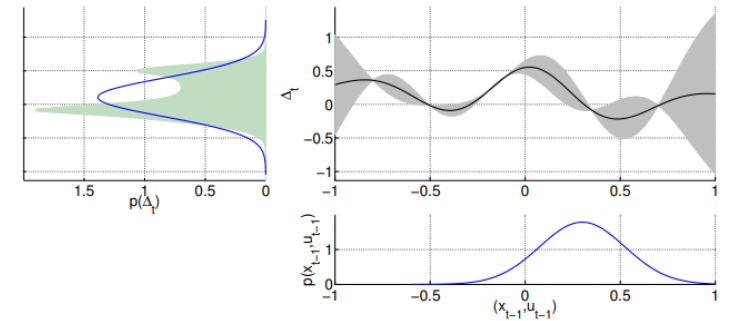
- Minimizing and evaluating  $J^\pi(\theta)$  requires long term predictions of the state evolution.
- To obtain the state distributions  $p(\mathbf{x}_1), \dots, p(\mathbf{x}_T)$ , the model cascade on step predictions, which require mapping uncertain test inputs through the GP dynamics model.

- Approximate the joint state-control distribution

$$p(\tilde{\mathbf{x}}_{t-1}) = N(\tilde{\mathbf{x}}_{t-1} | \tilde{\mu}_{t-1}, \tilde{\Sigma}_{t-1})$$

- Then, the predictive distribution  $p(\Delta_t)$  is computed as

$$p(\Delta_t) = \int p(f(\tilde{\mathbf{x}}_{t-1}) | \tilde{\mathbf{x}}_{t-1}) p(\tilde{\mathbf{x}}_{t-1}) d\tilde{\mathbf{x}}_{t-1}$$



- ✓ Computing the exact predictive distribution is analytically intractable, thus approximated as Gaussian using exact moment matching.
- Finally, a Gaussian approximation of the desired distribution  $p(\mathbf{x}_t)$  is given as  $N(\mathbf{x}_t | \mu_t, \Sigma_t)$  with

$$\mu_t = \mu_{t-1} + \mu_\Delta$$

$$\Sigma_t = \Sigma_{t-1} + \Sigma_\Delta + \text{cov}[\mathbf{x}_{t-1}, \Delta_t] + \text{cov}[\Delta_t, \mathbf{x}_{t-1}]$$

$$\text{cov}[\mathbf{x}_{t-1}, \Delta_t] = \text{cov}[\mathbf{x}_{t-1}, \mathbf{u}_{t-1}] \Sigma_u^{-1} \text{cov}[\mathbf{u}_{t-1}, \Delta_t]$$

# Learn a policy by Backpropgating dynamic model into policy

“PILCO: A Model-Based and Data-Efficient Approach to Policy Search”

## Policy Improvement

- The approach allows analytically computing the gradients of the expected return  $J^\pi(\theta)$  with respect to the policy parameter  $\theta$  by repeated application of the chain rule.
- Setting  $\mathcal{E}_t \equiv E_{\mathbf{x}_t}[c(\mathbf{x}_t)]$

$$\begin{aligned}\frac{d\mathcal{E}_t}{d\theta} &= \frac{d\mathcal{E}_t}{dp(\mathbf{x}_t)} \frac{dp(\mathbf{x}_t)}{d\theta} := \frac{\partial \mathcal{E}_t}{\partial \mu_t} \frac{d\mu_t}{d\theta} + \frac{\partial \mathcal{E}_t}{\partial \Sigma_t} \frac{d\Sigma_t}{d\theta} \\ \frac{dp(\mathbf{x}_t)}{d\theta} &= \frac{\partial p(\mathbf{x}_t)}{\partial p(\mathbf{x}_{t-1})} \frac{dp(\mathbf{x}_{t-1})}{d\theta} + \frac{\partial p(\mathbf{x}_t)}{\partial \theta}, \\ \frac{\partial p(\mathbf{x}_t)}{\partial p(\mathbf{x}_{t-1})} &= \left\{ \frac{\partial \mu_t}{\partial p(\mathbf{x}_{t-1})}, \frac{\partial \Sigma_t}{\partial p(\mathbf{x}_{t-1})} \right\}. \\ \frac{d\mu_t}{d\theta} &= \frac{\partial \mu_t}{\partial \mu_{t-1}} \frac{d\mu_{t-1}}{d\theta} + \frac{\partial \mu_t}{\partial \Sigma_{t-1}} \frac{d\Sigma_{t-1}}{d\theta} + \frac{\partial \mu_t}{\partial \theta}. \\ \frac{\partial \mu_t}{\partial \theta} &= \frac{\partial \mu_\Delta}{\partial p(\mathbf{u}_{t-1})} \frac{\partial p(\mathbf{u}_{t-1})}{\partial \theta} = \frac{\partial \mu_\Delta}{\partial \mu_u} \frac{\partial \mu_u}{\partial \theta} + \frac{\partial \mu_\Delta}{\partial \Sigma_u} \frac{\partial \Sigma_u}{\partial \theta}.\end{aligned}$$

---

### Algorithm 1 PILCO

---

- 1: **init:** Sample controller parameters  $\theta \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ .  
Apply random control signals and record data.
  - 2: **repeat**
  - 3:   Learn probabilistic (GP) dynamics model, see Sec. 2.1, using all data.
  - 4:   Model-based policy search, see Sec. 2.2–2.3.
  - 5:   **repeat**
  - 6:     Approximate inference for policy evaluation, see Sec. 2.2: get  $J^\pi(\theta)$ , Eqs. (10)–(12), (24).
  - 7:     Gradient-based policy improvement, see Sec. 2.3: get  $dJ^\pi(\theta)/d\theta$ , Eqs. (26)–(30).
  - 8:     Update parameters  $\theta$  (e.g., CG or L-BFGS).
  - 9:     **until** convergence; **return**  $\theta^*$
  - 10:   Set  $\pi^* \leftarrow \pi(\theta^*)$ .
  - 11:   Apply  $\pi^*$  to system (single trial/episode) and record data.
  - 12: **until** task learned
-

# Learn a policy by Backpropgating dynamic model into policy

“PILCO: A Model-Based and Data-Efficient Approach to Policy Search”

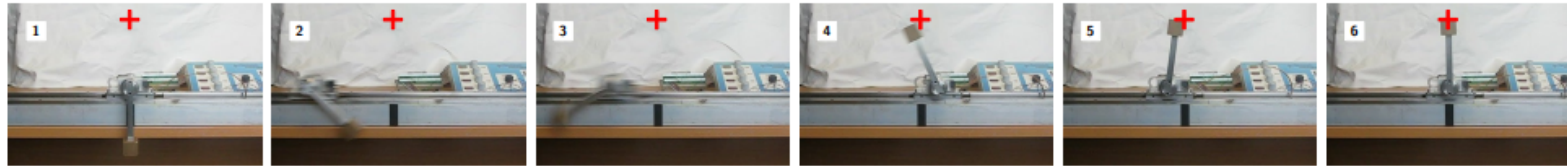
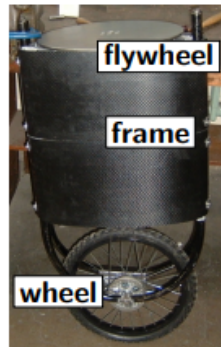
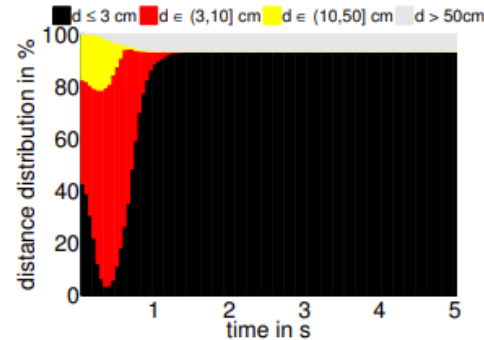


Figure 3. Real cart-pole system. Snapshots of a controlled trajectory of 20 s length after having learned the task. To solve the swing-up plus balancing, PILCO required only 17.5 s of interaction with the physical system.



(a) Robotic unicycle.



(b) Histogram (after 1,000 test runs) of the distances of the flywheel from being upright.

Figure 4. Robotic unicycle system and simulation results. The state space is  $\mathbb{R}^{12}$ , the control space  $\mathbb{R}^2$ .

Table 1. PILCO’s data efficiency scales to high dimensions.

|                 | cart-pole          | cart-double-pole    | unicycle            |
|-----------------|--------------------|---------------------|---------------------|
| state space     | $\mathbb{R}^4$     | $\mathbb{R}^6$      | $\mathbb{R}^{12}$   |
| # trials        | $\leq 10$          | 20–30               | $\approx 20$        |
| experience      | $\approx 20$ s     | $\approx 60$ s–90 s | $\approx 20$ s–30 s |
| parameter space | $\mathbb{R}^{305}$ | $\mathbb{R}^{1816}$ | $\mathbb{R}^{28}$   |

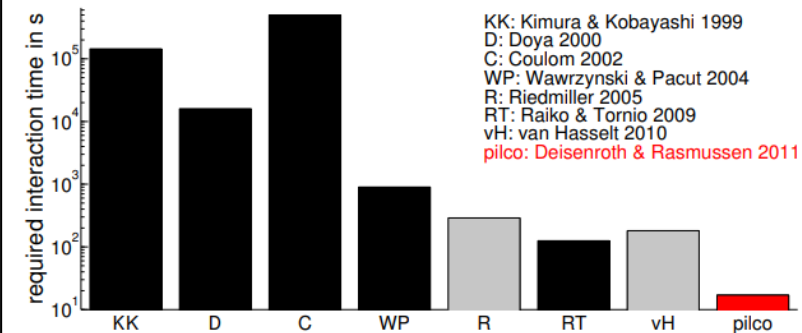
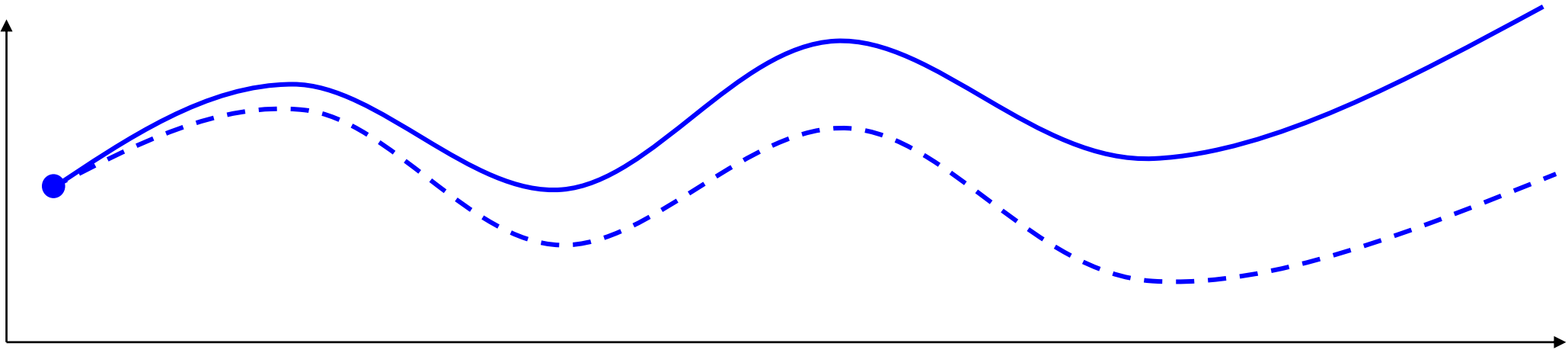
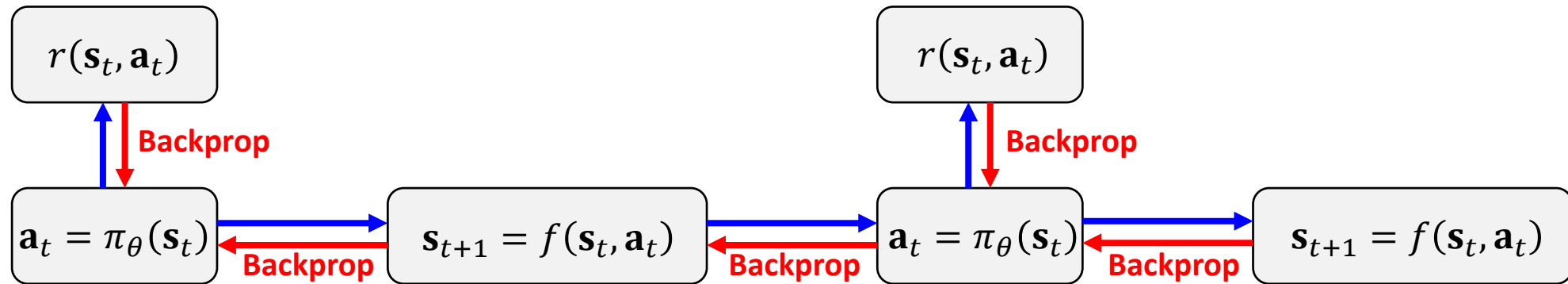


Figure 5. Data efficiency for learning the cart-pole task in the absence of expert knowledge. The horizontal axis chronologically orders the references according to their publication date. The vertical axis shows the required interaction time with the cart-pole system on a log-scale.



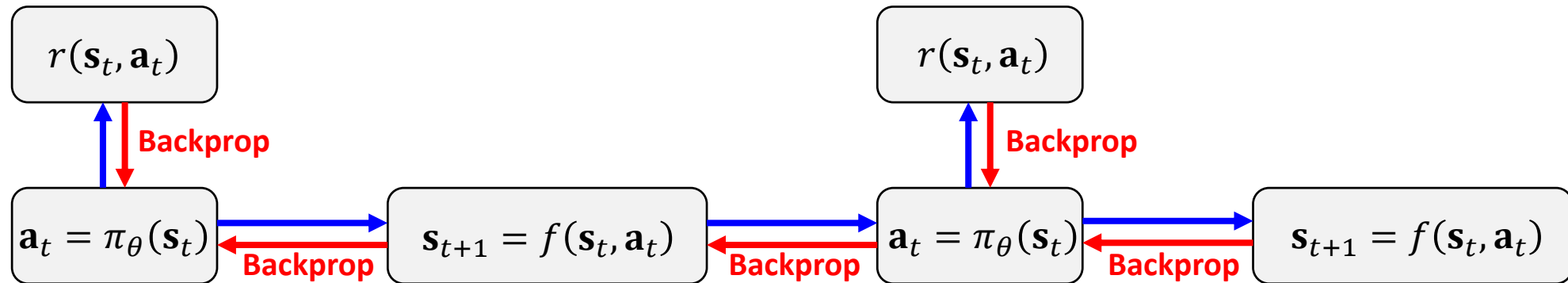
# Learn a policy by Backpropagating dynamic model into policy

- Backpropagate directly into the policy?



# Learn a policy by Backpropagating dynamic model into policy

- Backpropagate directly into the policy?



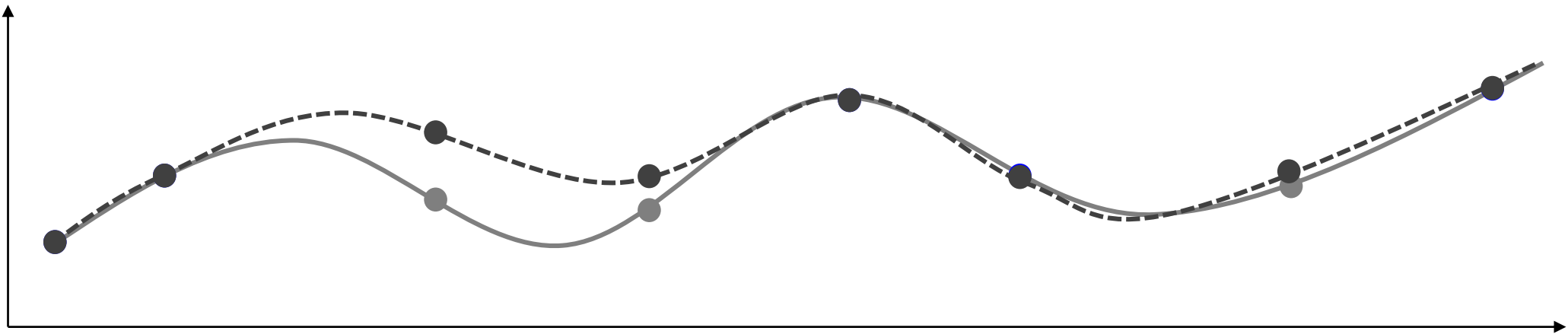
## Issues in directly backpropagating into the policy

- Parameter sensitivity problems
  - ✓ Similar issues as shooting methods
  - no longer have convenient second order LQR-like method, because policy parameters couple all the time steps, so no dynamic programming
- Vanishing and exploding gradients
  - ✓ Similar issues in training long RNNs with BPTT
  - Unlike LSTM, we can't just "choose" a simple dynamics, dynamics are chosen by nature

# Learn a policy by Imitating optimal control in a constrained opt.

- Optimization with **dynamics** as a constraints

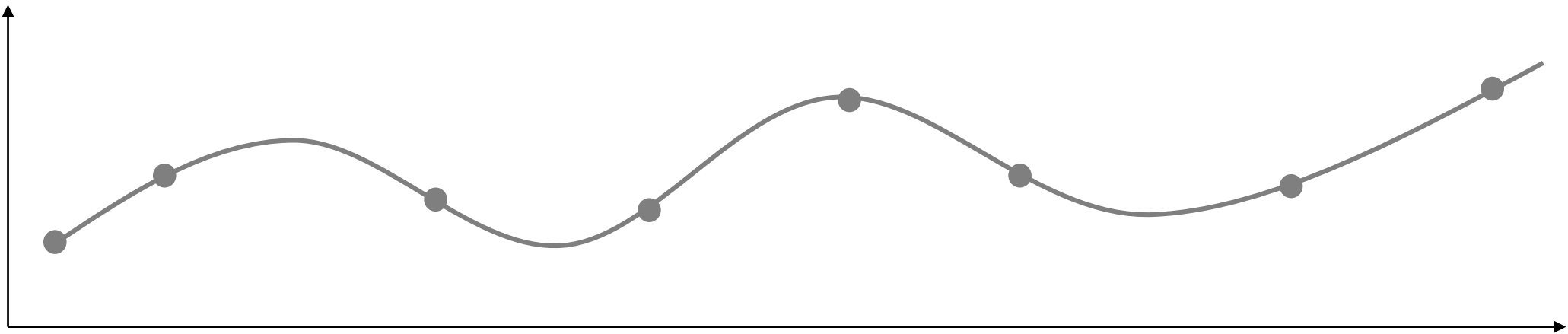
$$\begin{aligned} & \underset{\mathbf{u}_1, \dots, \mathbf{u}_T, \mathbf{x}_1, \dots, \mathbf{x}_T}{\text{minimize}} && \sum_{t=1}^T c(\mathbf{x}_t, \mathbf{u}_t) \\ & \text{s.t.} && \mathbf{x}_t = f(\mathbf{x}_t, \mathbf{u}_t) \end{aligned}$$



# Learn a policy by Imitating optimal control in a constrained opt.

- Optimization with dynamics as a constraints and imitation learning

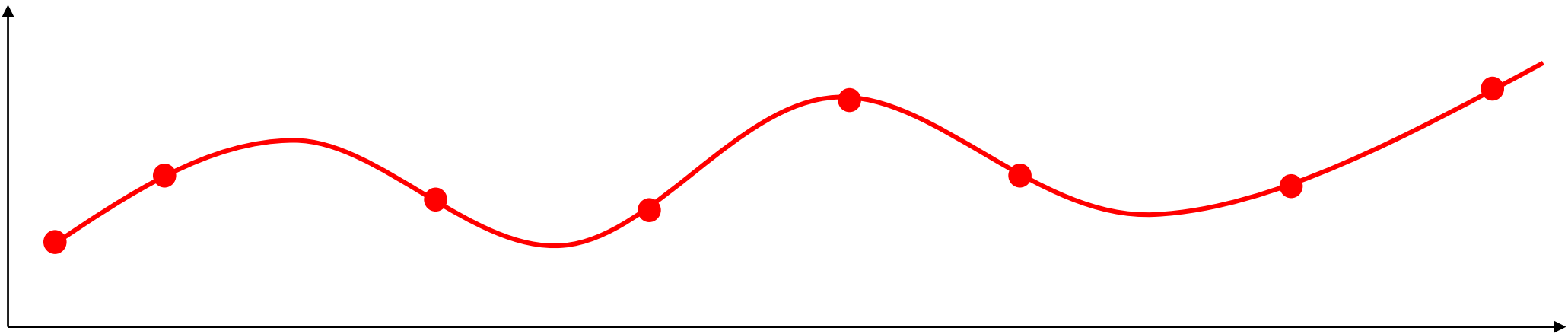
$$\begin{aligned} & \underset{\mathbf{u}_1, \dots, \mathbf{u}_T, \mathbf{x}_1, \dots, \mathbf{x}_T, \theta}{\text{minimize}} && \sum_{t=1}^T c(\mathbf{x}_t, \mathbf{u}_t) \\ & \text{s.t.} && \mathbf{x}_t = f(\mathbf{x}_t, \mathbf{u}_t) \\ & && \mathbf{u}_t = \pi_\theta(\mathbf{x}_t) \end{aligned}$$



# Learn a policy by Imitating optimal control in a constrained opt.

- Trajectory optimization with dynamics as a constraints and imitation learning

$$\begin{aligned} & \underset{\tau, \theta}{\text{minimize}} \sum_{t=1}^T c(\tau) \\ & \text{s.t. } \mathbf{u}_t = \pi_{\theta}(\mathbf{x}_t) \end{aligned}$$



# Learn a policy by Imitating optimal control in a constrained opt.

|                                              |                                                                                                                                  | Constraint imposed by the learned dynamics model    | Constraint imposed by the planned (optimized) action |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|------------------------------------------------------|
| Optimization                                 | $\min_{\mathbf{u}} c(\mathbf{u})$                                                                                                |                                                     |                                                      |
| Dynamic Optimization (Optimal Control)       | $\min_{\mathbf{u}_1, \dots, \mathbf{u}_T, \mathbf{x}_1, \dots, \mathbf{x}_T} \sum_{t=1}^T c(\mathbf{x}_t, \mathbf{u}_t)$         | s.t. $\mathbf{x}_t = f(\mathbf{x}_t, \mathbf{u}_t)$ |                                                      |
| Dynamic Optimization with Imitation Learning | $\min_{\mathbf{u}_1, \dots, \mathbf{u}_T, \mathbf{x}_1, \dots, \mathbf{x}_T, \theta} \sum_{t=1}^T c(\mathbf{x}_t, \mathbf{u}_t)$ | s.t. $\mathbf{x}_t = f(\mathbf{x}_t, \mathbf{u}_t)$ | s.t. $\mathbf{u}_t = \pi_{\theta}(\mathbf{x}_t)$     |

## Why use imitation learning?

- Overcomes optimization challenges of Backpropgating into policy directly
- Relatively stable and easy to use and sample efficient
- Supervised learning works very well + control/planning works very well with the model
- The combination of two works very well
- Input remapping trick: can exploit availability of additional information at training time to learn policy from raw observation

# Learn a policy by Imitating optimal control in a constrained opt.

(Recap) Dual Gradient Decent Algorithm : We are going to use the constraint for imitation learning

$$\min_{\mathbf{x}}. f(\mathbf{x}) \text{ s.t. } \mathcal{C}(\mathbf{x}) = 0$$

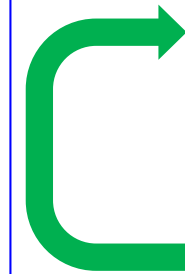
$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda \mathcal{C}(\mathbf{x})$$

$$\mathbf{x}^*(\lambda) = \operatorname{argmin}_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \lambda)$$

$$g(\lambda) = \mathcal{L}(\mathbf{x}^*(\lambda), \lambda)$$

$$\frac{dg}{d\lambda} = \cancel{\frac{d\mathcal{L}}{d\mathbf{x}^*} \frac{d\mathbf{x}^*}{d\lambda}} + \frac{d\mathcal{L}}{d\lambda}$$

If  $\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \lambda)$ , then  $\frac{d\mathcal{L}}{d\mathbf{x}^*} = \mathbf{0}$



1. Find  $\mathbf{x}^* = \operatorname{argmin}_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \lambda)$

2. Compute  $\frac{dg}{d\lambda} = \frac{d\mathcal{L}}{d\lambda}(\mathbf{x}^*(\lambda), \lambda)$

3. Update  $\lambda \leftarrow \lambda + \alpha \frac{dg}{d\lambda}$

# Learn a policy by Imitating optimal control in a constrained opt.

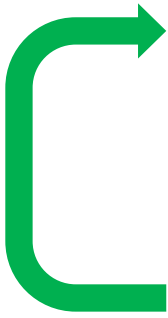
- Trajectory optimization with dynamics as a constraints and imitation learning

$$\begin{aligned} & \underset{\tau, \theta}{\text{minimize}} \sum_{t=1}^T c(\tau) \\ & \text{s.t. } \mathbf{u}_t = \pi_{\theta}(\mathbf{x}_t) \end{aligned}$$

Constraining trajectory optimization with dual gradient descent:

$$\mathcal{L}(\tau, \theta, \lambda) = c(\tau) + \sum_{t=1}^T \lambda_t (\pi_{\theta}(\mathbf{x}_t) - \mathbf{u}_t)$$

$$\bar{\mathcal{L}}(\tau, \theta, \lambda) = c(\tau) + \sum_{t=1}^T \lambda_t (\pi_{\theta}(\mathbf{x}_t) - \mathbf{u}_t) + \sum_{t=1}^T \rho_t (\pi_{\theta}(\mathbf{x}_t) - \mathbf{u}_t)^2 \quad (\text{Augmented Lagrangian})$$

- 
1. Find  $\tau^* \leftarrow \min_{\tau} \bar{\mathcal{L}}(\tau, \theta, \lambda)$  (e.g., via iLQR) (constrained trajectory optimization)
  2. Find  $\theta^* \leftarrow \min_{\theta} \bar{\mathcal{L}}(\tau, \theta, \lambda)$  (e.g., via SGD) (supervised learning)
  3. Update  $\lambda \leftarrow \lambda + \alpha \frac{d\bar{\mathcal{L}}}{d\lambda}(\tau^*, \theta^*, \lambda)$

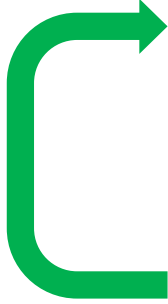


# Learn a policy by Imitating optimal control in a constrained opt.

- Trajectory optimization with dynamics as a constraints and imitation learning

$$\begin{aligned} & \underset{\tau, \theta}{\text{minimize}} \sum_{t=1}^T c(\tau) \\ & \text{s.t. } \mathbf{u}_t = \pi_{\theta}(\mathbf{x}_t) \end{aligned}$$

Constraining trajectory optimization with dual gradient descent:

- 
1. Find  $\tau^* \leftarrow \min_{\tau} \bar{\mathcal{L}}(\tau, \theta, \lambda)$  (e.g., via iLQR)
  2. Find  $\theta^* \leftarrow \min_{\theta} \bar{\mathcal{L}}(\tau, \theta, \lambda)$  (e.g., via SGD)
  3. Update  $\lambda \leftarrow \lambda + \alpha \frac{d\bar{\mathcal{L}}}{d\lambda}(\tau^*, \theta^*, \lambda)$

- Can be interpreted as constrained trajectory optimization method
- Can be interpreted as imitation of an optimum control excerpter, since step 2 is just supervised learning
- The optimal control “teacher” adapts to the learner, and avoids action that the learner can’t mimic

# Learn a policy by Imitating optimal control in a constrained opt.

## General guided policy search scheme

1. Optimize  $p(\tau)$  with respect to some surrogate  $\tilde{c}(\mathbf{x}_t, \mathbf{u}_t)$
2. Optimize  $\theta$  with respect to some supervised objective
3. Increment or modify dual variables  $\lambda$

## Need to choose:

- ✓ Form of  $p(\tau)$  or  $\tau$  (if deterministic)
- ✓ Optimization method for  $p(\tau)$  or  $\tau$  (if deterministic)
- ✓ Surrogate  $\tilde{c}(\mathbf{x}_t, \mathbf{u}_t)$
- ✓ Supervised objective for  $\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)$

# Learn a policy by Imitating optimal control in a constrained opt.

## Deterministic case

$$\begin{aligned} & \min_{\tau, \theta} c(\tau) \\ & \text{s.t. } \mathbf{u}_t = \pi_{\theta}(\mathbf{x}_t) \\ & \bar{\mathcal{L}}(\tau, \theta, \lambda) = c(\tau) + \underbrace{\sum_{t=1}^T \lambda_t (\pi_{\theta}(\mathbf{x}_t) - \mathbf{u}_t) + \sum_{t=1}^T \rho_t (\pi_{\theta}(\mathbf{x}_t) - \mathbf{u}_t)^2}_{\tilde{c}(\tau)} \end{aligned}$$

- 
1. Optimize  $\tau$  with respect to some surrogate  $\tilde{c}(\tau)$
  2. Optimize  $\theta$  with respect to some supervised objective
  3. Increment or modify dual variables  $\lambda$

# Learn a policy by Imitating optimal control in a constrained opt.

Deterministic case (with multiple trajectories)

$$\begin{aligned} & \min_{\tau_1, \dots, \tau_N, \theta} \sum_{i=1}^N c(\tau_i) \\ & \text{s.t. } \mathbf{u}_{t,i} = \pi_{\theta}(\mathbf{x}_{t,i}) \text{ for all } i \text{ and } t \\ \bar{\mathcal{L}}(\tau_i, \theta, \lambda) &= \underbrace{c(\tau_i) + \sum_{t=1}^T \lambda_t (\pi_{\theta}(\mathbf{x}_{t,i}) - \mathbf{u}_{t,i}) + \sum_{t=1}^T \rho_t (\pi_{\theta}(\mathbf{x}_{t,i}) - \mathbf{u}_{t,i})^2}_{\tilde{c}(\tau_i)} \end{aligned}$$

- 
1. Optimize each  $\tau_i$  *in parallel* with respect to  $\tilde{c}(\tau_i)$
  2. Optimize  $\theta$  with respect to some supervised objective
  3. Increment or modify dual variables  $\lambda$

# Learn a policy by Imitating optimal control in a constrained opt.

## Stochastic (Gaussian) case (Guided Policy Search)

$$\begin{aligned} \min_p \quad & \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)} [c(\mathbf{x}_t, \mathbf{u}_t)] \\ \text{s. t.} \quad & D_{KL}(p(\tau) || \bar{p}(\tau)) \leq \epsilon \\ \text{s. t.} \quad & p(\mathbf{u}_t | \mathbf{x}_t) = \pi_{\theta}(\mathbf{u}_t | \mathbf{x}_t) \end{aligned}$$

Imitation learning: local policy  $\rightarrow$  global policy

where  $D_{KL}(p(\tau) || \bar{p}(\tau)) = E_{p(\mathbf{x}_t, \mathbf{u}_t)} [-\log \bar{p}(\mathbf{u}_t | \mathbf{x}_t) - \mathcal{H}(p(\mathbf{u}_t | \mathbf{x}_t))]$

$$p(\mathbf{u}_t | \mathbf{x}_t) = N(K_t(\mathbf{x}_t - \hat{\mathbf{x}}_t) + \mathbf{k}_t + \hat{\mathbf{u}}_t, \Sigma_t)$$

- 
1. Optimize  $p(\tau)$  with respect to some surrogate  $\tilde{c}(\mathbf{x}_t, \mathbf{u}_t)$

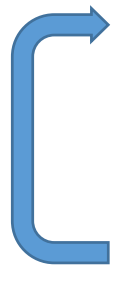
$$\min_p \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)} [\tilde{c}(\mathbf{x}_t, \mathbf{u}_t)] \quad \text{s. t. } D_{KL}(p(\tau) || \bar{p}(\tau)) < \epsilon$$

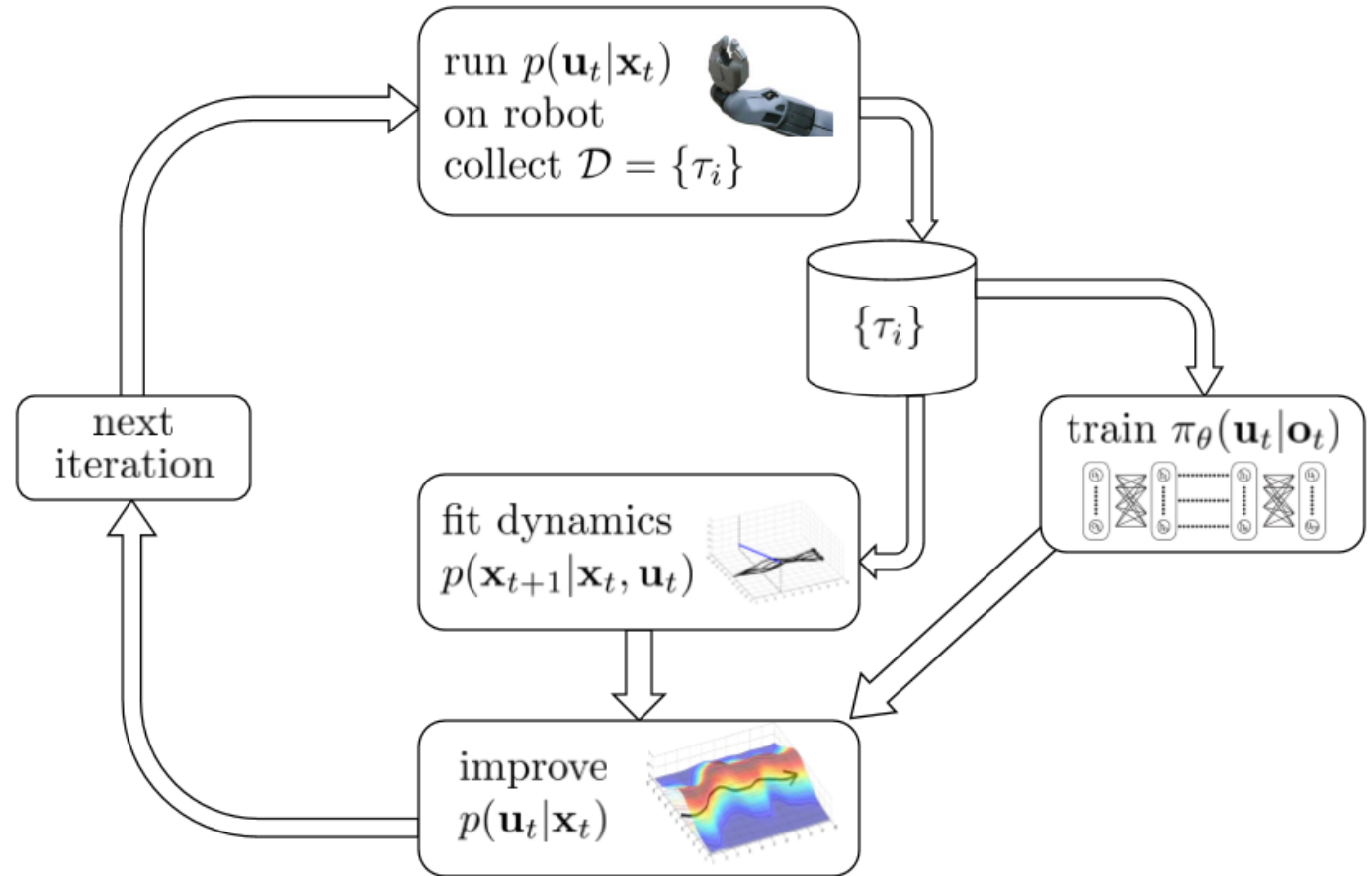
2. Optimize  $\theta$  with respect to some supervised objective
3. Increment or modify dual variables  $\lambda$

# Learn a policy by Imitating optimal control in a constrained opt.

## Stochastic (Gaussian) case (Guided Policy Search)

$$\begin{aligned} \min_{p, \theta} \quad & \sum_{t=1}^T \mathbb{E}_{p(\mathbf{x}_t, \mathbf{u}_t)} [c(\mathbf{x}_t, \mathbf{u}_t)] \\ \text{s.t.} \quad & D_{KL}(p(\tau) || \bar{p}(\tau)) \leq \epsilon \\ \text{s.t.} \quad & p(\mathbf{u}_t | \mathbf{x}_t) = \pi_{\theta}(\mathbf{u}_t | \mathbf{x}_t) \end{aligned}$$

- 
1. Optimize  $p(\tau)$
  2. Optimize  $\theta$
  3. Update dual variables  $\lambda$



# Lean a policy by Imitating optimal control in a constrained opt.

“Learning Neural Network Policies with Guided Policy Search under Unknown Dynamics”

- Propose an algorithm for optimizing trajectories under unknown dynamics.
- Combine the trajectory optimization method with guided policy search.
- Guided policy search optimizes policies by using trajectory optimization in an iterative fashion,
  - ✓ trajectory is optimized to minimize cost and match the policy.
  - ✓ policy is optimized to match the trajectory, and
- By turning policy optimization into a supervised problem, GPS algorithms can train complex policies with thousands of parameters,
- and since our trajectory optimization algorithm exploits the structure of linear-Gaussian controllers, it can optimize the individual trajectories with fewer samples than general-purpose model-free methods.
  - As a result, the combined approach can learn complex policies that are difficult to train with prior methods

# Learn a policy by Imitating optimal control in a constrained opt.

“Learning Neural Network Policies with Guided Policy Search under Unknown Dynamics”

Objective function:

$$\min_{\theta, p(\tau)} E_{p(\tau)}[\ell(\tau)] \text{ s.t. } D_{\text{KL}}(p(\mathbf{x}_t)\pi_{\theta}(\mathbf{u}_t|\mathbf{x}_t)||p(\mathbf{x}_t, \mathbf{u}_t)) = 0 \quad \forall t.$$

Lagrangian of this problem:

$$\mathcal{L}_{\text{GPS}}(\theta, p, \lambda) = E_{p(\tau)}[\ell(\tau)] + \sum_{t=1}^T \lambda_t D_{\text{KL}}(p(\mathbf{x}_t)\pi_{\theta}(\mathbf{u}_t|\mathbf{x}_t)||p(\mathbf{x}_t, \mathbf{u}_t)).$$

- The GPS Lagrangian is minimized with respect to  $\theta$  and  $p(\tau)$  in alternating fashion, with dual variables  $\lambda_t$  updated to enforce constraint satisfaction.
  - ✓ Step1: Optimizing  $\mathcal{L}_{\text{GPS}}(\theta, p, \lambda)$  with respect to trajectory distribution  $p(\tau)$  corresponds to trajectory optimization
  - ✓ Step2: Optimizing  $\mathcal{L}_{\text{GPS}}(\theta, p, \lambda)$  with respect to parameter  $\theta$  (for policy) corresponds to supervised policy optimization to minimize the weighted sum of KL-divergence
  - ✓ Step3: The constrained GPS method also uses dual gradient descent to update the dual variables (the paper does not update  $\lambda$  but just schedule for better performance).
- ❖ Optimizing the trajectory  $p(\tau)$  automatically provides the optimum local policy  $p(\mathbf{u}_t|\mathbf{x}_t)$  due to the linear LQR structure (optimization is conducted by iLQG i.e., linear dynamic theory)



# Learn a policy by Imitating optimal control in a constrained opt.

“Learning Neural Network Policies with Guided Policy Search under Unknown Dynamics”

---

**Algorithm 1** Guided policy search with unknown dynamics

---

- 1: **for** iteration  $k = 1$  to  $K$  **do**
  - 2:   Generate samples  $\{\tau_i^j\}$  from each linear-Gaussian controller  $p_i(\tau)$  by performing rollouts
  - 3:   Fit the dynamics  $p_i(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$  to the samples  $\{\tau_i^j\}$
  - 4:   Minimize  $\sum_{i,t} \lambda_{i,t} D_{\text{KL}}(p_i(\mathbf{x}_t) \pi_\theta(\mathbf{u}_t|\mathbf{x}_t) \| p_i(\mathbf{x}_t, \mathbf{u}_t))$  with respect to  $\theta$  using samples  $\{\tau_i^j\}$
  - 5:   Update  $p_i(\mathbf{u}_t|\mathbf{x}_t)$  using the algorithm in Section 3 and the supplementary appendix
  - 6:   Increment dual variables  $\lambda_{i,t}$  by  $\alpha D_{\text{KL}}(p_i(\mathbf{x}_t) \pi_\theta(\mathbf{u}_t|\mathbf{x}_t) \| p_i(\mathbf{x}_t, \mathbf{u}_t))$
  - 7: **end for**
  - 8: **return** optimized policy parameters  $\theta$
- 

(Learning dynamics)  
(Imitation learning)  
(Trajectory optimization)  
(Update the dual variables)

# Learn a policy by Imitating optimal control in a constrained opt.

“Learning Neural Network Policies with Guided Policy Search under Unknown Dynamics”

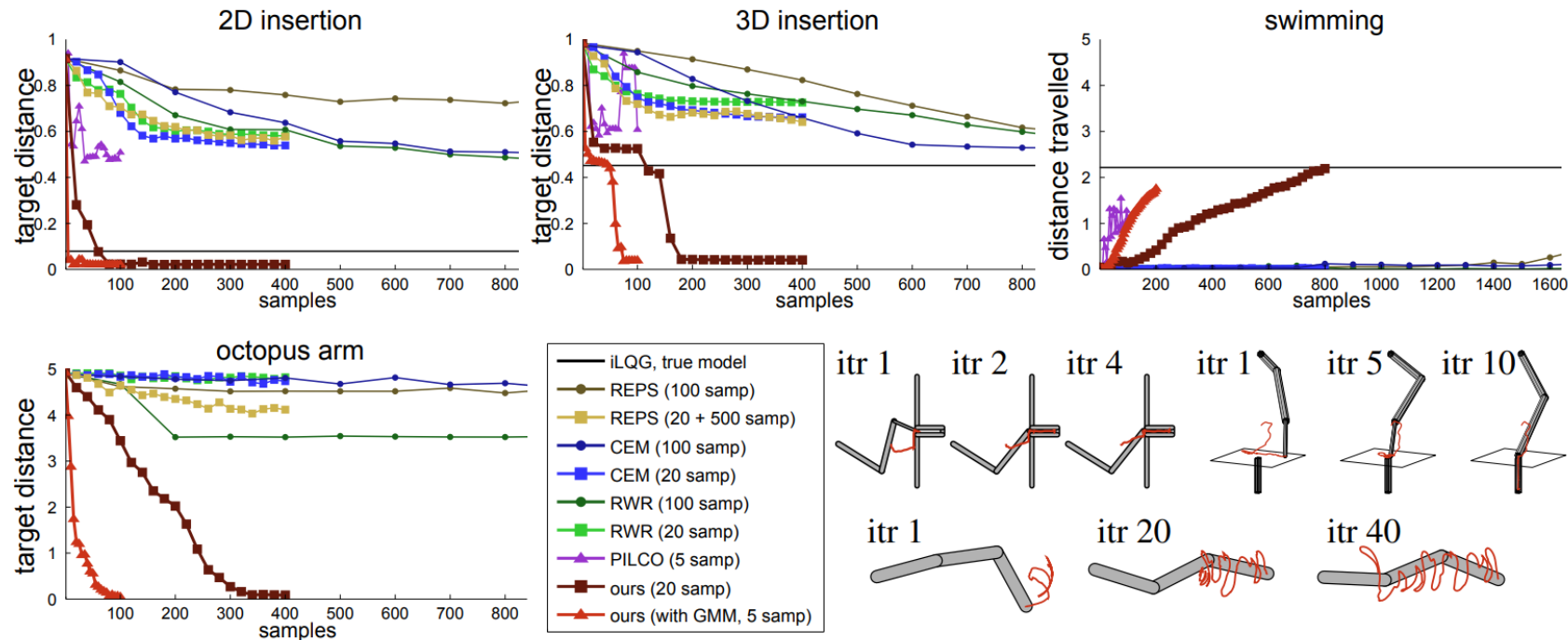


Figure 1: Results for learning linear-Gaussian controllers for 2D and 3D insertion, octopus arm, and swimming. Our approach uses fewer samples and finds better solutions than prior methods, and the GMM further reduces the required sample count. Images in the lower-right show the last time step for each system at several iterations of our method, with red lines indicating end effector trajectories.

- The improved performance of our method is due in part to the use of stronger assumptions about the task, compared to general policy search methods: **time-varying linear Gaussians** are a reasonable local approximation for the dynamics.
- While this assumption is sensible for physical systems, it would require additional work to extend to hybrid discrete-continuous tasks.

# Learn a policy by Imitating optimal control in a constrained opt.

“End-to-End Training of Deep Visuomotor Policies”

$$\min_{p, \pi_\theta} E_p[\ell(\tau)] \text{ s.t. } p(\mathbf{u}_t | \mathbf{x}_t) = \pi_\theta(\mathbf{u}_t | \mathbf{x}_t) \quad \forall \mathbf{x}_t, \mathbf{u}_t, t,$$

$$\mathcal{L}_\theta(\theta, p) = \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t)] + E_{p(\mathbf{x}_t) \pi_\theta(\mathbf{u}_t | \mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] - E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^\theta(\theta, p)$$

$$\mathcal{L}_p(p, \theta) = \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t)] + E_{p(\mathbf{x}_t) \pi_\theta(\mathbf{u}_t | \mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] - E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^p(\theta, p),$$

$$\theta \leftarrow \arg \min_{\theta} \sum_{t=1}^T E_{p(\mathbf{x}_t) \pi_\theta(\mathbf{u}_t | \mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^\theta(\theta, p)$$

$$p \leftarrow \arg \min_p \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t) - \lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^p(p, \theta)$$

$$\lambda_{\mathbf{x}_t, \mathbf{u}_t} \leftarrow \lambda_{\mathbf{x}_t, \mathbf{u}_t} + \alpha \nu_t (\pi_\theta(\mathbf{u}_t | \mathbf{x}_t) p(\mathbf{x}_t) - p(\mathbf{u}_t | \mathbf{x}_t) p(\mathbf{x}_t)).$$

# Learn a policy by Imitating optimal control in a constrained opt.

## “End-to-End Training of Deep Visuomotor Policies”

- The proposed learn policies that map raw image observations directly to torques at the robot's motors.
  - The policy is represented by deep convolutional neural networks (Control using partial observation)
  - The policy is trained using a guided policy search method, which transforms policy search into supervised learning (imitation learning)
  - The label for supervision is achieved using trajectory-centric reinforcement learning method (Trajectory optimization)

# Learn a policy by Imitating optimal control in a constrained opt.

## “End-to-End Training of Deep Visuomotor Policies”

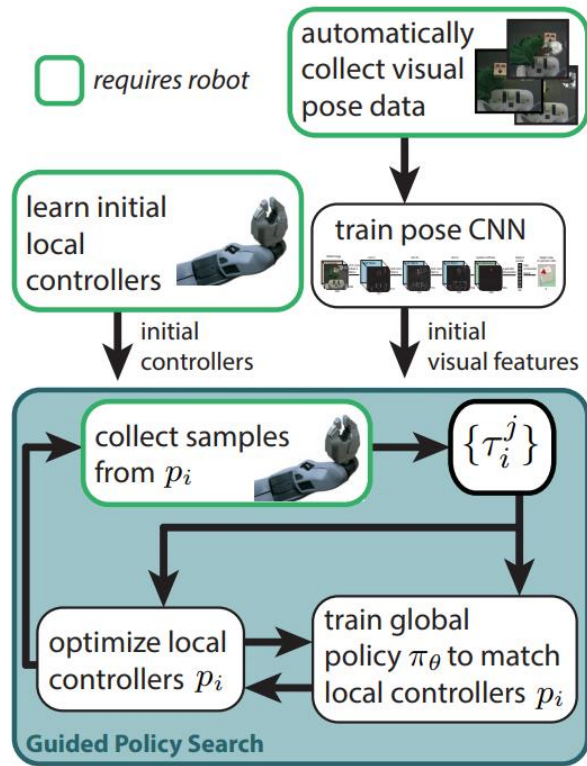


Figure 3: Diagram of our approach, including the main guided policy search phase and initialization phases.

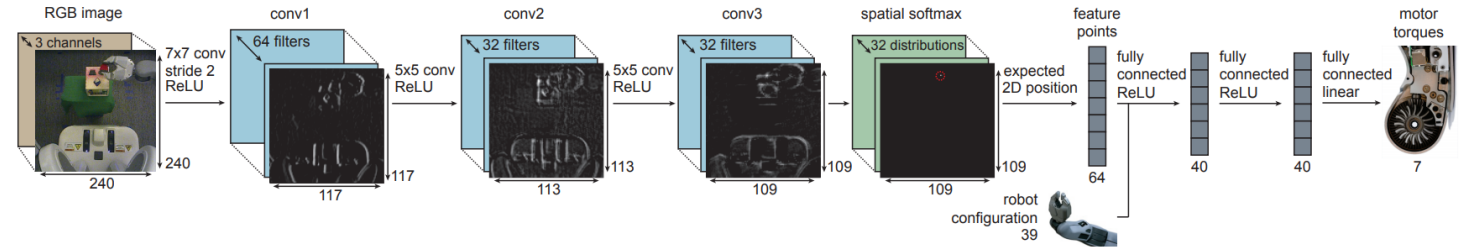


Figure 2: Visuomotor policy architecture. The network contains three convolutional layers, followed by a spatial softmax and an expected position layer that converts pixel-wise features to feature points, which are better suited for spatial computations. The points are concatenated with the robot configuration, then passed through three fully connected layers to produce the torques.

# Lean a policy by Imitating optimal control in a constrained opt.

## “End-to-End Training of Deep Visuomotor Policies”

Objective function for the learning problem is defined as:

$$\min_{p, \pi_\theta} E_p[\ell(\tau)] \text{ s.t. } p(\mathbf{u}_t|\mathbf{x}_t) = \pi_\theta(\mathbf{u}_t|\mathbf{x}_t) \quad \forall \mathbf{x}_t, \mathbf{u}_t, t,$$

Largrangian of the problem is defined as:

$$\mathcal{L}_\theta(\theta, p) = \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t)] + E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] - E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^\theta(\theta, p)$$

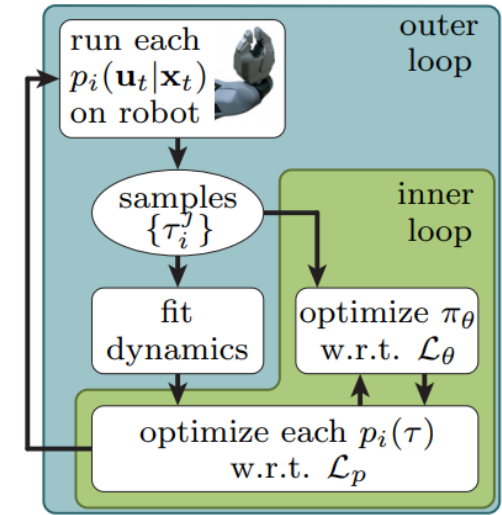
$$\mathcal{L}_p(p, \theta) = \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t)] + E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] - E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^p(\theta, p),$$

Dual descent with alternating primal minimization is then described by the following steps:

$$\theta \leftarrow \arg \min_{\theta} \sum_{t=1}^T E_{p(\mathbf{x}_t)\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)}[\lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^\theta(\theta, p)$$

$$p \leftarrow \arg \min_p \sum_{t=1}^T E_{p(\mathbf{x}_t, \mathbf{u}_t)}[\ell(\mathbf{x}_t, \mathbf{u}_t) - \lambda_{\mathbf{x}_t, \mathbf{u}_t}] + \nu_t \phi_t^p(p, \theta)$$

$$\lambda_{\mathbf{x}_t, \mathbf{u}_t} \leftarrow \lambda_{\mathbf{x}_t, \mathbf{u}_t} + \alpha \nu_t (\pi_\theta(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t) - p(\mathbf{u}_t|\mathbf{x}_t)p(\mathbf{x}_t)).$$





# Learn a policy by Imitating optimal control in a constrained opt.

## “End-to-End Training of Deep Visuomotor Policies”

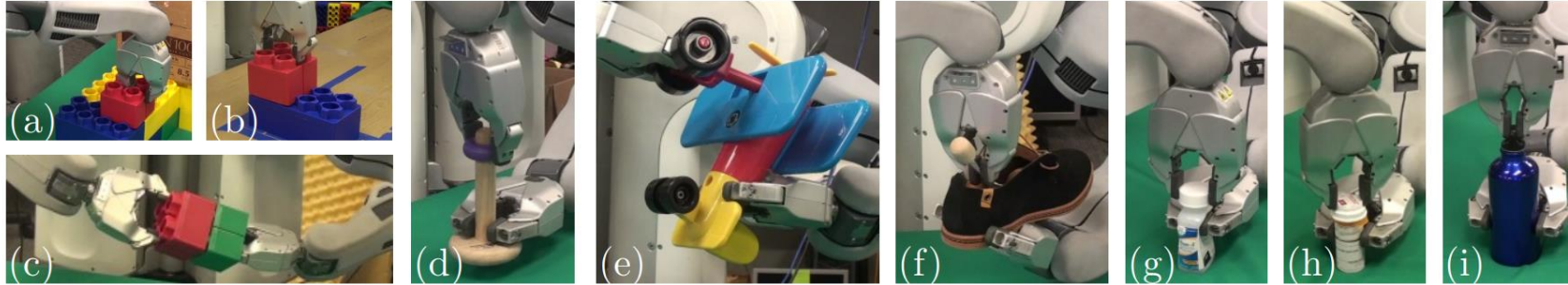


Figure 6: Tasks for linear-Gaussian controller evaluation: (a) stacking lego blocks on a fixed base, (b) onto a free-standing block, (c) held in both gripper; (d) threading wooden rings onto a peg; (e) attaching the wheels to a toy airplane; (f) inserting a shoe tree into a shoe; (g,h) screwing caps onto pill bottles and (i) onto a water bottle.

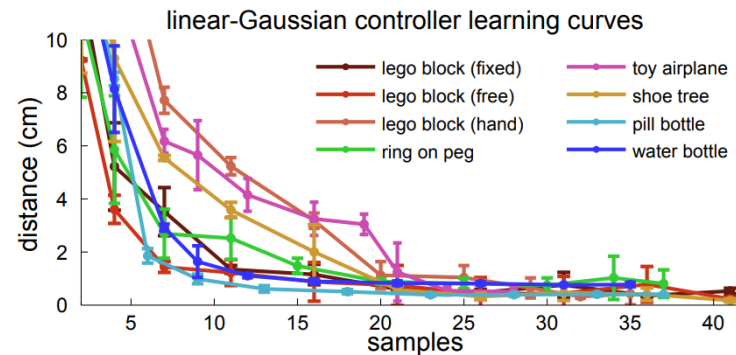


Figure 7: Distance to target point during training of linear-Gaussian controllers. The actual target may differ due to perturbations. Error bars indicate one standard deviation.

# Lean a policy by Imitating optimal control in a constrained opt.

## “PLATO: Policy Learning using Adaptive Trajectory Optimization”

- When the policy is trained to process raw sensory inputs, such as images and depth maps, it can also acquire a strategy that combines perception and control.
  - ✓ Representation learning and Policy learning
- Effectively processing such complex inputs requires an expressive policy class, such as a large neural network. These high-dimensional policies are difficult to train, **especially when learning to control safety-critical systems**.
- The paper proposes PLATO, a continuous, reset-free reinforcement learning algorithm that trains complex control policies with supervised learning, using model-predictive control (MPC) to generate the supervision
- PLATO uses an adaptive training method **to modify the behavior of MPC** to gradually match the learned policy in order to generate training samples at states that are likely to be visited by the learned policy
  - (Adaptive) MPC expert produces supervision that leads to good long-horizon performance of the resulting policy.
  - (Adaptive) MPC can still avoid dangerous on-policy actions in unexpected situations during training.



# Lean a policy by Imitating optimal control in a constrained opt.

“PLATO: Policy Learning using Adaptive Trajectory Optimization”

The role of MPC as a supervision:

- At training time, MPC chooses actions that make a tradeoff between succeeding at the task and matching the behavior of the current policy.
- By gradually adapting to the policy, MPC ensures that the states visited during training will allow the policy to learn good long-horizon performance.
- MPC makes use of full state information, which could be obtained, for example, by instrumenting the environment at training time.
- The final policy, however, is trained to mimic the MPC actions using **only the observations** available to the robot, which makes it possible to run the resulting policy at test time without any instrumentation.

# Lean a policy by Imitating optimal control in a constrained opt.

“PLATO: Policy Learning using Adaptive Trajectory Optimization”

Naïve Approach:

- One naïve approach to learn a policy from a computational teacher such as MPC would be to generate a training set with MPC, and then train the policy with supervised learning to maximize the log-likelihood of this dataset.
  - ✓ The teacher can safely choose robust, near-optimal trajectories.
  - ✓ However, this type of supervision ignores the fact that the state distribution for the teacher and that of the learner are different.
  - Formally, the distribution of states at test time will not match the distribution at training time, and we therefore cannot expect good long-horizon performance from the learned policy.

# Lean a policy by Imitating optimal control in a constrained opt.

“PLATO: Policy Learning using Adaptive Trajectory Optimization”

PLATO resolves this issues:

- PLATO uses an adaptive MPC teacher that modifies its actions in order to bring the state distribution in the training data closer to that of the learned policy, while still producing robust trajectories and reacting intelligently to unexpected perturbations that cannot be handled by a partially trained policy.
- To that end, the teacher generates actions at each time step  $t$  from a controller obtained by optimizing the following objective:

$$\pi_{\lambda}^t(\mathbf{u}|\mathbf{x}_t, \theta) \leftarrow \min_{\pi} J_t(\pi|\mathbf{x}_t) + \lambda D_{KL} \left( \overset{\text{MPC label}}{\pi(\mathbf{u}|\mathbf{x}_t)} || \pi_{\theta}(\mathbf{u}|\mathbf{o}_t) \right)$$

- ✓ The only difference with ordinary MPC is the inclusion of KL-divergence
- ✓ The particular MPC algorithm used is based on iterative iLQG, using a maximum entropy variant that produces linear-Gaussian stochastic controllers of form

$$\pi(\mathbf{u}|\mathbf{x}_t) = N(\mathbf{K}_t\mathbf{x}_t + \mathbf{k}_t, \Sigma_t)$$

# Lean a policy by Imitating optimal control in a constrained opt.

“PLATO: Policy Learning using Adaptive Trajectory Optimization”

## Adaptive MPC teacher:

- choose the actions at each time step according to an MPC policy  $\pi_\lambda^t$  (teacher policy) that minimizes the expected long-term sum of costs  $J_t(\pi_\lambda^t|\mathbf{x}_t)$  but only greedily minimizes the KL-divergence against  $\pi_\theta$  at the current time step  $t$ , where the observation  $\mathbf{o}_t$  is already available, resulting in the objective in Equation

$$\pi_\lambda^t(\mathbf{u}|\mathbf{x}_t, \theta) \leftarrow \min_{\pi} J_t(\pi|\mathbf{x}_t) + \lambda D_{KL}(\pi(\mathbf{u}|\mathbf{x}_t) || \pi_\theta(\mathbf{u}|\mathbf{o}_t))$$

- Since MPC reoptimizes the local policy at each time step, this method produces a sequence of policies  $\pi_\lambda^{1:T}$ , each of which is optimized with respect to its long-horizon cost and immediate disagreement with  $\pi_\theta$ .

## Training the learner's policy

- Want the learner's policy  $\pi_\theta(\mathbf{u}|\mathbf{o}_t)$  to approach the optimal policy  $\pi^*(\mathbf{u}|\mathbf{x}_t)$
- Estimate a (locally) optimal policy  $\pi^*$  at each state  $\mathbf{x}_t$  with iLQG, simply by repeating the optimization at each time step but excluding the KL-divergence  $t$
- During the supervised learning phase, minimize the KL-divergence between the learner  $\pi_\theta$  and the precomputed near-optimum policies  $\pi^*$  at the observations stored in the data set

$$\theta \leftarrow \operatorname{argmin}_{\theta} \sum_{(\mathbf{x}_t, \mathbf{o}_t)} D_{KL}(\pi_\theta(\mathbf{u}|\mathbf{o}_t) || \pi^*(\mathbf{u}|\mathbf{x}_t))$$

# Learn a policy by Imitating optimal control in a constrained opt.

“PLATO: Policy Learning using Adaptive Trajectory Optimization”

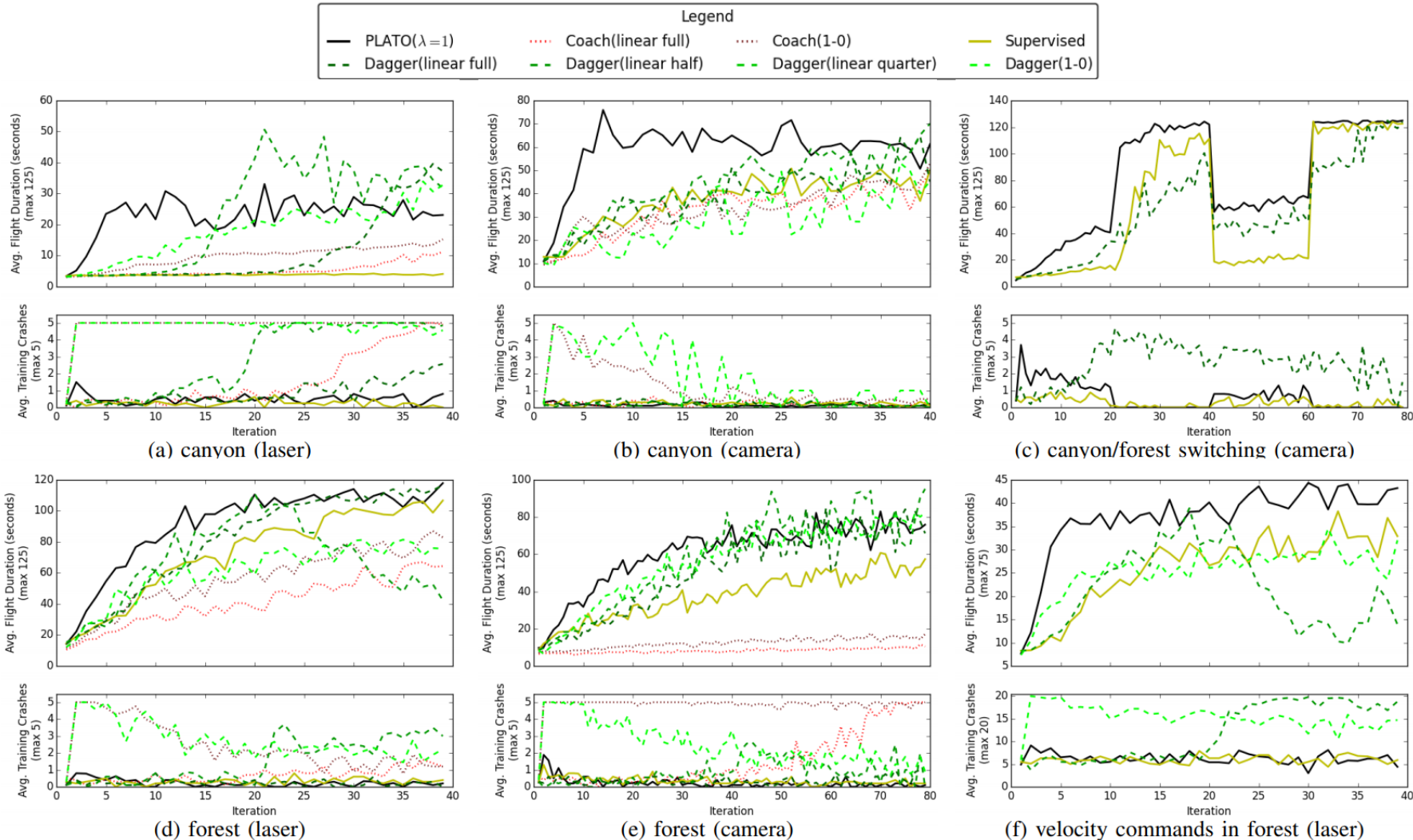


Fig. 2: **Experiments:** We compare PLATO to baseline methods in a winding canyon, a dense forest, and an alternating canyon/forest. For each scenario and learning method, we trained 10 different policies using different random seeds. Each iteration required 2 minutes of flight time. Then for each policy, we evaluated the neural network policy trained at each iteration by flying through the scenario 20 times. Therefore each datapoint corresponds to 200 samples.

# Learn a policy by using data simulated by the learned model

- Use derivative-free (“model-free”) RL algorithms, with the model used to generate synthetic samples
  - Essentially “model-based acceleration” for model-free RL
  - Policy gradient might be more stable (if enough samples are used) because it does not require multiplying many Jacobians

✓ Policy gradient:

$$\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta} \left( a_t^{(i)} \middle| s_t^{(i)} \right) \hat{Q}_t^{\pi}$$

✓ Backprop (partwise) gradient:

$$\nabla_{\theta} J(\theta) = \sum_{i=1}^N \frac{dr_t}{d\mathbf{s}_t} \prod_{t'=2}^t \frac{d\mathbf{s}_{t'}}{d\mathbf{a}_{t'-1}} \frac{d\mathbf{a}_{t'-1}}{d\mathbf{s}_{t'-1}}$$

# Learn a policy by using data simulated by the learned model

- Dyna-Q algorithm integrates both direct RL and model learning, where planning is one-step tabular Q-planning, and learning is one-step tabular Q-learning.
- Real or simulated experience improves both the model via model learning and value function/policy via direct RL.
- Dyna-Q is an example of an RL algorithm that can combine both real and simulated experience, which makes it potentially suitable for applications where real / historical data is insufficient to produce good models.

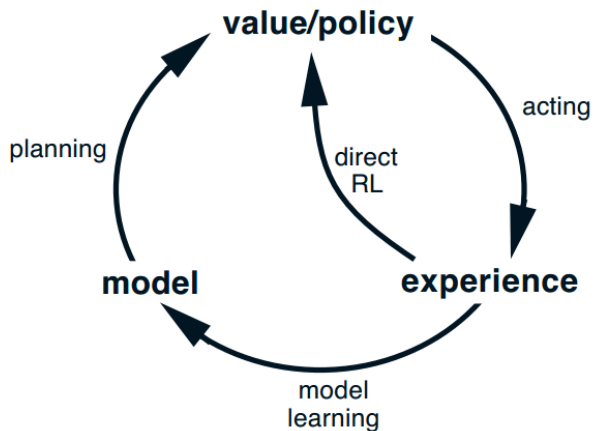


Figure 8.1: Relationships among learning, planning, and acting.

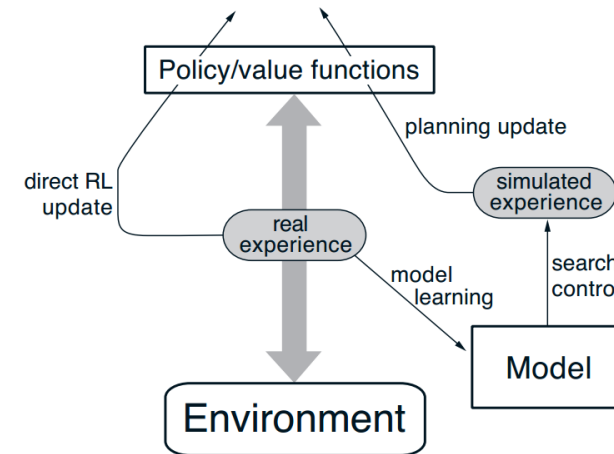


Figure 8.2: The general Dyna Architecture. Real experience, passing back and forth between the environment and the policy, affects policy and value functions in much the same way as does simulated experience generated by the model of the environment.

# Learn a policy by using data simulated by the learned model

**Dyna algorithm:** Online Q-learning algorithm that performs model-free RL with a model

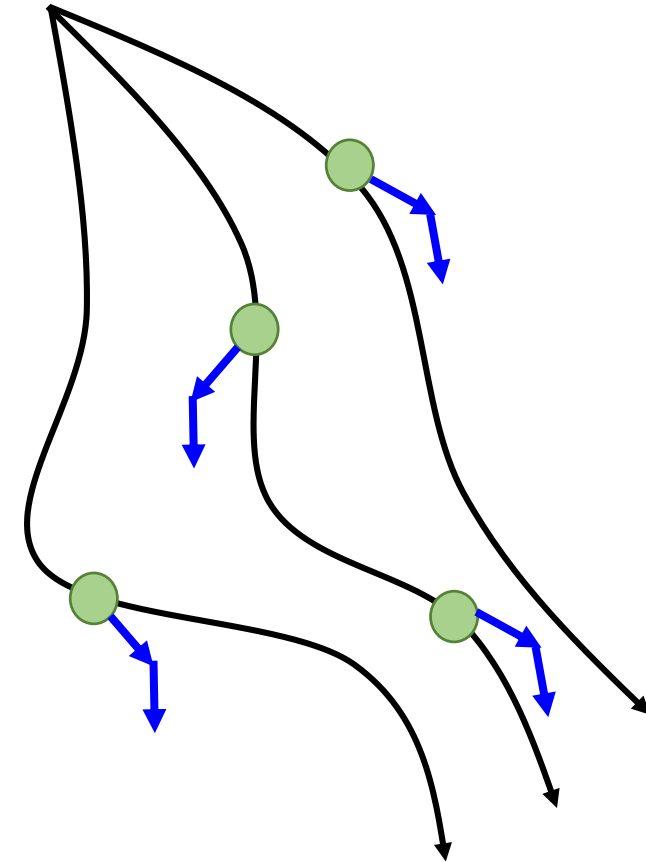
1. Given state  $\mathbf{s}$ , pick action  $\mathbf{a}$  using exploration policy
2. Observe  $\mathbf{s}'$  and  $r$ , to get transition  $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
3. Update model  $\hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$  and  $\hat{r}(\mathbf{s}, \mathbf{a})$  using  $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
4. Q-update:  $Q(\mathbf{s}, \mathbf{a}) \leftarrow Q(\mathbf{s}, \mathbf{a}) + \alpha E_{\mathbf{s}', r} \left[ r + \max_{\mathbf{a}'} Q(\mathbf{s}', \mathbf{a}') - Q(\mathbf{s}, \mathbf{a}) \right]$  (Model based backup + Planning)
5. Repeat  $K$  times:
  - 5.1 sample  $(\mathbf{s}, \mathbf{a}) \sim \mathcal{B}$  from buffer of past states and actions
  - 5.2 Q-update:  $Q(\mathbf{s}, \mathbf{a}) \leftarrow Q(\mathbf{s}, \mathbf{a}) + \alpha E_{\mathbf{s}', r} \left[ r + \max_{\mathbf{a}'} Q(\mathbf{s}', \mathbf{a}') - Q(\mathbf{s}, \mathbf{a}) \right]$  (Model free RL)



# Learn a policy by using data simulated by the learned model

## General “Dyna-style” model-based RL receipt

1. Collect some data, consisting of transitions  $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
  2. Learn model  $\hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$  ( and optimally,  $\hat{r}(\mathbf{s}, \mathbf{a})$  ) (Model learning)
  3. Repeat  $K$  times:
    1. Sample  $\mathbf{s} \sim \mathcal{B}$  from buffer
    2. Choose action  $\mathbf{a}$  (from buffer  $\mathcal{B}$ , from  $\pi$ , or random)
    3. Simulate  $\mathbf{s}' \sim \hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$  ( and  $r = \hat{r}(\mathbf{s}, \mathbf{a})$  )
    4. Train on  $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$  with model-free RL (Model free RL)
    5. (optimal) take N more model-based steps (Model based Planning)
- Only requires short (as few as one step) rollouts from model
  - Algorithm can still see diverse states



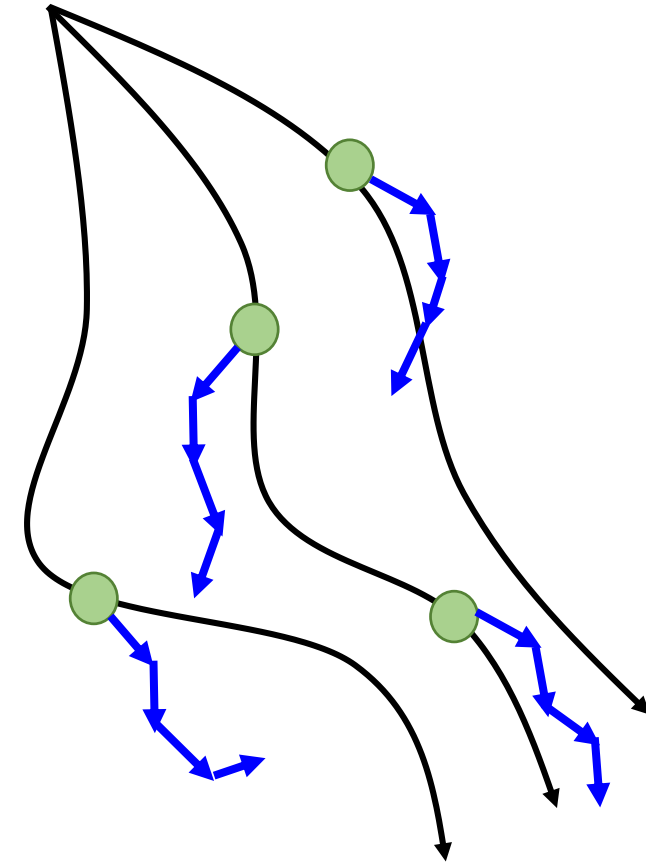
# Learn a policy by using data simulated by the learned model

**Model-Based Acceleration (MBA)**

**Model-Based Value Expansion (MVE)**

**Model-Based Policy Optimization (MBPO)**

1. Take some action  $\mathbf{a}_i$  and observe  $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$ , and add it to  $\mathcal{B}$  (Data from Env.)
2. Sample mini-batch  $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i\}$  from  $\mathcal{B}$  uniformly
3. Use  $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i\}$  to update model  $\hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$  (Model learning)
4. Sample  $\{\mathbf{s}_j\}$  from  $\mathcal{B}$
5. For each  $\mathbf{s}_j$ , perform model-based rollout with  $\mathbf{a} = \pi(\mathbf{s})$  (Model based Planning)
6. Use all transitions  $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$  along rollout to update Q-function (Model free RL)



“Continuous deep Q-learning with model-based acceleration”

“Model-based value expansion”

“When to trust your model: model-based policy optimization”

# Learn a policy by using data simulated by the learned model

## “Continuous deep Q-learning with model-based acceleration”

- Dyna-Q methods have focused on relatively small, discrete domains. The proposed method is extended into a variant of Dyna-Q to achieve substantially faster learning on a range of continuous control tasks with complex neural network policies
- The paper proposes two complementary techniques for improving the efficiency of model free reinforcement learning
- First, the study proposes a continuous variant of the Q-learning algorithm, called normalized advantage functions (NAF), as an alternative to the more commonly used policy gradient and actor-critic methods.
  - NAF representation allows us to apply Q-learning with experience replay to continuous tasks, and substantially improves performance on a set of simulated robotic control tasks.
- The study explores the use of learned models for accelerating model-free reinforcement learning
  - Propose an alternative approach to incorporating learned models into our continuous-action Q-learning algorithm based on *imagination rollouts*: on-policy samples generated under the learned model, analogous to the Dyna-Q method (Sutton, 1990).

# Learn a policy by using data simulated by the learned model

## “Continuous deep Q-learning with model-based acceleration”

### 1. Continuous Q-learning with Normalized Advantage Functions

- Normalized advantage functions (NAF) represents Q-function  $Q(\mathbf{x}_t, \mathbf{u}_t)$  in Q-learning in such a way that its maximum,  $\operatorname{argmax}_{\mathbf{u}} Q(\mathbf{x}_t, \mathbf{u}_t)$  can be determined easily and analytically during the Q-learning update.
- The study use neural network that separately outputs a value function terms  $V(x_t)$  and an advantage term  $A(x_t, u_t)$

$$Q(\mathbf{x}, \mathbf{u}|\theta^Q) = A(\mathbf{x}, \mathbf{u}|\theta^A) + V(\mathbf{x}|\theta^V)$$

$$A(\mathbf{x}, \mathbf{u}|\theta^A) = -\frac{1}{2}(\mathbf{u} - \boldsymbol{\mu}(\mathbf{x}|\theta^\mu))^T \mathbf{P}(\mathbf{x}|\theta^P)(\mathbf{u} - \boldsymbol{\mu}(\mathbf{x}|\theta^\mu))$$

- ✓  $\mathbf{P}(\mathbf{x}|\theta^P) = \mathbf{L}(\mathbf{x}|\theta^P)\mathbf{L}(\mathbf{x}|\theta^P)^T$  is a state-dependent, positive-definite square matrix constructed by the lower triangular matrix  $\mathbf{L}(\mathbf{x}|\theta^P)$  whose entries come from a linear output layer of a NN.
- ✓ Although this representation is restrictive, it guarantees that the action that maximizes the Q-function is always given by  $\boldsymbol{\mu}(\mathbf{x}|\theta^P)$

# Learn a policy by using data simulated by the learned model

## “Continuous deep Q-learning with model-based acceleration”

### 2. Model-Guided Exploration

- Use the learned model to generate good exploratory behaviors using planning or trajectory optimization.
- Utilize the iLQG algorithm to generate good trajectories under the model, and then mix these trajectories together with on-policy experience by appending them to the replay buffer.
- ❖ it's not enough to simply show the algorithm good actions, it must also experience bad actions to understand which actions are better and which are worse.

# Learn a policy by using data simulated by the learned model

## “Continuous deep Q-learning with model-based acceleration”

### 3. Accelerating Learning with Imagination Rollouts

- Incorporating off-policy exploration from good, narrow distributions, such as those induced by iLQG, often does not result in significant improvement for Q-learning.
  - These results suggest that Q-learning, which learns a policy based on minimizing temporal differences, inherently requires noisy on-policy actions to succeed.
- In real-world domains such as robots and autonomous vehicles, this can be undesirable for two reasons:
  - First, it suggests that large amounts of on-policy experience are required in addition to good off-policy samples,
  - Second, it implies that the policy must be allowed to make “its own mistakes” during training, which might involve taking undesirable or dangerous actions that can damage real-world hardware.

# Learn a policy by using data simulated by the learned model

## “Continuous deep Q-learning with model-based acceleration”

### 3. Accelerating Learning with Imagination Rollouts

- One way to avoid these problems while still allowing for a large amount of on-policy exploration is to generate synthetic on-policy trajectories under a learned model.
  - Adding these synthetic samples, referred to as *imagination rollouts*, to the replay buffer effectively augments the amount of experience available for Q-learning.
- Imagination rollouts can suffer from severe bias when the learned model is inaccurate.
  - For example, it is very difficult to train nonlinear neural network models for the dynamics that would actually improve the efficiency of Q-learning when used for imagination rollouts.
  - The study found that using iteratively refitted time-varying linear dynamics produced substantially better results.

# Learn a policy by using data simulated by the learned model

## “Continuous deep Q-learning with model-based acceleration”

### Algorithm 1 Continuous Q-Learning with NAF

```
Randomly initialize normalized Q network  $Q(\mathbf{x}, \mathbf{u}|\theta^Q)$ .
Initialize target network  $Q'$  with weight  $\theta^{Q'} \leftarrow \theta^Q$ .
Initialize replay buffer  $R \leftarrow \emptyset$ .
for episode=1,  $M$  do
  Initialize a random process  $\mathcal{N}$  for action exploration
  Receive initial observation state  $\mathbf{x}_1 \sim p(\mathbf{x}_1)$ 
  for t=1,  $T$  do
    Select action  $\mathbf{u}_t = \mu(\mathbf{x}_t|\theta^\mu) + \mathcal{N}_t$ 
    Execute  $\mathbf{u}_t$  and observe  $r_t$  and  $\mathbf{x}_{t+1}$ 
    Store transition  $(\mathbf{x}_t, \mathbf{u}_t, r_t, \mathbf{x}_{t+1})$  in  $R$ 
    for iteration=1,  $I$  do
      Sample a random minibatch of  $m$  transitions from  $R$ 
      Set  $y_i = r_i + \gamma V'(\mathbf{x}_{i+1}|\theta^{Q'})$ 
      Update  $\theta^Q$  by minimizing the loss:  $L = \frac{1}{N} \sum_i (y_i - Q(\mathbf{x}_i, \mathbf{u}_i|\theta^Q))^2$ 
      Update the target network:  $\theta^{Q'} \leftarrow \tau\theta^Q + (1 - \tau)\theta^{Q'}$ 
    end for
  end for
end for
```

### Algorithm 2 Imagination Rollouts with Fitted Dynamics and Optional iLQG Exploration

```
Randomly initialize normalized Q network  $Q(\mathbf{x}, \mathbf{u}|\theta^Q)$ .
Initialize target network  $Q'$  with weight  $\theta^{Q'} \leftarrow \theta^Q$ .
Initialize replay buffer  $R \leftarrow \emptyset$  and fictional buffer  $R_f \leftarrow \emptyset$ .
Initialize additional buffers  $B \leftarrow \emptyset, B_{old} \leftarrow \emptyset$  with size  $nT$ .
Initialize fitted dynamics model  $\mathcal{M} \leftarrow \emptyset$ .
for episode = 1,  $M$  do
  Initialize a random process  $\mathcal{N}$  for action exploration
  Receive initial observation state  $\mathbf{x}_1$ 
  Select  $\mu'(\mathbf{x}, t)$  from  $\{\mu(\mathbf{x}|\theta^\mu), \pi_t^{iLQG}(\mathbf{u}_t|\mathbf{x}_t)\}$  with probabilities  $\{p, 1 - p\}$ 
  for  $t = 1, T$  do
    Select action  $\mathbf{u}_t = \mu'(\mathbf{x}_t, t) + \mathcal{N}_t$ 
    Execute  $\mathbf{u}_t$  and observe  $r_t$  and  $\mathbf{x}_{t+1}$ 
    Store transition  $(\mathbf{x}_t, \mathbf{u}_t, r_t, \mathbf{x}_{t+1}, t)$  in  $R$  and  $B$ 
    if  $\text{mod}(\text{episode} \cdot T + t, m) = 0$  and  $\mathcal{M} \neq \emptyset$  then
      Sample  $m(\mathbf{x}_i, \mathbf{u}_i, r_i, \mathbf{x}_{i+1}, i)$  from  $B_{old}$ 
      Use  $\mathcal{M}$  to simulate  $l$  steps from each sample
      Store all fictional transitions in  $R_f$ 
    end if
    Sample a random minibatch of  $m$  transitions  $I \cdot l$  times from  $R_f$  and  $I$  times from  $R$ , and update  $\theta^Q, \theta^{Q'}$  as in Algorithm 1 per minibatch.
  end for
  if  $B_f$  is full then
     $\mathcal{M} \leftarrow \text{FitLocalLinearDynamics}(B_f)$  (see Section 5.3)
     $\pi^{iLQG} \leftarrow \text{iLQG\_OneStep}(B_f, \mathcal{M})$  (see appendix)
     $B_{old} \leftarrow B_f, B_f \leftarrow \emptyset$ 
  end if
end for
```

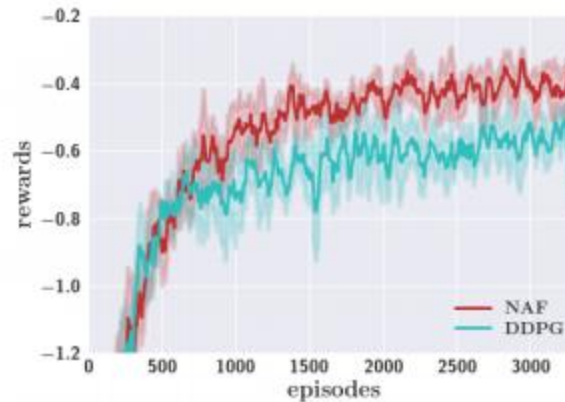


# Learn a policy by using data simulated by the learned model

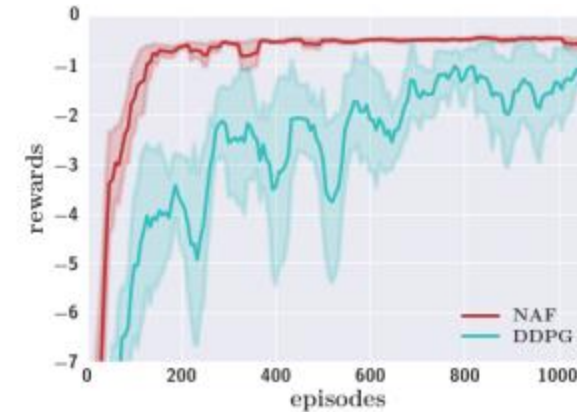
“Continuous deep Q-learning with model-based acceleration”



(a) Example task domains.



(b) NAF and DDPG on multi-target reacher.



(c) NAF and DDPG on peg insertion.

Figure 1. (a) Task domains: top row from left (manipulation tasks: peg, gripper, mobile gripper), bottom row from left (locomotion tasks: cheetah, swimmer6, ant). (b,c) NAF vs DDPG results on three-joint reacher and peg insertion. On reacher, the DDPG policy continuously fluctuates the tip around the target, while NAF stabilizes well at the target.

| Domains  | -    | DDPG        | episodes    | NAF           | episodes    |
|----------|------|-------------|-------------|---------------|-------------|
| Cartpole | -2.1 | -0.601      | 420         | -0.604        | <b>190</b>  |
| Reacher  | -2.3 | -0.509      | 1370        | <b>-0.331</b> | <b>1260</b> |
| Peg      | -11  | -0.950      | 690         | <b>-0.438</b> | <b>130</b>  |
| Gripper  | -29  | 1.03        | 2420        | <b>1.81</b>   | <b>1920</b> |
| GripperM | -90  | -20.2       | 1350        | <b>-12.4</b>  | <b>730</b>  |
| Canada2d | -12  | -4.64       | 1040        | <b>-4.21</b>  | 900         |
| Cheetah  | -0.3 | <b>8.23</b> | <b>1590</b> | 7.91          | 2390        |
| Swimmer6 | -325 | -174        | 220         | <b>-172</b>   | <b>190</b>  |
| Ant      | -4.8 | -2.54       | 2450        | -2.58         | <b>1350</b> |
| Walker2d | 0.3  | <b>2.96</b> | <b>850</b>  | 1.85          | 1530        |

Table 1. Best test rewards of DDPG and NAF policies, and the episodes it requires to reach within 5% of the best value. “-” denotes scores by a random agent.

# Summary

Ways to learn a policy using the learned dynamic model:

- Learn a policy by **Backpropgating dynamic model into policy**
  - Simple but potentially unstable
- Learn a policy by **Imitating optimal control** in a constrained optimization framework
  - An effective planning method should be used
    - ✓ iLQR
    - ✓ MCTS
    - ✓ MPC (nonlinear optimization)
- Learn a policy by **using data simulated** by the learned model (e.g., Dyna)
  - Simple but not as sample-efficient as other methods

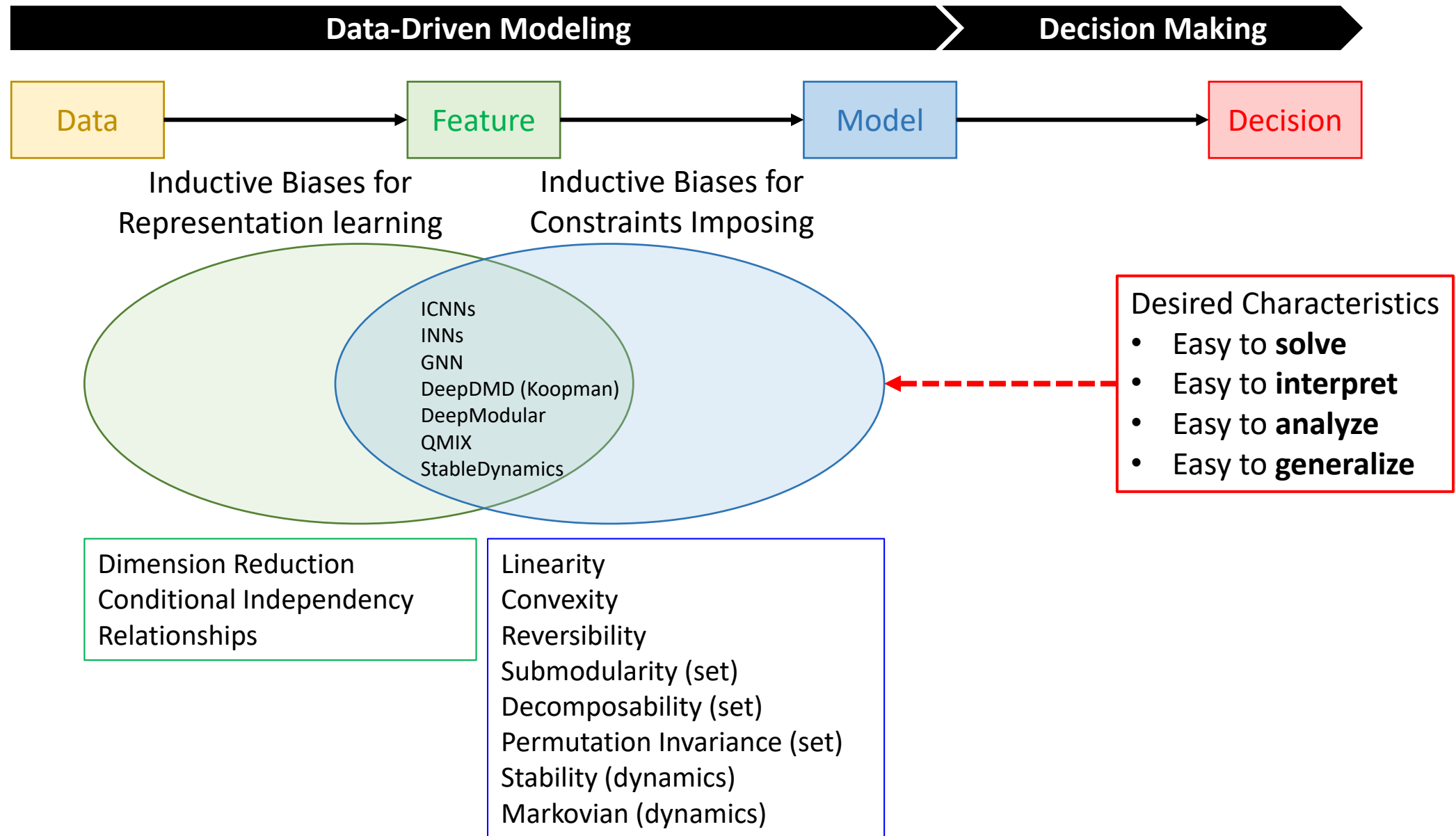
# Summary

## Limitations

- Need some kind of model
  - ✓ Not always available
  - ✓ Sometimes harder to learn than the policy
- Learning the model takes time & data
  - ✓ Need to balance between the expressability and tractability
    - Sometimes expressive model classes are not fast (e.g., neural network)
    - Sometimes fast model classes (linear models) are not expressive
- Learning model relies on additional assumptions
  - Linearizability
  - Continuity
  - Smoothness
  - Etc.

# Summary

## Decision-centric data driven modeling for dynamic systems with functional inductive biases



## MBRL Lecture 4. End-to-End Planning

Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

Planning

Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data

Data-Driven  
Dynamic  
Model

Planning

Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data

Inductive  
Biases on  
Dynamics

Planning

Policy

Model-Free RL

Data

Dynamic  
Model

Policy

The model is implicitly learned while deriving planning strategy

➤ The dynamic model is implicitly considered as constraint while optimizing the actions

✓ “Task-based End-to-end Model Learning in Stochastic Optimization”

✓ “Differentiable MPC for End-to-end Planning and Control”

## MBRL Lecture 5. End-to-End Policy Learning



Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

Planning

Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data

Data-Driven  
Dynamic  
Model

Planning

Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data

Inductive  
Biases on  
Dynamics

Planning

Policy

Model-Free RL

Data

Dynamic  
Model

Policy

Learn simultaneously both the dynamic model and the policy

- ✓ “Learning Continuous Control Policies by Stochastic Value Gradients”
- ✓ “Dream to Control: Learning Behaviors by Latent Imagination”
- ✓ “Graph Networks as Learnable Physics Engines for Inference and Control”

## MBRL Lecture 4. End-to-End Planning and Policy Learning

Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

Planning

Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data

Data-Driven  
Dynamic  
Model

Planning

Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data

Inductive  
Biases on  
Dynamics

Planning

Policy

Model-Free RL

Data

Dynamic  
Model

Policy

The model is implicitly learned while deriving planning strategy

- The dynamic model is implicitly considered as constraint while optimizing the actions
- ✓ “Task-based End-to-end Model Learning in Stochastic Optimization”
- ✓ “Differentiable MPC for End-to-end Planning and Control”

Analytical  
Optimal Control

Data

Analytical  
Dynamic  
Model

Planning

Policy

**2-Stages** Model Based RL  
Model Building + Planning/Control

Data

Data-Driven  
Dynamic  
Model

Planning

Policy

**End-to-End** Model Based RL  
Planning/Control using Dynamics  
as inductive Biases

Data

Inductive  
Biases on  
Dynamics

Planning

Policy

Model-Free RL

Data

Dynamic  
Model

Policy

Learn simultaneously both the dynamic model and the policy

- ✓ “Learning Continuous Control Policies by Stochastic Value Gradients”
- ✓ “Dream to Control: Learning Behaviors by Latent Imagination”
- ✓ “Graph Networks as Learnable Physics Engines for Inference and Control”

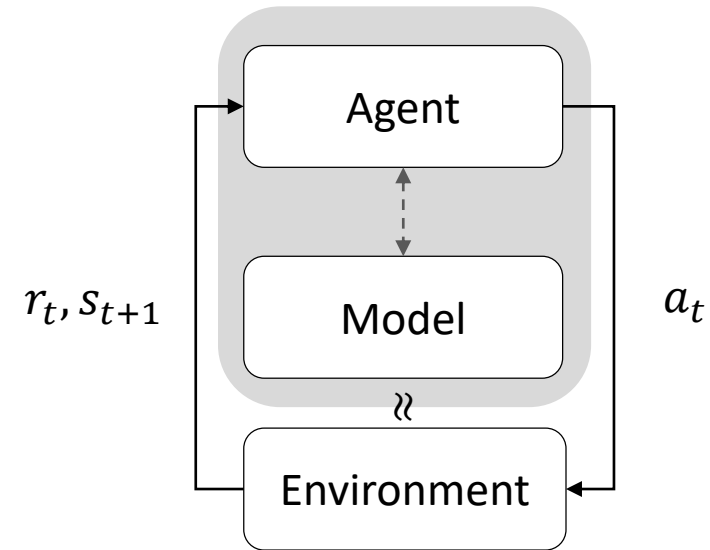
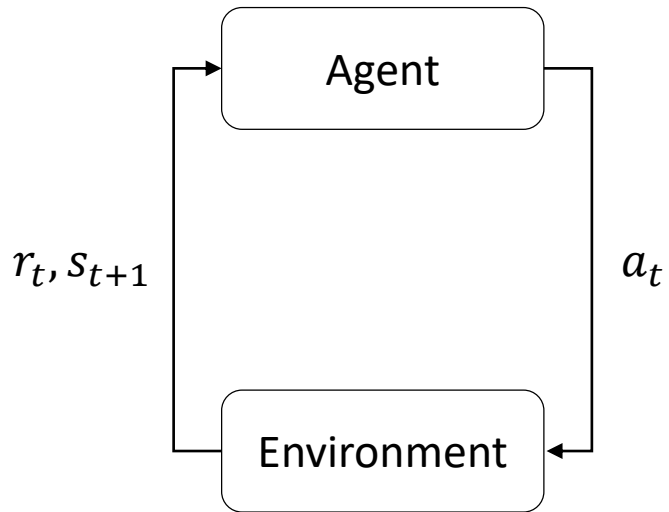
# Contents

---

- OptNet: Differentiable Optimization as a Layer in Neural Networks (ICML 2017)
- Differentiable MPC for End-to-end Planning and Control (NeurIPS 2018)



# Model-free RL vs Model-based RL



| Model-free RL                           |                  | Model-based RL      |
|-----------------------------------------|------------------|---------------------|
| No                                      | Model?           | Yes                 |
| Use policy function $\pi_{\theta}(s_t)$ | Decision-making? | Use planning method |
| End-to-end learning                     | Goodness?        | Sample-efficient    |

# Motivation

---

- On Model Predictive Control (MPC) framework, the agent solves the following:

$$\tau_{1:T}^* = (x_t^*, u_t^*)_{1:T} = MPC(x_{init}; C_t, f_\theta) = \underset{x_{1:T}, u_{1:T}}{\operatorname{argmin}} \sum_{t=1}^T C_t(x_t, u_t) \text{ s.t. } x_{t+1} = f_\theta(x_t, u_t), x_1 = x_{init}$$

- with a learned model  $f_\theta(x, u)$ 
  - The model is usually learned by minimizing the predictive loss  $L(x_{t+1}, f_\theta(x_t, u_t))$
  - Independent of task, dependent of dataset
- **Idea: Treat  $MPC(\cdot)$  as a policy class to cast an end-to-end learning!**
  - The model and the action are derived by minimizing the task loss function  $l(\cdot)$  over  $\theta$

# Overview

---

|                     | Model-free RL       | Model-based RL        | Differentiable MPC                              |
|---------------------|---------------------|-----------------------|-------------------------------------------------|
| Model               | No                  | Yes                   | Yes                                             |
| Thing to be trained | Policy $\pi(\cdot)$ | Model $f(\cdot)$      | Model $f(\cdot)$ (potentially cost $c(\cdot)$ ) |
| Loss function       | Task loss           | Model prediction loss | Task loss                                       |
| Execution           | Policy              | Planning              | Planning                                        |

# Useful When...

---

- Your task is not just minimizing your control objective  $c(x, u)$ 
  - Imitation learning
    - Goal of training: try to find  $\theta$  which minimize  $(\pi_\theta(s) - \pi(s))^2$  where  $\pi(s)$  is generated by an expert
    - In this case, differentiable MPC can be useful
    - Ex) HVAC control, (possibly) WONIK
- BTW, is the argmin operator differentiable? If it is, is the backward fast enough?

## (Partially) YES, it is!

---

- Suppose the optimization problem is QP, which is a convex problem.

$$\begin{aligned}\tau_{1:T}^* = \operatorname{argmin}_{\tau_{1:T}} & \frac{1}{2} \tau_t^T C_t \tau_t + c_t^T \tau_t \\ \text{s.t. } & x_1 = x_{init} \\ & x_{t+1} = F_t \tau_t + f_t\end{aligned}$$

where  $\tau_{1:T}^* = \{x_t, u_t\}_{1:T}$ .

- State KKT conditions of a Lagrangian and take the differentials of them!

# Differentiating the Argmin Operator

- Lagrangian  $L(\tau, \lambda) = \sum_{t=1}^T \left( \frac{1}{2} \tau_t^T C_t \tau_t + c_t^T \tau_t \right) + \sum_{t=0}^{T-1} \lambda_t^T (F_t \tau_t + f_t - x_{t+1})$
- Taking differentials of KKT condition and we can obtain the gradient
  - $\nabla_{C_t} l = \frac{1}{2} (d_{\tau_t}^* \tau_t^{*T} + \tau_t^* d_{\tau_t}^{*T}), \nabla_{c_t} l = d_{\tau_t}^*, \nabla_{x_{init}} l = d_{\lambda_0}^*$
  - $\nabla_{F_t} l = d_{\lambda_{t+1}}^* \tau_t^{*T} + \lambda_{t+1}^* d_{\tau_t}^{*T}, \nabla_{f_t} l = d_{\lambda_t}^*$

where 
$$\begin{bmatrix} \vdots \\ d_{\tau_t}^* \\ d_{\lambda_t}^* \\ \vdots \end{bmatrix} = -K^{-1} \begin{bmatrix} \vdots \\ \nabla_{\tau_t}^* l \\ 0 \\ \vdots \end{bmatrix},$$

Goes to Appendix to see the detail derivation.

$$\overbrace{\begin{bmatrix} \ddots & & & & \\ & C_t & F_t^\top & & \\ & F_t & [-I & 0] & \\ & & \begin{bmatrix} -I \\ 0 \end{bmatrix} & C_{t+1} & F_{t+1}^\top \\ & & & F_{t+1} & \ddots \end{bmatrix}}^K \begin{bmatrix} \vdots \\ \tau_t^* \\ \lambda_t^* \\ \tau_{t+1}^* \\ \lambda_{t+1}^* \\ \vdots \end{bmatrix} = - \begin{bmatrix} \vdots \\ c_t \\ f_t \\ c_{t+1} \\ f_{t+1} \\ \vdots \end{bmatrix}$$

# Differentiable LQR

---

- Consider a discrete-time finite-horizon LQR:

$$\tau_{1:T}^* = \operatorname{argmin}_{\tau_{1:T}} \sum_{t=1}^T \frac{1}{2} \tau_t^T C_t \tau_t + c_t^T \tau_t \text{ s.t. } x_1 = x_{\text{init}}, x_{t+1} = F_t \tau_t + f_t$$

- Note that computing

---

**Module 1** Differentiable LQR

---

(The LQR algorithm is defined in [appendix A](#))

**Input:** Initial state  $x_{\text{init}}$

**Parameters:**  $\theta = \{C, c, F, f\}$

**Forward Pass:**

- 1:  $\tau_{1:T}^* = \text{LQR}_T(x_{\text{init}}; C, c, F, f)$
- 2: Compute  $\lambda_{1:T}^*$  with (7)

▷ Solve (2)

**Backward Pass:**

- 1:  $d_{\tau_{1:T}}^* = \text{LQR}_T(0; C, \nabla_{\tau^*} \ell, F, 0)$  ▷ Solve (9), ideally reusing the factorizations from the forward pass
  - 2: Compute  $d_{\lambda_{1:T}}^*$  with (7)
  - 3: Compute the derivatives of  $\ell$  with respect to  $C, c, F, f$ , and  $x_{\text{init}}$  with (8)
-

# What if Non-Linear Dynamics & Cost?

- Consider a general MPC:

$$\tau_{1:T}^* = \underset{\tau_{1:T}}{\operatorname{argmin}} \sum_t C_{\theta,t}(\tau_t) \text{ s.t. } x_1 = x_{\text{init}}, x_{t+1} = f_{\theta}(\tau_t), \underline{u} \leq u \leq \bar{u}$$

- After obtaining a fixed point by iLQR, use Taylor approximation to form LQR.
- Backward pass is faster than unrolling iLQR.

---

## Module 2 Differentiable MPC

(The MPC algorithm is defined in [appendix A](#))

**Given:** Initial state  $x_{\text{init}}$  and initial control sequence  $u_{\text{init}}$

**Parameters:**  $\theta$  of the objective  $C_{\theta}(\tau)$  and dynamics  $f_{\theta}(\tau)$

### Forward Pass:

- 1:  $\tau_{1:T}^* = \text{MPC}_{T,\underline{u},\bar{u}}(x_{\text{init}}, u_{\text{init}}; C_{\theta}, F_{\theta})$  ▷ Solve [eq. \(10\)](#)
- 2: The solver should reach the fixed point in [\(11\)](#) to obtain approximations to the cost  $H_{\theta}^n$  and dynamics  $F_{\theta}^n$
- 3: Compute  $\lambda_{1:T}^*$  with [\(7\)](#)

### Backward Pass:

- 1:  $\tilde{F}_{\theta}^n$  is  $F_{\theta}^n$  with the rows corresponding to the tight control constraints zeroed
  - 2:  $d_{\tau_{1:T}}^* = \text{LQR}_T(0; H_{\theta}^n, \nabla_{\tau^*} \ell, \tilde{F}_{\theta}^n, 0)$  ▷ Solve [\(19\)](#), ideally reusing the factorizations from the forward pass
  - 3: Compute  $d_{\lambda_{1:T}}^*$  with [\(7\)](#)
  - 4: Differentiate  $\ell$  with respect to the approximations  $H_{\theta}^n$  and  $F_{\theta}^n$  with [\(8\)](#)
  - 5: Differentiate these approximations with respect to  $\theta$  and use the chain rule to obtain  $\partial \ell / \partial \theta$
-



# Drawback

---

- Sometimes the fixed point of iLQR does not exist.
- If this is the case, use unrolling procedure to compute the backward pass.

# Experiment: MPC Solver Performance

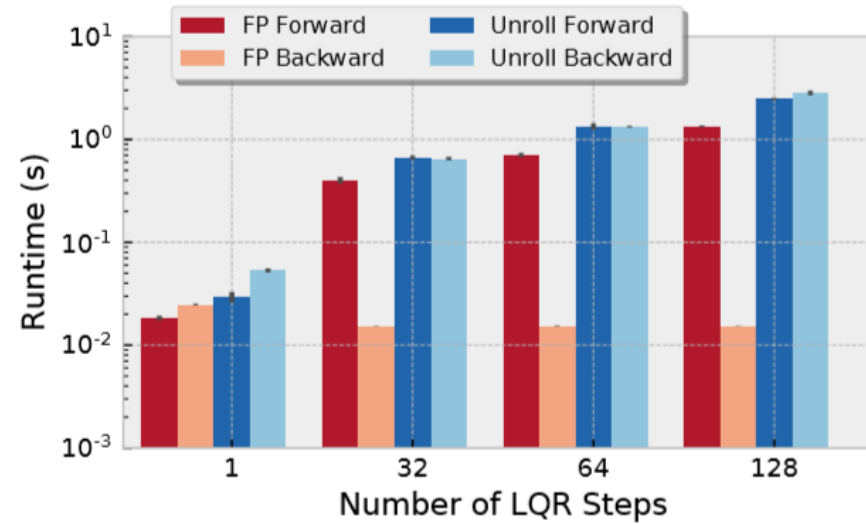


Figure 2: Runtime comparison of fixed point differentiation (FP) to unrolling the iLQR solver (Unroll), averaged over 10 trials.

# Experiment: Imitation Learning with Realizable Expert

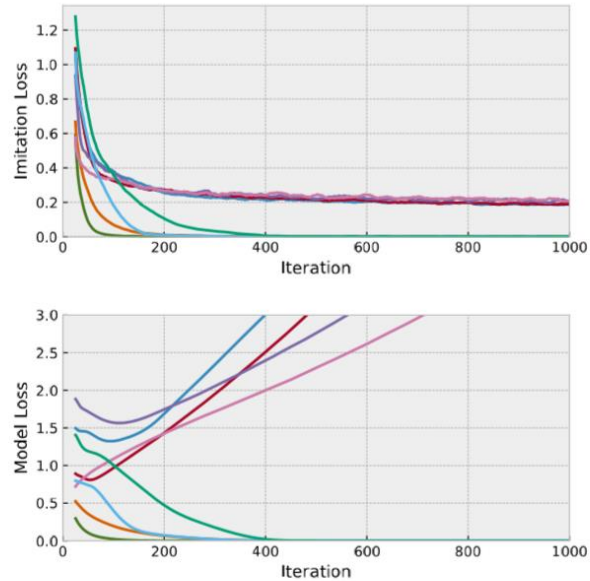


Figure 3: Model and imitation losses for the LQR imitation learning experiments.

## 1. Convex Control (LQR)

$$\mathcal{L}(\theta) = \mathbb{E}_x \left[ \left\| \tau_{1:T}(x; \theta) - \tau_{1:T}(x; \hat{\theta}) \right\|_2^2 \right]$$

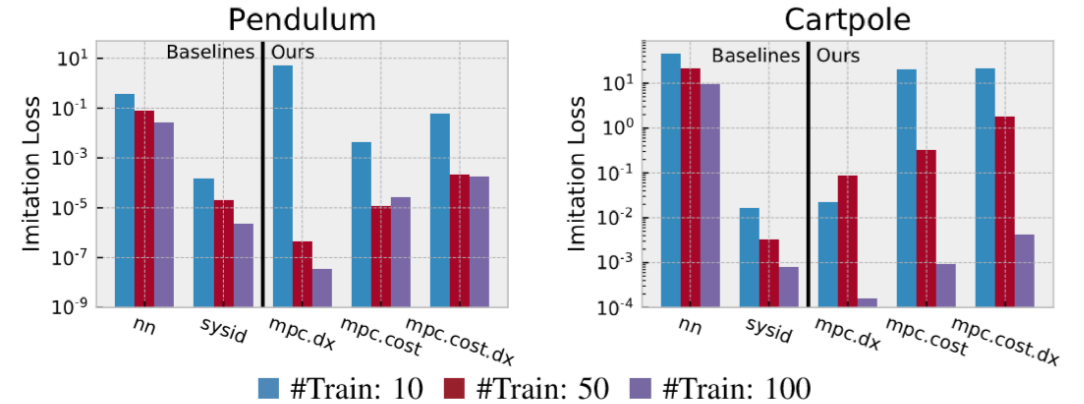


Figure 4: Learning results on the (simple) pendulum and cartpole environments. We select the best validation loss observed during the training run and report the best test loss.

## 2. Non-Convex Control (Pendulum & Cartpole)

$$\mathcal{L}(\theta) = \mathbb{E}_x \left[ \left\| u_{1:T}(x; \theta) - u_{1:T}(x; \hat{\theta}) \right\|_2^2 \right]$$

# Experiment: Imitation Learning with Non-Realizable Expert

- Expert dynamic is outside of real dynamic
- Proposed method shows less imitation loss than MBRL

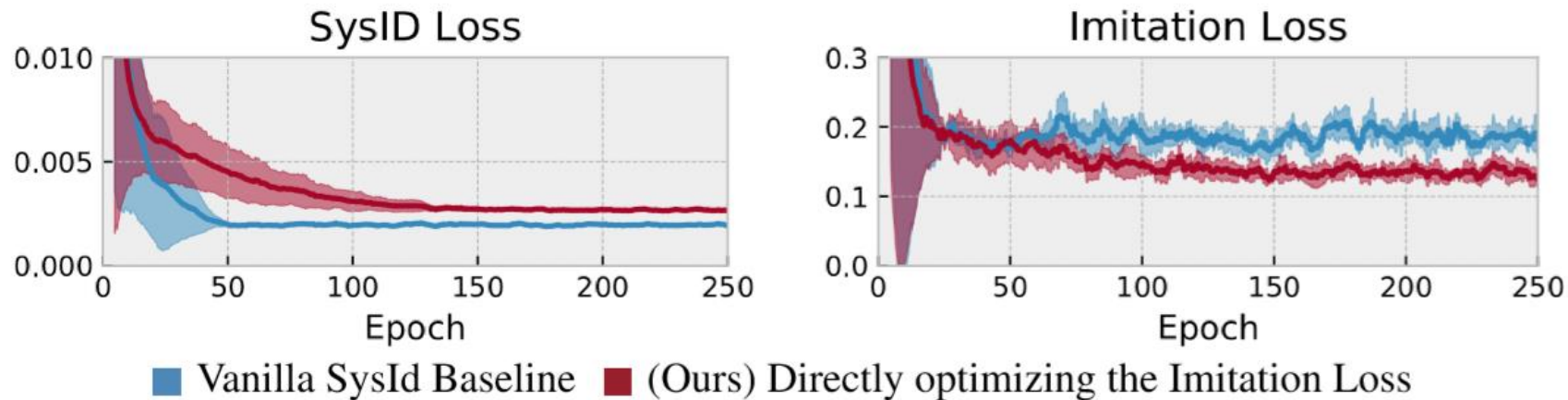


Figure 5: Convergence results in the non-realizable Pendulum task.

# Appendix) Full Derivation of Gradients of the task loss

- Problem

$$\begin{aligned}\tau_{1:T}^* = \operatorname{argmin}_{\tau_{1:T}} & \frac{1}{2} \tau_t^T C_t \tau_t + c_t^T \tau_t \\ \text{s.t. } & x_1 = x_{init} \\ & x_{t+1} = F_t \tau_t + f_t\end{aligned}$$

where  $\tau_{1:T}^* = \{x_t, u_t\}_{1:T}$ .

- Lagrangian is  $L(\tau, \lambda) = \sum_{t=1}^T \left( \frac{1}{2} \tau_t^T C_t \tau_t + c_t^T \tau_t \right) + \sum_{t=0}^{T-1} \lambda_t^T (F_t \tau_t + f_t - x_{t+1})$ 
  - Where  $\lambda$ : dual variable of the equality constraints,  $F_1 = 0, f_1 = x_{init}$
- Differentiating the Lagrangian yields
  - $\nabla_{\tau_t} \mathcal{L}(\tau^*, \lambda^*) = C_t \tau_t^* + c_t + F_t^T \lambda_t^* - \begin{bmatrix} \lambda_{t-1}^* \\ 0 \end{bmatrix} = 0$
  - Thus, the optimal  $\lambda_{1:T}^*$  can be calculated on backward way

# Appendix) Full Derivation of Gradients of the task loss

- Note that solving the LQR is the alternative way to solve the KKT system problem:

$$\overbrace{\begin{bmatrix} \ddots & & & & \\ & C_t & F_t^\top & & \\ & F_t & [-I & 0] & \\ & & \begin{bmatrix} -I \\ 0 \end{bmatrix} & C_{t+1} & F_{t+1}^\top \\ & & & F_{t+1} & \ddots \end{bmatrix}}^K \begin{bmatrix} \vdots \\ \tau_t^* \\ \lambda_t^* \\ \tau_{t+1}^* \\ \lambda_{t+1}^* \\ \vdots \end{bmatrix} = - \begin{bmatrix} \vdots \\ c_t \\ f_t \\ c_{t+1} \\ f_{t+1} \\ \vdots \end{bmatrix}$$

# Appendix) Full Derivation of Gradients of the task loss

- (From Optnet: Differentiable Optimization as a Layer in Neural Network)
- Let me simplify the problem into QP problem

$$\min_z \frac{1}{2} z^T Q z + q^T z$$
$$s. t. A z = b$$

- Lagrangian is  $L(z, v) = \frac{1}{2} z^T Q z + q^T z + v^T (A z - b)$ 
  - You can compute optimal  $z^*$  and  $v^*$
- Here, the KKT conditions are
  - $Q z^* + q + A^T v^* = 0$
  - $A z^* - b = 0$
- Instead of directly differentiating  $z^*$  with respect to each parameter, we take the differentials of these conditions to compute  $\frac{\partial z^*}{\partial \theta}$

# Appendix) Full Derivation of Gradients of the task loss

---

- (From Optnet: Differentiable Optimization as a Layer in Neural Network)
- By taking differential form of these conditions,
  - $dQz^* + Qdz + dq + dA^T v^* + A^T dv = 0$
  - $dAz^* + Adz - db = 0$
  - Eventually, you will get  $\begin{bmatrix} Q & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} dz \\ dv \end{bmatrix} = - \begin{bmatrix} dQz^* + dq + dA^T v^* \\ dAz^* - db \end{bmatrix}$
  - Now you can compute  $\frac{\partial z}{\partial \theta}$  by setting  $db = I$  and others to 0.



# Appendix) Full Derivation of Gradients of the task loss

- (From Optnet: Differentiable Optimization as a Layer in Neural Network)

- To compute  $\frac{\partial l}{\partial \theta} = \frac{\partial l}{\partial z^*} \frac{\partial z^*}{\partial \theta}$ , we use the chain rule. Then

- $\nabla_Q l = \frac{1}{2}(d_z z^T + z d_z^T)$ ,  $\nabla_q l = d_z$ ,  $\nabla_A l = d_v z^T + v d_z^T$ ,  $\nabla_b l = -d_v$  where

- $\begin{bmatrix} d_z \\ d_v \end{bmatrix} = \begin{bmatrix} Q & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} \left(\frac{\partial l}{\partial z^*}\right)^T \\ 0 \end{bmatrix}$

- By changing notation, we can derive

- $\nabla_{C_t} l = \frac{1}{2}(d_{\tau_t}^* z^T + \tau_t^* d_{\tau_t}^{*T})$ ,  $\nabla_{C_t} l = d_{\tau_t}^*$ ,  $\nabla_{x_{init}} l = d_{\lambda_0}^*$

- $\nabla_{F_t} l = d_{\lambda_{t+1}}^* \tau_t^{*T} + \lambda_{t+1}^* d_{\tau_t}^{*T}$ ,  $\nabla_{f_t} l = d_{\lambda_t}^*$

Where  $\begin{bmatrix} \vdots \\ d_{\tau_t}^* \\ d_{\lambda_t}^* \\ \vdots \end{bmatrix} = -K^{-1} \begin{bmatrix} \vdots \\ \nabla_{\tau_t}^* l \\ 0 \\ \vdots \end{bmatrix}$

$$\overbrace{\begin{bmatrix} \ddots & & & & \\ & C_t & F_t^\top & & \\ & F_t & \begin{bmatrix} -I & 0 \end{bmatrix} & & \\ & & \begin{bmatrix} -I \\ 0 \end{bmatrix} & C_{t+1} & F_{t+1}^\top \\ & & & F_{t+1} & \ddots \end{bmatrix}}^K \begin{bmatrix} \vdots \\ \tau_t^* \\ \lambda_t^* \\ \tau_{t+1}^* \\ \lambda_{t+1}^* \\ \vdots \end{bmatrix} = - \begin{bmatrix} \vdots \\ c_t \\ f_t \\ c_{t+1} \\ f_{t+1} \\ \vdots \end{bmatrix}$$